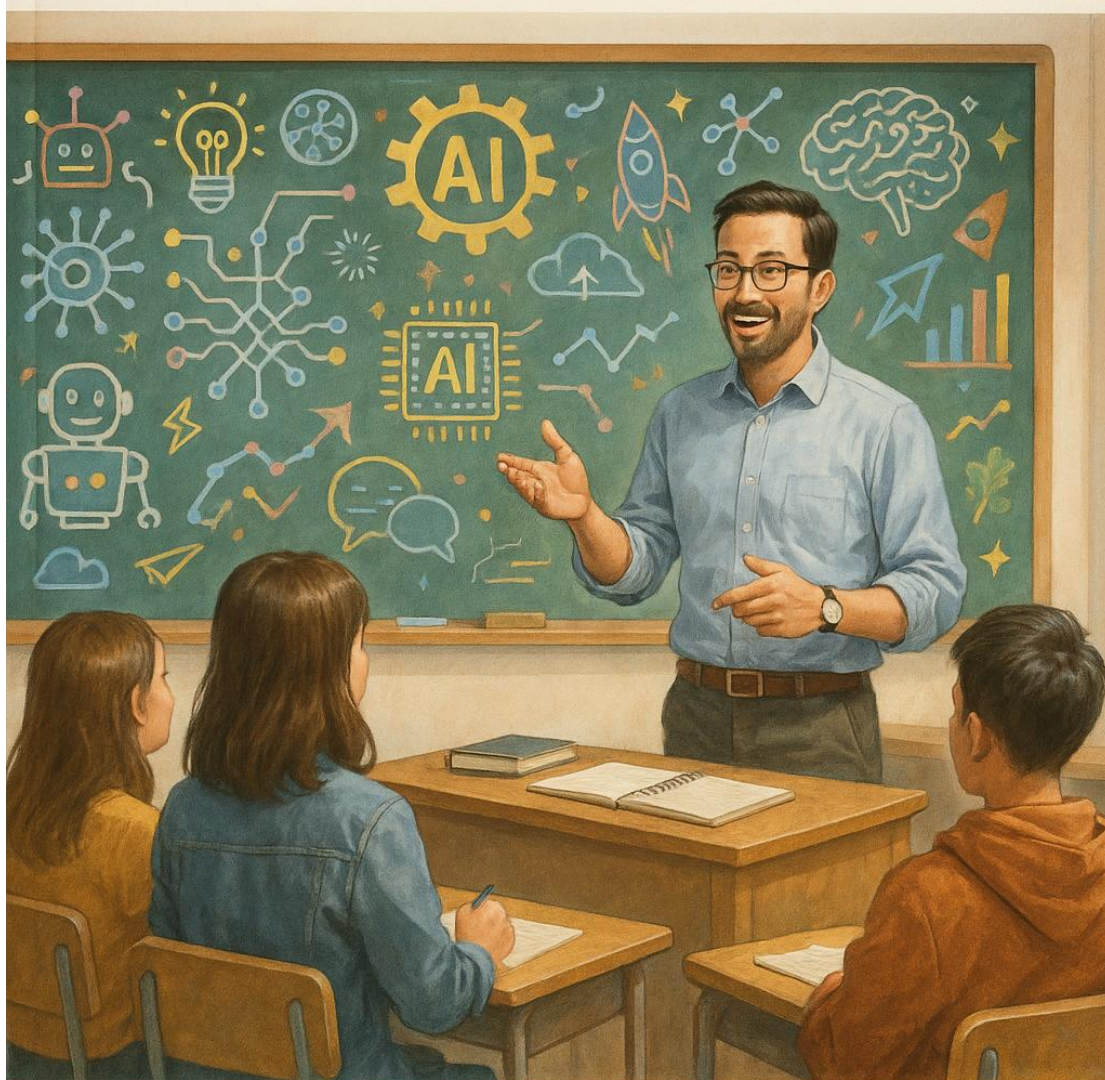


iPAS AI 應用規劃師—中級能力鑑定

L21人工智慧技術 應用與規劃學習筆記



iPAS 輔考小老師 出版

L21 《人工智慧技術應用與規劃》學習筆記 前言	3
L21101 自然語言處理技術與應用	6
一、自然語言處理簡介	6
二、NLP 技術核心分類	6
三、代表應用場景與模型配適	38
四、語言模型與生成式應用	38
L21102 電腦視覺技術與應用	44
一、電腦視覺核心任務分類	44
二、關鍵卷積神經網路架構與演算法演進	48
三、經典卷積神經網路架構與技術演進路徑	52
四、延伸卷積結構與強化模組	54
五、物件檢測與辨識技術細節	57
六、輔助視覺任務與強化技術	61
L21103 生成式 AI 技術與應用	64
一、生成式 AI 概論	64
二、生成式 AI 核心技術	64
三、大型語言模型與生成式人工智慧的關係	71
四、生成式 AI 應用實例與架構設計	73
五、進階話題：RAG、MoE、LLM AGENTS	74
L21104 多模態人工智慧應用	77
一、多模態人工智慧的定義與背景	77
二、核心架構與模型組件（以 MM-LLMs 為例）	77
三、模型訓練階段與優化方式	81
四、典型應用場景與技術實例	83
五、新興技術趨勢	83
L21201 AI 導入評估	87
一、AI 的商業價值與可行性分析	87
二、可行性分析技術	89
三、AI 技術選型與解決方案規劃	91
四、解決方案模組化規劃	94
五、技術選型考量因素	97
L21202 AI 導入規劃	99

一、需求分析 (需求識別與問題建模)	99
二、技術應用方案設計 (模型與演算法選型)	101
三、資源分配 (人力、資安、計算資源與 MLOps)	105
L21203 AI 風險管理	106
一、風險識別技術	106
二、AI 安全性與合規技術	109
三、AI 倫理技術	112
四、負責任 AI 技術	116
五、可信 AI 安全標準與實作框架	119
六、AI 審計與可責任化機制設計	120
L21301 數據準備與模型選擇	123
一、AI 應用情境分析與技術選擇	123
二、數據收集與資料前處理	125
三、特徵工程	128
四、模型比較與選擇	131
五、模型評估指標	133
六、AUTOML 與模型選擇工具	136
七、進階實作與實例分析	139
L21302 AI 技術系統集成與部署	141
一、AI 技術系統開發與部署概論	141
二、系統架構設計	146
三、模型優化	150
四、應用整合與 API 開發	155
五、自動化 CI/CD 與部署管線	160
六、實務應用場景	161
七、延伸技術資源	161
八、系統測試與驗證	163

L21 《人工智慧技術應用與規劃》學習筆記 | 前言

本筆記為 iPAS 輔考小老師 依據 iPAS 應用規劃師中級能力測驗科目《L21 人工智慧技術應用與規劃》所編製，透過 AI 系統進行內容整理與技術精修，旨在協助考生有系統地掌握人工智慧應用發展之關鍵觀念與實作流程。

筆記內容涵蓋 AI 技術發展趨勢、應用規劃架構、導入評估策略等核心議題，並搭配演算法模型與實務案例圖示，協助學習者強化理解與實務連結，建立 AI 導入與應用設計的全貌觀。

本重點筆記內容，不作為保證考取之依據，建議考生仍須搭配科考模擬題庫、反覆練習與整合思維，方能有效應對實際測驗情境，順利通過中級能力認證。

中級能力鑑定科目及授證資格

(一)考試科目：(科目 2 或科目 3 擇一報考)

1. 人工智慧技術應用與規劃
2. 大數據處理分析與應用
3. 機器學習技術與應用

(二)授證資格：

➢ 2 項科目皆達及格標準(70 分)。

A:科目 1+科目 2

B:科目 1+科目 3

科目名稱	主題	單元內容	備註
L21 人工智慧技術應用與規劃	L211 AI 相關技術應用	L21101 自然語言處理技術與應用	人工智慧的核心技術與應用（如機器學習、深度學習、資料處理分析、自然語言處理、電腦視覺、生成式人工智慧等） 如文字、圖像、聲音等
		L21102 電腦視覺技術與應用	
		L21103 生成式 AI 技術與應用	
		L21104 多模態人工智慧應用	
	L212 AI 導入評估規劃	L21201 AI 導入評估	AI 的商業價值與可行性分析 AI 技術選型與解決方案規劃
		L21202 AI 導入規劃	AI 導入規劃流程如需求分析、技術應用方案設計、目標設置、資源分配等
		L21203 AI 風險管理	AI 導入風險與合規管理如風險識別、安全與合規性、AI 倫理、負責任 AI 等
	L213 AI 技術應用與系統部署	L21301 數據準備與模型選擇	AI 技術應用評估與準備（如 AI 應用的場景分析、模型選擇、數據處理與準備等）如數據收集、數據清洗與預處理、特徵工程、不同模型的優缺點等
		L21302 AI 技術系統集成與部署	AI 技術系統開發與部署（如系統架構設計、模型優化、應用整合與 API 開發等）如 AI 技術系統架構設計、模型部署技術、模型的效能監控和更新管理、系統測試與驗證、系統的穩定性和可用性、雲端環境建置等
L22 大數據處理分析與應用	L221 機率統計基礎	L22101 敘述性統計與資料摘要技術	敘述性統計與資料摘要技術
		L22102 機率分佈與資料分佈模型	機率分佈與資料分佈模型
		L22103 假設檢定與統計推論	假設檢定與統計推論
	L222 大數據處理技術	L22201 數據收集與清理	數據基本概念（定義與分類）
		L22202 數據儲存與管理	數據處理的技術與方法
		L22203 數據處理技術與工具	
	L223 大數據分析方法與工具	L22301 統計學在大數據中的應用	數據分析流程與統計方法
		L22302 常見的大數據分析方法	數據分析流程與統計方法
		L22303 數據可視化工具	

科目名稱	主題	單元內容	備註
	L224 大數據在人工智慧之應用	L22401 大數據與機器學習	人工智慧與大數據基本技術概念 大數據在 AI 應用發展案例 大數據隱私安全與合規性
		L22402 大數據在鑑別式 AI 中的應用	
		L22403 大數據在生成式 AI 中的應用	
		L22404 大數據隱私保護、安全與合規	
L23 機器學習技術與應用	L231 機器學習基礎數學	L23101 機率/統計之機器學習基礎應用	機率/統計之機器學習基礎應用
		L23102 線性代數之機器學習基礎應用	線性代數之機器學習基礎應用
		L23103 數值優化技術與方法	數值優化技術與方法
	L232 機器學習與深度學習	L23201 機器學習原理與技術	機器學習與深度學習基本原理與框架
		L23202 常見機器學習演算法	常見的演算法與模型
		L23203 深度學習原理與框架	機器學習與深度學習應用案例
	L233 機器學習建模與參數調校	L23301 數據準備與特徵工程	機器學習數據處理流程（如數據特性、數據清洗與前處理、特徵工程等）
		L23302 模型選擇與架構設計	機器學習模型與演算法應用
		L23303 模型訓練、評估與驗證	模型訓練、評估與優化
		L23304 模型調整與優化	
	L234 機器學習治理	L23401 數據隱私、安全與合規	機器學習治理概述與相關規範 數據隱私與安全性管理
		L23402 演算法偏見與公平性	模型訓練與偏差控制 AI 倫理與負責任 AI

L21101 自然語言處理技術與應用

一、自然語言處理簡介

自然語言處理 (Natural Language Processing, NLP) 是人工智慧與計算語言學的交叉領域，旨在讓機器能理解、解釋、生成和回應人類語言。核心應用包含語音助理 (如 Siri)、翻譯系統、對話機器人、文字摘要、自動寫作等。

(一) NLP 的發展脈絡與演進

- 1950s–1980s：以語法規則為主 (Chomsky 結構語法)
- 1990s–2000s：引入統計模型 (HMM、n-gram、CRF)
- 2010s–Now：深度學習主導 (RNN、LSTM、Transformer)

二、NLP 技術核心分類

下列為自然語言處理的五大核心技術分類，每一類技術皆有對應的目標、方法與應用場景：

技術核心	子分類技術	使用演算法 / 模型	應用說明
語意理解	意圖辨識、語義角色標註	BERT、GPT、Semantic Parsing	聊天機器人、QA 系統、虛擬助理
結構分析	語法解析、依存分析、命名實體辨識	CRF、BiLSTM-CRF、SpaCy	文本抽取、文法檢查
語境建模	上下文學習、注意力機制	Transformer、Self-Attention	對話、生成式 AI 應用
表示學習	詞向量、句向量、語意嵌入	Word2Vec、FastText、BERT	搜尋引擎、推薦系統
語料驅動	分詞、詞性標註、語料前處理	jieba、HanLP、HMM、LTP	中文分詞、文本淨化

■ 自然語言處理核心技術說明

(一) 語意理解 (Semantic Understanding)

◆ 1. 意圖識別 (Intent Detection)

● 技術目標與定義

意圖識別是將使用者輸入 (通常為一句話或一段對話) 分類至預定的語意類別，這些類別代表使用者的「行動需求」，如「查天氣」「訂票」「取消預約」等，是對話系統的核心理解模組。

● 常見演算法與模型展開

- (1) TF-IDF + SVM / Logistic Regression：傳統特徵工程基礎模型，適合意圖類別固定的小規模系統。

- (2) Naive Bayes：統計機率基準模型，對少量樣本有效，但無法捕捉語境。
- (3) CNN for Text Classification (Kim 2014)：使用詞嵌入後的詞向量做 1D 卷積，擷取區域關鍵特徵。
- (4) RNN / GRU / LSTM：擅長捕捉語序依賴關係，適合語句中後段出現關鍵資訊的情況。
- (5) BiLSTM + Attention：將序列雙向編碼後結合注意力權重，突出對意圖判斷重要的詞語。
- (6) BERT / ALBERT / DistilBERT：Transformer-based 預訓練模型，以 [CLS] token 表示整句語意進行 fine-tune。
- (7) ELECTRA / RoBERTa：訓練速度更快、語意精度更高的替代 BERT 模型。
- (8) DIET classifier (from Rasa)：結合 Transformer + CRF 模型，可處理多任務意圖與槽位辨識。

● 進階技術延伸

- (1) 多任務學習 (Multi-task Learning)：結合意圖與命名實體辨識共同訓練，提升泛化能力。
- (2) 小樣本意圖識別 (Few-shot Intent Classification)：使用 ProtoNet、Matching Network、TARS，處理新增意圖樣本極少的場景。
- (3) 意圖偵測開放域 (Zero-shot Intent Detection)：使用自然語言描述意圖類別 (如 GPT-3 Prompt)，在新任務上不需微調也能分類。

● 實務應用案例

- (1) 語音助理：分辨「開啟藍牙」、「調高音量」、「幫我訂咖啡」等語意。
- (2) 智慧客服：將客戶訊息分類為「查詢進度」、「申訴」、「取消訂單」等對應工單流程。
- (3) 金融聊天機器人：精確判斷「查詢股價」vs「我要投保」等不同意圖進行轉接或自動回覆。

◆ 2. 語義角色標註 (Semantic Role Labeling, SRL)

● 技術目標與定義

語義角色標註旨在分析句子中各語詞在事件中的語意角色，如「動作者 (Agent)」、「受詞 (Patient)」、「地點 (Location)」、「工具 (Instrument)」等。這有助於機器理解「誰對誰做了什麼」。

● 主要演算法與技術細節

統計與規則法：

基於 FrameNet / PropBank 語料庫的模板匹配，以 MaxEnt / CRF 建模語義角色機率。

序列標註架構：

- (1) BiLSTM + CRF：將句子視為序列，對每一詞進行 BIO 格式語義標記。
- (2) Tree-LSTM：在語法樹結構上擴展語意計算，效果佳於線性結構。

預訓練語境模型：

- (1) BERT + CRF：在 token 表徵上加強上下文依存後進行角色判別。
- (2) SpanBERT：設計專門處理 phrase span 之預訓練語境嵌入。
- (3) AllenNLP SRL 模型 (ELMo-based)：將 SRL 當作 span 預測任務來處理，效果穩定。

最新架構：

- (1) GloRE (Global Roles Embedding)：使用 global role consistency 訓練全語料共通語義角色。
- (2) UniSRL (Unified Model for SRL)：多語言、多任務統一框架。

● 進階處理方式

Argument pruning：提前刪除無關詞減少模型計算負擔。

SRL + Syntactic parsing：結合語法分析結果，提升跨句語義一致性。

● 實務應用

- (1) 法律或金融條文解析：抽取「主體-行為-物件」結構。
- (2) 多輪對話系統：輔助歧義句斷義。
- (3) 知識圖譜構建：從句子中抽取三元組 (主詞、動詞、受詞) 成為節點與邊。

◆ 3. 自然語言推理 (Natural Language Inference, NLI)

● 技術目標與定義

給定兩段句子 (前提 premise + 假設 hypothesis)，模型需判斷其語意關係：蘊含 (Entailment)、矛盾 (Contradiction) 或中立 (Neutral)。

● 代表性模型與進化歷程

傳統序列模型：

- (1) InferSent (BiLSTM + Max-pooling，固定向量比對)。
- (2) Decomposable Attention Model (DA) 簡化跨句注意力。
- (3) ESIM (Enhanced Sequential Inference Model)：加入 intra-句 attention、BiLSTM 處理語意交互。

預訓練轉換：

- (1) BERT、RoBERTa、ALUM、XLNet (將兩句拼接作為輸入)。
- (2) ELECTRA (基於 discriminative 預訓練)。

整合生成架構：

T5 / BART：將三類輸出作為生成任務處理，如「The relation is: contradiction.」

多語種與知識融合：

- (1) XNLI：支援 15 國語言的跨語推理資料集與模型。
- (2) K-BERT：將知識圖譜嵌入語義推理架構，結合實體資訊。

● 技術擴展方向

- (1) Fine-grained NLI：將語意關係分類細化為 more-than-3-classes。
- (2) 句法+語意共模 (Syntax-aware NLI)：融合 Dependency Tree。
- (3) 弱監督訓練：基於對比語料生成虛擬推理樣本進行訓練。

● 實務應用

- (1) 判斷用戶查詢是否與知識庫文句存在蘊涵。
- (2) 訓練 AI 判斷法律條文與事實是否一致。
- (3) 自動摘要的語意一致性檢查。

◆ 4. 詞義消歧 (Word Sense Disambiguation, WSD)

● 技術目標與定義

多義詞 (如：bank) 在不同語境下代表不同語意，詞義消歧即為判斷特定語境下應選擇的正確詞義。

● 演算法與模型說明

知識驅動 (Knowledge-based)：

- (1) Lesk (1986)：找出詞典定義中與上下文重合度最高的詞義。
- (2) Adapted Lesk：延伸考慮詞根與同義詞網 (WordNet)。

統計/監督模型：

- (1) Naive Bayes、SVM、kNN：基於上下文詞彙統計共現關係。
- (2) Decision Trees：建構特徵與詞義對應規則。

向量化嵌入方法：

- (1) Word2Sense、Sense2Vec：將不同詞義對應不同向量。

(2) Gloss Embedding：將定義文句向量化作為分類依據。

預訓練模型：

- (1) GlossBERT：將「上下文句 + 詞義定義句」拼接進 BERT 比較語意相似度。
- (2) EWISER：融合 WordNet + BERT + edge attention，精細建模定義間的邏輯聯結。

雙階段模型：

- (1) 第一階段：粗略詞義預選（如規則、共現）。
- (2) 第二階段：語境向量選擇精確詞義。

● 應用情境

- (1) 自動機器翻譯（MT）：如「watch」是手錶或觀看？
- (2) 上下文補全 / 語音轉文字糾錯（ASR / TTS）。
- (3) 教育語料與語意檢索系統精度強化。

◆ 5. 語義相似度判斷（Semantic Textual Similarity, STS）

● 技術目標與定義

計算兩個句子或段落在語意上的「相似程度」，數值可為連續或分類型，常用於相似問題搜尋、文件推薦等應用。

● 演算法與模型種類豐富

傳統方法：

- (1) TF-IDF + cosine similarity：對詞頻向量比對。
- (2) WordNet 距離度量：基於詞義樹結構距離計算。

平均詞向量模型：

Avg Word2Vec、TF-IDF weighted Avg Embedding。

神經結構：

- (1) Siamese LSTM / GRU：兩端句子輸入共享模型產生嵌入，比對 cosine 相似度。
- (2) Manhattan LSTM（MaLSTM）：使用 $L1 \text{ distance} + \exp(-x)$ 。

預訓練句子模型：

- (1) USE（Universal Sentence Encoder）。
- (2) InferSent（使用 SNLI 訓練的句子向量）。

BERT 擴展模型：

- (1) SBERT：用 siamese BERT 架構處理大規模檢索。
- (2) SimCSE：self-supervised 對比學習方式精煉向量。
- (3) ST5：將相似度視為 seq2seq 生成問題。

多段結構建模：

ColBERT：late interaction 檢索框架，兼具速度與語意精度。

● 應用範例

- (1) 智慧客服 FAQ 查詢匹配。
- (2) 搜尋引擎語意增強。
- (3) 文件分類、去重、推薦排序。
- (4) 內容創作抄襲檢測或內容聚合。

(二) 結構分析 (Syntactic & Structural Analysis)

◆ 1. 詞性標註 (Part-of-Speech Tagging, POS Tagging)

● 技術目的與定義

詞性標註是將句子中每個詞標上語法類別 (如名詞、動詞、形容詞等)，為後續語法分析、語意理解、實體辨識等任務提供結構性資訊。

● 常見演算法與模型

基於規則的方法 (Rule-based Tagger)：

使用上下文依據 + 遷移規則 (如「to」後通常接原形動詞)，最早如 Brill tagger，需人工設計大量語法規則

統計式標註器：

- (1) HMM (Hidden Markov Model)：以詞性為隱藏狀態，透過轉移機率與觀察機率找出最可能的詞性序列 (Viterbi 演算法推斷)。
- (2) MEMM (Maximum Entropy Markov Model)：類似 HMM，但允許更多上下文特徵輸入，並且基於最大熵分類器進行決策。
- (3) CRF (Conditional Random Field)：條件機率模型，建模詞性標籤間相依性，支援全序列全局最優解。

深度學習方法

- (1) BiLSTM：使用雙向 LSTM 編碼上下文，提升對長距依賴的處理能力。
- (2) BiLSTM-CRF：結合 BiLSTM 的語境表徵與 CRF 的序列優化策略。
- (3) CharCNN + BiLSTM：加入字符層資訊，處理詞根變化與新詞推斷。

預訓練語境嵌入模型

- (1) Flair Embeddings：基於 character-level LSTM 的語境表示，適用多語 POS tagging。
- (2) BERT + Linear Layer：針對每個 token 微調後接上 softmax 預測 POS 標籤。
- (3) XLM-R + CRF：適用於跨語言的詞性分析。

● 應用範例

- (1) 對中文分詞進行詞類劃分 (如區分「銀行 (名詞)」與「銀行 (動詞/機構名)」)。
- (2) 作為命名實體識別、依存語法分析前置模組。
- (3) 自動校稿、語法教學輔助工具的核心分析引擎。

◆ 2. 命名實體辨識 (Named Entity Recognition, NER)

● 技術目標與定義

命名實體辨識是將句子中的實體片段辨識出來並標記其類型，例如：人名、地名、機構名、日期、金額等，是資訊萃取、知識圖譜建構的關鍵模組。

● 使用的演算法與模型

傳統方法

- (1) HMM / CRF / MaxEnt：使用詞、詞性、上下文特徵與位置進行序列標註。
- (2) 規則+詞典法：依賴大量人工定義的關鍵詞與正則模式，適合結構性高資料 (如履歷或報表)。

深度學習模型

- (1) BiLSTM + CRF：將詞的語境向量輸入 CRF 進行序列標註，是近年最主流方法之一。
- (2) CharCNN / CharBiLSTM + Word Embedding：融合字形與詞語層級資訊，強化稀有詞與拼寫變形處理。
- (3) Lattice-LSTM (中文專用)：將詞典詞圖資訊融合進 LSTM 結構，以更精確處理中文詞彙邊界問題。

Transformer-based 架構

- (1) BERT + CRF：使用 BERT 生成 contextual embedding，再透過 CRF 預測實體標籤。
- (2) LUKE (Language Understanding with Knowledge-based Embeddings)：強化實體與詞語關聯性，針對實體特別設計預訓練流程。
- (3) GlobalPointer / W2NER：支援實體重疊、多重標記的 NER 模型，解決傳統 BIO 標註框架限制。

多語言/多任務訓練

- (1) 使用 XLM-R、mBERT 處理多語種 NER。
- (2) 將 NER 與 POS、chunking、slot filling 結合做 joint learning。

● 應用場景

- (1) 金融：財經新聞中辨識公司、事件、貨幣金額
- (2) 醫療：病例與醫囑中抽取疾病、藥品、手術名稱
- (3) 法律：契約書、判決書中抽取當事人、法條條目
- (4) 社群與評論分析：辨識品牌、產品、地點資訊等實體

◆ 3. 依存分析 (Dependency Parsing)

● 技術目標與定義

依存分析是將句子解析成詞與詞之間的語法依賴關係，建立以動詞為核心的語法結構樹，例如主詞依賴、受詞依賴、修飾依賴等。

● 常見演算法與技術架構

基於轉移系統 (Transition-based Parsing)、模擬堆疊操作 (如 Arc-Standard、Arc-Eager) 來構建語法依賴結構。

經典工具：MaltParser、spaCy Dependency Parser

結合神經網路：Stack-LSTM (使用 LSTM 控制器替代傳統堆疊)

- (1) 基於圖模型 (Graph-based Parsing)、將句子轉為圖，使用最長 (或最大權重) 生成樹演算法建構依存結構。
- (2) Biaffine Parser (Dozat & Manning, 2017)：將每個詞轉換為 query / key / value，預測詞間依存邊，並使用 Biaffine 層計算邊的分數
- (3) Deep Biaffine + ELMo / BERT：結合 contextual embedding 增強結構泛化能力

預訓練模型導入

使用 BERT 對輸入語句預先編碼，再將 token vector 接入 Graph Parser。

專用架構如：Hit-SCIR、UDify (支援多語 Universal Dependencies)。

● 應用實例

QA 系統中理解提問的語法結構與焦點

對話系統斷句與核心主詞、動詞識別

句子語法檢查、風格分析、學術文章句構辨識

◆ 4. 句法分析 (Constituency Parsing)

● 技術目標與定義

句法分析 (成份分析) 將句子劃分成語法片語，如名詞片語 (NP)、動詞片語 (VP)、介系詞片語 (PP) 等，構建樹狀結構展現語句層次關係。

● 演算法與技術模型

統計文法分析

- (1) PCFG (Probabilistic Context-Free Grammar)：對語法規則加入機率，透過 CKY 演算法 (或 Earley Parsing) 尋找最可能的語法樹。
- (2) 使用 Penn Treebank 進行 supervised 機率估計。
- (3) Neural Parser (神經網路模型)。
- (4) Span-based Parsing (Stern et al. 2017)：將句子中所有 span 作為候選片語，並使用神經網路判斷其成分類別與層次。
- (5) Chart Parser + BiLSTM Encoder：為每個 span 建立 embedding，再進行結構組裝。
- (6) Berkeley Neural Parser：使用 BERT 生成詞語嵌入，搭配 span scoring 層預測成份結構。

預訓練語言模型應用

- (1) 使用 BERT、RoBERTa 作為 encoder，支援跨語 / 零樣本解析。
- (2) 結合 CRF 標註每個 token 的 chunk tag 再轉換成成份結構。

結構解碼優化

使用 Chart pruning、Label scoring、Boundary refinement 減少不合理結構。

● 應用情境

- (1) 學習系統句子解析圖解 (語言學輔助工具)。
- (2) 自動句子複述、風格改寫 (語法結構調整)。
- (3) 分析寫作風格句式結構偏好、摘要生成句構重建。

(三) 語境建模 (Contextual Modeling)

◆ 1. 上下文建模 (Contextual Encoding)

● 技術目標與定義

上下文建模旨在為每個詞或子詞建立語境相關的表徵，使模型理解詞語的真正含義會根據上下文改變。

例如：「bank」在 “river bank” 中為地形，在 “go to the bank” 中則為金融機構。

● 使用的模型與演算法

傳統方法：

- (1) Bag-of-Words (BOW)：不考慮上下文，表示能力極為有限。
- (2) N-gram 模型：可捕捉短期語序依賴，但會面臨資料稀疏與維度爆炸問題。

遞迴神經網路模型：

- (1) RNN (Recurrent Neural Network)：理論上可處理長序列，但面臨梯度消失問題。
- (2) GRU (Gated Recurrent Unit)：改良 RNN 結構，引入更新門與重置門，更穩定地學習長依賴。
- (3) LSTM (Long Short-Term Memory)：具記憶單元與三個門控機制 (輸入、遺忘、輸出)，廣泛應用於文本理解。
- (4) BiLSTM (Bidirectional LSTM)：同時從左至右與右至左捕捉前後語境。

注意力與多層語境建模：

- (1) LSTM + Attention：結合語義注意力，將關鍵詞的權重強化。
- (2) Hierarchical Attention Network (HAN)：處理長文檔時，分別建模詞級與句級語境。

預訓練語言模型：

- (1) ELMo (Embeddings from Language Models)：使用兩層 LSTM，從不同語境產生動態詞向量。
- (2) BERT：基於 Transformer 的雙向編碼器，輸入整個句子，學習 masked word 的語境表示。

● 應用實例

- (1) 情感分析：同一個詞在不同上下文表現出不同情緒 (如 “sick” 在體育領域常為正向)。
- (2) 對話系統上下文追蹤：判斷用戶前後對話語意一致性。
- (3) 語義分類：結合上下文特徵辨識虛詞、修辭語、隱含語義。

◆ 2. 注意力機制 (Attention Mechanism)

● 技術目標與定義

注意力機制模仿人類專注於輸入資訊中的關鍵部分，讓模型學會對重要字詞賦予較高權重，尤其在處理長文本時能解決信息稀釋的問題。

● 發展與模型結構

- (1) Soft Attention (Bahdanau Attention) : 動態計算目標詞與來源詞的相似度分數，用以加權 sum 資訊。用於 Seq2Seq 模型。
- (2) Luong Attention : 更簡潔的 dot-product 或 general attention，應用於機器翻譯。
- (3) Self-Attention (自注意力) : 計算序列中每個詞對其餘詞的重要性，是 Transformer 架構的核心。
- (4) Scaled Dot-Product Attention : 優化計算效率與穩定性，為 Transformer 中使用的標準注意力形式。
- (5) Multi-Head Attention : 透過多組注意力頭學習不同子空間的語義關聯，擴大語境理解多樣性。
- (6) Sparse Attention / Longformer : 對長文本進行局部化注意力 (如 window-based、dilated)，避免注意力矩陣爆炸。

● 技術衍伸與強化模型

- (1) Transformer (Vaswani et al. 2017) : 完全基於 self-attention 的架構，取代 RNN。
- (2) Transformer-XL : 引入 recurrence 機制以解決長距上下文建模限制。
- (3) Performer / Linformer : 將 self-attention 降維成近似計算，支援百萬級字元序列。
- (4) Gated Attention Network : 結合注意力與門控單元，使模型可動態控制資訊流通。

● 應用實例

- (1) 機器翻譯：翻譯每個詞時只聚焦於對應語意片段。
- (2) 自動摘要：將關鍵句子提取為摘要。
- (3) 文本分類與問答：聚焦於問題與答案對應詞之間的匹配強度。

◆ 3. 語境預訓練 (Contextual Pretraining)

● 技術目標與定義

語境預訓練透過大規模無監督語料學習語言知識，再將學到的語言模型轉移至下游任務 (如分類、問答、生成)，提升泛化能力與精度。

● 模型類型與核心架構

Masked Language Model (MLM) :

- (1) BERT : 隨機遮蔽輸入的 15% 詞彙，讓模型預測被遮蔽的詞。
- (2) ERNIE (百度) : 加入知識圖譜實體遮蔽策略。

- (3) SpanBERT：遮蔽整段片語，提升對語義單元的建模能力。

Autoregressive Language Model (左到右):

- (1) GPT、GPT-2、GPT-3：每次生成一個詞，預測下一個詞。
- (2) CTRL：加入控制碼 (control tokens) 以引導生成內容風格與主題。

Permutation-based :

- (1) XLNet：以所有詞序的排列組合作為預測目標，結合 BERT + GPT 優點。
- (2) Encoder-Decoder 雙塔預訓練：
- (3) T5 (Text-to-Text Transfer Transformer)：將所有任務格式轉為文字輸入 → 文字輸出。
- (4) BART (Bidirectional Auto-Regressive Transformer)：同時支援理解與生成，遮蔽後再重建句子。

● 微調方式與技術延伸

- (1) Fine-tuning：在下游任務上進行端到端訓練，需適當學習率與資料擴增。
- (2) Prompt-tuning：設計自然語言提示引導模型行為，常見於 GPT 類模型。
- (3) Adapter tuning：凍結主模型，僅訓練小型 adapter 模組，提高參數效率。
- (4) Instruction tuning：以任務說明方式對模型進行提示與任務轉換。

● 應用場景

- (1) 自然語言理解任務：文本分類、意圖識別、命名實體識別。
- (2) 自然語言生成任務：摘要、對話、故事創作。
- (3) 代碼生成與語音轉文字：Codex、Whisper 等也源於語境預訓練技術。
- (4) 多語任務：mBERT、XLM-R 支援跨語言的遷移學習。

◆ 4. 雙向建模 (Bidirectional Modeling)

● 技術目標與定義

雙向建模是語境建模的重要強化方式，透過前向與後向同時考慮語意，有效改善單向語言模型只看到部分上下文的限制，適用於理解與預測任務。

● 代表性模型與建構方式

- (1) BiRNN / BiLSTM：分別從左到右與右到左建立兩組隱狀態，再串接或加權合併。
- (2) BiGRU：較簡潔但仍保留雙向序列資訊，參數量少於 BiLSTM。
- (3) BERT：基於 Transformer encoder 雙向架構，全局 self-attention 可同時關注上下文。
- (4) ALBERT：參數共享版本的 BERT，維持雙向能力下降低參數量，訓練效率提升。
- (5) DeBERTa：提出解耦式位置編碼與相對注意力方式，進一步強化雙向語意結構。

- 技術強化與變體

- (1) BiAttentive Classification Network (BCN) : 同時比較雙向表示與 attention 。
- (2) BiLSTM + CRF : 將雙向表徵與序列標記優化結合，應用於 NER、POS 等任務。
- (3) Dense Bi-directional Transformer : 堆疊多層 Bi-Transformer，增強長語句學習能力。

- 應用實例

- (1) 情感分析：全面分析句首與句尾的情緒語詞對整體語意的影響。
- (2) 自然語言推理 (NLI) : 理解兩句話之間的雙向邏輯關係。
- (3) 語意角色標註：同時參考角色出現前後的語法結構進行分類。

(四) 表示學習 (Representation Learning)

◆ 1. 詞嵌入 (Word Embedding)

- 技術目標與定義

詞嵌入技術旨在將每個「詞」轉換為低維度實數向量，使詞與詞之間的語義相似性可透過向量空間表示與計算 (如餘弦相似度)，為 NLP 模型提供語義敏感的輸入。

- 使用的演算法與模型

- (1) One-hot Encoding (傳統基礎) : 每個詞分配唯一 index，轉換為稀疏向量，缺乏語義資訊。
- (2) Word2Vec (Mikolov et al., 2013)、CBOW (Continuous Bag of Words) : 用上下文詞預測中心詞。
- (3) Skip-Gram : 用中心詞預測上下文詞 (擅長捕捉低頻詞語關係)。
- (4) GloVe (Global Vectors for Word Representation, 2014) : 利用整體語料中詞共現矩陣進行矩陣分解，將詞與上下文同時嵌入向量空間。
- (5) FastText (Facebook AI) : 將詞分解為多個子詞 (subword n-grams)，對低資源語言與新詞 (out-of-vocabulary) 處理能力更強。
- (6) Swivel : 結合共現矩陣與詞上下文分布的預測目標，以雙向詞關聯性建模。
- (7) Retrofitting (Faruqui et al.) : 融合詞向量與知識圖譜 (如 WordNet) 資訊進行後處理。

- 應用場景

- (1) 文本分類：將詞嵌入後加總或平均形成句子向量。
- (2) 情緒分析、主題分類、新聞分類、短文本匹配。
- (3) 資料視覺化：使用 t-SNE、UMAP 可視化詞嵌入空間理解語義聚類。

◆ 2. 句子嵌入 (Sentence Embedding)

- 技術目標與定義

將整個句子編碼為固定維度的向量，保留語序與語義信息，使不同語意相似的句子在向量

空間中距離接近。比起單詞嵌入，句子嵌入強調更高層次語意理解。

● 使用的演算法與模型

平均詞向量法：將詞嵌入平均或加權平均（如 TF-IDF weighted）形成句向量，效果有限。

基於神經網路：

- (1) LSTM Encoder / GRU Encoder：將句子序列輸入 RNN，取最後 hidden state 作為整句表示。
- (2) BiLSTM + Max Pooling (InferSent)：雙向語境建模 + 全句 pooling，以 SNLI 語料預訓練。
- (3) HAN (Hierarchical Attention Network)：對句中每個詞與句群使用不同層 attention。

預訓練句子向量模型：

- (1) Universal Sentence Encoder (USE)：Google 提出，基於 Transformer 或 Deep Averaging Network 結構，支援多語言。
- (2) SBERT (Sentence-BERT)：在 BERT 上引入 siamese 結構，支援大規模句對相似度運算與搜尋。
- (3) SimCSE：使用對比學習 (Contrastive Learning) 微調 BERT，使正向句對靠近，負樣本拉遠。
- (4) T5、BART 句嵌入提取：Encoder 或 Encoder-Decoder 的中間表示用作句向量（如 avg hidden state）。

● 應用場景

- (1) 相似句檢索與 FAQ 問答匹配（句向量 + 向量近鄰檢索）。
- (2) 文本聚類與聚合。
- (3) 社群留言、評論、標題摘要語意比對。
- (4) 語句層級的風格轉換與語意平移（如翻譯風格比較）。

◆ 3. 上下文嵌入 (Contextual Embedding)

● 技術目標與定義

上下文嵌入為詞或子詞在不同上下文中生成不同的向量表示，解決傳統詞嵌入固定不變、無法區分多義詞（如「apple 是水果或品牌」）的問題。

● 使用的演算法與模型

- (1) ELMo (Embeddings from Language Models, 2018)：基於 character-level CNN + 2-layer BiLSTM，從左至右與右至左學習語境，生成詞在特定上下文的表徵。

- (2) 多層輸出可視任務取不同層次特徵：BERT (Bidirectional Encoder Representations from Transformers, 2019)
- (3) 利用多層 bidirectional Transformer 編碼器，在句子中為每個詞生成 contextualized vector。
- (4) 每個詞嵌入受到整句其他詞的影響：RoBERTa、ALBERT、ELECTRA：BERT 的訓練方式與架構變體，精度與效率進一步提升。
- (5) FLAIR：使用 character-level BiLSTM 做詞向量編碼，支援疊加多語嵌入。
- (6) GPT 系列 (GPT-2、GPT-3)：雖為單向自回歸模型，但輸出仍為語境相關嵌入，適合生成類任務。
- (7) Adapter Fusion：將多個語境嵌入 (如 sentiment embedding、domain embedding) 動態融合。

● 技術特點

- (1) 可應用於任何下游任務：文本分類、NER、問答、機器翻譯等。
- (2) 向量維度高但語意豐富，適合精細分類與語義理解。

● 應用場景

- (1) NER、情緒分析、關鍵句偵測等 token-level 任務。
- (2) 文本相似度評估 (使用嵌入距離、餘弦相似度)。
- (3) 上下文補全、文句排序 (如摘要與自動改寫)。
- (4) 多語/跨語學習與表示對齊 (mBERT、XLM-R)。

◆ 4. 概念嵌入 (Concept Embedding / Knowledge Embedding)

● 技術目標與定義

概念嵌入主要將知識圖譜中的實體 (entity)、關係 (relation) 與屬性轉化為可計算向量，將符號邏輯與分布式語意嵌入統一表示，用於推論與知識檢索。

● 使用的演算法與模型

基於三元組 (head, relation, tail) 表示

TransE：將 $\text{head} + \text{relation} \approx \text{tail}$ 表示為空間平移操作。

- (1) TransH / TransR：引入關係特有空間與投影方式，處理一對多、多對一結構。
- (2) RotatE：將嵌入空間從加法平移轉為複數旋轉操作 (應用於知識循環推理)。
- (3) ComplEx / DistMult：建模對稱、反對稱與復雜關係。

Graph Neural Network 模型

- (1) R-GCN (Relational Graph Convolutional Network)：對不同關係型態進行 GNN

訊息傳播。

- (2) CompGCN：融合 entity 與 relation 嵌入進行關係感知擴散。
- (3) GraphSAGE / GAT：可應用於知識子圖的結構學習。

預訓練結合模型

- (1) K-BERT：將知識圖譜節點插入句子中一併訓練 BERT，使模型語意更精確。
- (2) ERNIE 2.0 (百度)：融合結構化語義與知識圖譜關係訓練語言模型。
- (3) KEPLER：聯合知識三元組預測與語言建模，學習統一概念向量空間。

● 應用範例

- (1) 知識圖譜補全與連結預測。
- (2) 問答系統的知識查詢與關係推理 (例如：“台北的市長是誰？” → 推理節點關係)。
- (3) 醫學、法律等專業領域的概念圖建構。
- (4) 語意搜尋與推薦系統 (使用 entity linking + 概念嵌入作為內容表示)。

✦ 比較不同詞向量方式：

方法	是否上下文相關	向量長度固定	特點
Word2Vec	否	是	基於詞共現，無語境變化
GloVe	否	是	統計全局共現矩陣
ELMo	是	否	基於雙向 LSTM 輸出各層語意表示
BERT	是	是	深層雙向 Transformer、可做微調

(五) 語料驅動 (Corpus-driven Preprocessing)

◆ 1. 分詞 (Tokenization)

● 技術目標與定義

將一段連續文本切割成「最小語意單元」(Token)，如詞、子詞、標點，是 NLP 的最初步驟。中文分詞尤其關鍵，因語句無自然空格標示。

● 使用的演算法與方法

基於規則的方法：

- (1) 正向最大匹配 (FMM)：從左至右找最大詞典匹配 (容易誤切複合詞)。
- (2) 反向最大匹配 (RMM)：從右至左 (較適合處理後綴詞與助詞)。
- (3) 雙向最大匹配 (BiMM)：綜合 FMM 與 RMM。
- (4) 前綴詞典 + 有向無環圖 (DAG)：構造所有可能詞切分，再使用概率找出最佳路徑。

基於統計的方法：

- (1) 基於機率的 N-gram 模型：以詞共現機率為依據斷詞。
- (2) HMM (Hidden Markov Model)：將每個字視為狀態，學習詞的邊界轉移機率。

基於機器學習的方法：

- (1) CRF (Conditional Random Field)：序列標註模型，對每個字標 B/M/E/S 標籤。
- (2) SVM + 字特徵 (單字、窗口字、詞典標記) 進行分類切分點預測。

基於深度學習的方法：

- (1) BiLSTM + CRF：透過上下文建模與全局最優切分。
- (2) Transformer + CRF：用注意力機制對任意距離的字元建模。
- (3) CNN + BiGRU：處理多層次邊界、詞頭與詞尾特徵。

字節對編碼 (BPE)、SentencePiece (亞詞單位分詞)：

用於訓練大型語言模型，避免 OOV 問題與過度切詞。

● 應用場景

- (1) 中文 NLP 前置處理 (幾乎所有任務需用)。
- (2) 關鍵詞擷取、情感分析中的詞單位抽取。
- (3) 自然語言生成中的詞回推 (反斷詞)。
- (4) 關鍵字搜尋匹配、拼音輸入法分詞。

◆ 2. 文本清洗 (Text Cleaning)

● 技術目標與定義

去除語料中雜訊、無用符號、錯誤資料，使輸入資料乾淨一致，有助於模型專注於語言特徵學習。

● 技術方法展開

HTML/Markdown 清除：

- (1) 使用正則表達式移除標籤 (如 <div>, </p> 等)。
- (2) 使用 HTML parser：如 BeautifulSoup、lxml。

字元過濾：

- (1) 去除非 UTF-8 碼、控制字元 (如 \x00)。
- (2) 過長重複字元壓縮 (如「好 ~ ~ ~ 棒」→「好 ~ 棒」)。

表情符號與特殊標記處理：

- (1) Emoji 替換為語意標籤，如 😞 → <sad_face>。

- (2) 正則萃取 URL、email、phone number、hashtags 作為特徵。

語法與拼寫修正：

- (1) 自動拼字檢查：Norvig spelling corrector、SymSpell。
- (2) 文法改寫：LanguageTool、GPT-based Grammar Checker。

數據標準化：

- (1) 數字、貨幣統一格式 (如 \$1,000 → 1000)。
- (2) 日期正規化 (如 “明天下午” → “2025-04-13 15:00”)。

● 應用場景

- (1) 社群平台語料 (表情/異體字/雜訊多) 處理。
- (2) 多來源爬蟲資料 (結構混亂) 統整清洗。
- (3) 語音轉錄錯字修正與語法補全。
- (4) 智能客服資料庫整理、合規語料過濾。

◆ 3. 停用詞移除 (Stopword Removal)

● 技術目標與定義

移除頻率高但對語意分類與理解無貢獻的詞 (如「是」「的」「而且」)，減少模型學習負擔並提升特徵信噪比。

● 技術與流程細節

停用詞詞典：

- (1) 通用詞典：如 NLTK 中文/英文常用列表。
- (2) 自建領域詞典：針對金融、醫療等領域過濾特有冗詞。

移除策略：

- (1) 完全刪除 (移除該詞及其特徵向量)。
- (2) 替換為 placeholder (如 <stopword>，保留位置但不建模)。

精緻處理：

- (1) 情感分析中需保留感嘆詞與語氣助詞。
- (2) 頻率過高但語意強的詞不應移除 (如 “很好”、“沒問題”)。
- (3) 可搭配 TF-IDF、MI 等特徵選擇方法對停用詞做自動識別。

● 應用場景

- (1) 關鍵詞擷取、主題建模 (如 LDA) 時降噪。

- (2) 文本分類任務前的特徵壓縮。
- (3) 文本向量空間建模 (BOW, TF-IDF) 維度簡化。
- (4) 文件聚類、摘要時提升語義純度。

◆ 4. 詞性分析 (POS Analysis / POS Preprocessing)

● 技術目標與定義

在語料預處理階段對詞進行語法類別標記 (如名詞、動詞、副詞)，便於後續的語法分析、詞義消歧與抽取任務。

● 技術演算法補充

使用 POS Tagging 模型：HMM、CRF、BiLSTM、BERT。

POS + dependency：提供語法指導依存與角色標註。

POS + 分詞聯合模型：

Lattice-LSTM、CRF-LSTM 支援同步分詞與詞性預測。

POS-augmented Embedding：

融入 POS tag 作為詞向量的附加資訊 (如 POS tag one-hot vector concat)。

● 應用場景

- (1) 中文斷詞 disambiguation (如「上課」是動詞還是名詞組？)。
- (2) 結構分析、語義角色標註、句法樹生成。
- (3) 提升句子語法正確性辨識與改寫能力。
- (4) NER 與 WSD 等標註任務輔助特徵。

◆ 5. 命名實體標註 (NER Annotation)

● 技術目標與定義

為訓練模型辨識命名實體，需手動或半自動對語料進行實體類型標記。這是有監督 NER 模型訓練的資料來源基礎。

● 標註方法與格式

BIO/BILOU 格式：

- (1) B (Begin) 表示實體起始詞，I (Inside) 表示內部詞，O 為非實體。
- (2) BILOU 加入 L (Last)、U (Unit) 強化單詞與片語實體結構。

半自動標註：

- (1) 使用預訓練模型輔助標註，再由人工修正。

- (2) 使用規則或字典先過濾候選詞 (dictionary lookup)。

標註工具：

- (1) Doccano：Web UI 支援序列/分類/摘要標註。
- (2) BRAT：支援關係與層次實體標註。
- (3) Label Studio：多任務、團隊協作型標註工具。

實體歧義與多重標籤處理：

- (1) 支援一詞多義 (如「華盛頓」可能是地名或人名)。
- (2) 支援 span overlap、nested NER。

● 應用場景

- (1) 金融、醫療、電商等企業內部語料建立實體資料集。
- (2) 多輪對話資料標註意圖與實體槽位。
- (3) 法規、專利文書實體資料庫建構。
- (4) 知識圖譜與三元組抽取訓練資料準備。

(六) 完整 NLP 處理流程圖 (以中文處理為例)

◆ 1. 語料獲取 (Corpus Acquisition)

● 技術目標與定義

蒐集任務相關的原始語料，包括文本、語音、問答對、標註資料等，作為語言模型或下游任務的訓練素材。

● 技術與方法

網頁爬蟲 (Web Scraping)：

使用 BeautifulSoup、Scrapy 抓取新聞、部落格、討論區資料。
需處理反爬機制與結構清理。

開放語料庫：

中文：搜狗語料庫、人民日報、THUCNews。

英文：Wikipedia dump、Common Crawl、BookCorpus。

API 資料介接：

使用 Twitter API、Reddit API、Telegram Bot 拉取語料對話。

企業內部資料：

客服工單、產品評論、通訊紀錄等需脫敏處理。

- 應用範例

- (1) 訓練 Chatbot 使用的對話資料。
- (2) 構建領域語言模型 (如醫療、金融、法律)。
- (3) 生成式 AI 語料來源準備 (需具授權或開放條款)。

◆ 2. 語料預處理 (Corpus Preprocessing)

- 技術目標與定義

清理語料並轉換成適合模型使用的結構形式，包括：清洗、分詞、詞性標註、去除停用詞等。

- 使用技術與模型 (詳見前一節語料驅動技術)

- (1) 分詞：最大匹配、CRF、BiLSTM、SentencePiece。
- (2) 去雜訊：正則清洗、表情處理、emoji 替代。
- (3) 停用詞過濾：TF-IDF 策略、領域自建詞表。
- (4) 詞性標註：HMM、CRF、BERT-based POS。
- (5) 命名實體初步提取：字典匹配、規則過濾、NER 模型預標註。

- 應用實例

- (1) 使下游模型專注語義特徵，減少訓練成本與雜訊干擾。
- (2) 提升命名實體標註、情緒詞辨識的前置效率。

◆ 3. 特徵工程與表示學習 (Feature Engineering & Representation Learning)

- 技術目標與定義

將預處理後的文字轉換為數值向量，使機器可以進行學習與推論。

- 方法與演算法

傳統特徵：

- (1) TF (Term Frequency)、TF-IDF (Inverse Document Frequency)。
- (2) N-gram 特徵：詞組頻率向量、滑動視窗生成序列。
- (3) 詞共現矩陣：詞語出現關聯建圖。

詞嵌入：

Word2Vec (CBOW, Skip-Gram)、GloVe、FastText。

句子嵌入：

InferSent、SBERT、SimCSE、USE。

上下文嵌入：

ELMo、BERT、RoBERTa、GPT。

特徵選擇方法：

- (1) Mutual Information (MI)、Chi-square (卡方檢定)、Information Gain。
- (2) PCA / LDA (降維與主成分分析)。

● 應用場景

- (1) 將文字特徵輸入至分類器 (如 SVM、XGBoost、RNN)。
- (2) 強化語意相似度計算與群組聚類 (如 KMeans on sentence embedding)。

◆ 4. 模型訓練 (Model Training)

● 技術目標與定義

選擇適合的演算法或神經網路架構，透過監督式/非監督/自監督學習進行模型訓練與優化。

● 模型選擇與訓練策略

傳統機器學習：

Naive Bayes、SVM、Random Forest、Logistic Regression。

序列模型：

RNN、LSTM、GRU、BiLSTM + Attention。

CNN 應用於文本分類：

TextCNN、RCNN、DCNN (動態卷積網路)。

Transformer 架構：

- (1) BERT、RoBERTa、ALBERT：fine-tuning on classification or token classification。
- (2) GPT-2/3：Causal LM for generation。
- (3) T5/BART：seq2seq 任務，支援翻譯、摘要、生成。

多任務學習架構：

MTL-BERT、SpanBERT、ERNIE：同時學習多個語言理解任務。

訓練技巧：

- (1) Batch Normalization、Dropout、防止過擬合。

(2) Early stopping、Cross Validation、Grid Search 調參。

● 應用範例

- (1) 訓練 FAQ 分類模型、對話意圖模型、客服情緒判斷模型。
- (2) 微調 BERT 建立專屬法律/醫療文本理解器。

◆ 5. 模型推論與應用 (Inference & Application)

● 技術目標與定義

將訓練好的模型部署至應用端，針對用戶輸入即時做出語言理解或生成回應。

● 推論流程與工具

模型壓縮與加速：

- (1) Distillation (知識蒸餾)：將大型模型壓縮成小模型如 TinyBERT。
- (2) Quantization：將 float32 降為 int8 加速推理。
- (3) ONNX / TensorRT：轉換為高效部署格式。

API 部署：

- (1) Flask、FastAPI 建立 RESTful NLP 推論介面。
- (2) 使用 Huggingface Transformers Pipeline 快速部署。

前端整合：

- (1) Web App 對話介面 (React, Vue, Streamlit)。
- (2) 接入 Slack、Telegram、LINE 機器人框架。

● 應用場景

- (1) 問答系統、智慧客服回應。
- (2) 機器翻譯、語音文字轉換。
- (3) SEO 內容生成、社群內容摘要、自動寫稿。
- (4) 法務合約智能標註與審核推薦。

◆ 6. 模型評估與優化 (Evaluation & Optimization)

● 技術目標與定義

衡量模型的語言理解或生成能力是否達標，並根據評估結果做迭代優化。

● 評估指標與方法

分類任務：

- (1) Accuracy、Precision、Recall、F1-score。

(2) Confusion Matrix (混淆矩陣) 視覺化。

序列任務：

- (1) Token-level 精準率、完整率 (如 NER、POS)。
- (2) BIO Tagging Accuracy。

生成任務：

- (1) BLEU (翻譯)、ROUGE (摘要)、METEOR、BERTScore。
- (2) 人工評分 (fluency, relevance, diversity)。

檢索類任務：

MRR (Mean Reciprocal Rank)、nDCG (Normalized DCG)。

錯誤分析工具：

- (1) LIME / SHAP：模型可解釋性分析。
- (2) Dataset Slice-Based Evaluation：對少數族群樣本做特定測試。

● 應用實例

- (1) 選擇最佳 Chatbot 回應模型版本。
- (2) 微調模型 loss function 改善少數類別精度。
- (3) 持續性監控 (Model Drift Detection) 追蹤應用效能變化。

(七) 主要技術說明

◆ 1. 中文分詞 (Tokenization)

● 技術目標與定義

分詞是將連續的文字切割為具有語義的最小單位 (tokens)，英文以空格自然斷詞，但中文無明顯斷詞標記，因此中文 NLP 必須進行分詞作業。

● 常見演算法與模型

規則與詞典匹配法：

正向最大匹配 (FMM)、反向最大匹配 (RMM)、雙向匹配 (BiMM)

優點：效率高，適合結構固定文本。

缺點：新詞難識別、歧義無法解決。

統計模型：

- (1) HMM (隱馬可夫模型)：透過字與詞邊界機率計算最可能的切分序列 (Viterbi 演算

法)。

- (2) N-gram 模型：計算連續字詞對的機率，選擇最大機率切分點。

序列標註模型：

- (1) CRF：使用 B/M/E/S (Begin, Middle, End, Single) 標記字元序列，解決歧義與新詞問題。
- (2) BiLSTM + CRF：結合上下文記憶與條件機率的強大結構。

深度學習模型：

- (1) Transformer + CRF：長距離語意建模，並結合序列標註。
- (2) CNN + BiGRU：強化字詞邊界與語境表徵。

子詞單位方法：

BPE (Byte Pair Encoding)、SentencePiece：將詞切分為亞詞，避免 OOV (Out-of-Vocabulary)

● 實務應用

- (1) 預處理任務 (關鍵字抽取、摘要、NER)。
- (2) 文本檢索與搜尋引擎切詞。
- (3) 機器翻譯與語音輸入斷詞基礎。
- (4) 聊天機器人語義單元切分。

◆ 2. 詞性標註 (Part-of-Speech Tagging)

● 技術目標與定義

詞性標註是為句子中的每個詞賦予對應的語法類別標籤 (如名詞、動詞、副詞、形容詞等)，為語法分析、依存分析與語意角色提供結構支撐。

● 演算法與模型技術

統計模型：

- (1) HMM：根據詞與詞性間的轉移機率與觀察機率推斷序列 (Viterbi 演算法)。
- (2) MEMM (Maximum Entropy Markov Models)：可加入更多語境特徵。
- (3) CRF：條件機率建模，解決標註序列間相依性。

深度學習模型：

- (1) BiLSTM：捕捉上下文資訊，改善語境錯誤。
- (2) BiLSTM + CRF：最佳化標註序列，適用於中文與多語言任務。
- (3) CNN + BiLSTM：融合詞形與上下文特徵。

預訓練模型：

- (1) BERT + Linear Layer：針對每個 token 做分類，微調 BERT。
- (2) Flair Embeddings：使用 character-level LSTM 輸出語境表示。

● 實務應用場景

- (1) 語法分析、命名實體辨識。
- (2) 中文語義分析輔助 disambiguation。
- (3) 自然語言生成 (NLG) 中維持語法正確。
- (4) 教學語言工具、拼字與文法校正引擎。

◆ 3. 命名實體識別 (NER)

● 技術目標與定義

NER 用來識別文本中的實體並分類，如人名、地名、機構、日期、金額等，是資訊萃取與知識圖譜建構的重要基礎。

● 使用的模型與方法

- (1) CRF、HMM (傳統序列標註器)。
- (2) BiLSTM + CRF (主流神經模型)。
- (3) Transformer + CRF (BERT、RoBERTa 微調)。
- (4) W2NER、GlobalPointer (嵌套實體處理能力)。
- (5) 支援 BIO、BILOU、Span 格式的標註。

● 應用範例

- (1) 法律與醫療文書實體解析。
- (2) 金融報導的公司、指數抽取。
- (3) 問答系統與知識庫對接。

◆ 4. 依存語法分析 (Dependency Parsing)

● 技術目標與定義

依存分析建立詞與詞間語法依存關係 (如主語、賓語、修飾語)，形成以動詞為核心的語法依存樹。

● 演算法與模型

- (1) Transition-based (MaltParser)：透過堆疊移動建立語法邊。
- (2) Graph-based (Biaffine Parser)：計算詞對依存關係得分，選擇最大生成樹。
- (3) BiLSTM Encoder + MLP + Biaffine Layer (Dozat and Manning 2017)。
- (4) 預訓練模型加持：BERT + Dependency Parser (UDify, Hit-SCIR)。

- 應用場景

- (1) 語法正確性檢查、語言教學。
- (2) 文句斷義與理解 (QA、摘要)。
- (3) 事件抽取的參與者識別與角色關聯建模。

◆ 5. 語義角色標註 (Semantic Role Labeling)

- 技術目標與定義

判斷誰是主詞、受詞、施事、工具等語意角色 (即抽取「誰對誰做了什麼事」的資訊)，建立語句語義結構。

- 技術與模型

- (1) 傳統語義詞庫：FrameNet, PropBank
- (2) BiLSTM + CRF：序列學習與角色判斷
- (3) Span-based SRL：視語義成分為片段並進行分類
- (4) Transformer SRL：SRL-BERT、SpanBERT
- (5) AllenNLP 提供完整 SRL 模組

- 應用場景

- (1) 法律條文與事實摘要斷義
- (2) 結構化問答系統與事件檢索
- (3) 知識圖譜三元組抽取

◆ 6. 關鍵詞與主題提取 (Keyword & Topic Extraction)

- 技術目標與定義

從文本中抽取最能代表主旨的詞語或主題，應用於摘要、索引、搜尋與推薦系統。

- 技術方法

- (1) TF-IDF：統計權重排序。
- (2) TextRank：基於圖排序 (PageRank) 算法建構詞共現圖。
- (3) YAKE、RAKE：基於統計與語法的輕量級關鍵詞演算法。
- (4) LDA (Latent Dirichlet Allocation)：主題建模，將文本分配至多個主題。
- (5) NMF、Top2Vec、BERTopic：主題建模現代替代方案，支援 contextual embedding。

- 應用場景

- (1) 文件快速摘要與自動標籤
- (2) 內容推薦與聚類
- (3) SEO 優化、社群趨勢分析

◆ 7. 文本分類與情緒分析

● 技術目標與定義

將文本分類至固定標籤，如正負情緒、新聞類別、廣告類型。

● 模型技術

- (1) 傳統方法：Naive Bayes、SVM、Random Forest
- (2) 深度學習：TextCNN、RNN、BiLSTM、GRU
- (3) Transformer：BERT fine-tune、RoBERTa、ERNIE
- (4) 多標籤分類：Sigmoid + Binary Crossentropy，處理多類同時標籤情境

● 應用

- (1) 網路評論分析、民調追蹤
- (2) 客戶滿意度回饋分類
- (3) 廣告標籤、多任務預測

(八) 深度學習模型架構

◆ 1. RNN (Recurrent Neural Network)

● 技術核心

解決序列資料建模問題，透過時間步的「狀態遞迴」，記錄過去資訊影響現在決策。
適合短序列處理，如詞元輸入的語意理解。

● 結構細節

每一時間步輸入一個 token，計算 hidden state： $h_t = f(Wx_t + Uh_{t-1})$
無法學習長期依賴（梯度消失或爆炸）

● 延伸技術

- (1) Truncated BPTT：限制反向傳播長度以改善記憶控制。
- (2) Bidirectional RNN：從左至右與右至左同時學習，改善資訊不完整問題。

● 應用

初期語言模型、拼字校正、POS 標註。

◆ 2. LSTM (Long Short-Term Memory)

● 技術核心

對 RNN 改良，引入「記憶單元」控制資訊流通，保留長距離依賴。

- 門控機制與架構

Forget Gate : $f_t = \sigma(W_f \cdot [h_{t-1}, x_t])$

Input Gate : 控制新資訊進入記憶體

Output Gate : 決定輸出值與當前記憶輸出

- 延伸版本

(1) BiLSTM : 雙向上下文。

(2) Peephole LSTM : 加入 cell state 參與控制訊號的生成。

(3) LSTM + Attention : 捕捉重要語義詞。

- 應用

情感分析、NER、摘要生成、語音識別。

◆ 3. GRU (Gated Recurrent Unit)

- 技術核心

LSTM 精簡版本，結構更簡、參數更少、訓練速度更快。

- 結構組成

Reset Gate : 重設前一時間步資訊

Update Gate : 合併前狀態與新輸入

無明確 cell state，直接融合至 hidden state。

- 應用

資源受限環境下的序列建模任務。

◆ 4. TextCNN / DPCNN / RCNN (CNN for NLP)

- 技術核心

從字/詞序列中提取局部區域特徵，適合分類、匹配任務。

- 模型細節

(1) TextCNN : 使用多尺寸卷積核擷取 n-gram 特徵。

(2) RCNN : 結合 RNN 進行上下文建模與卷積擷取。

(3) DPCNN : 堆疊深層卷積層、Shortcut 連接與 Region Embedding。

- 特點

(1) 不需考慮長序列依賴，運算高效。

(2) 支援並行計算。

- 應用

文本分類、情感分析、評論分類、文章主題標記。

◆ 5. Seq2Seq + Attention

- 技術核心

- (1) Encoder-Decoder 結構，用於序列到序列任務（如翻譯、摘要）。
- (2) Attention 機制解決固定向量瓶頸。

- 架構細節

- (1) Encoder：LSTM/GRU 編碼輸入序列為 context vector。
- (2) Decoder：基於該向量逐步生成目標序列。
- (3) Bahdanau：可學的注意力權重，為每一時間步計算上下文對齊。
- (4) Luong：dot product attention，效率更高。

- 應用

翻譯（中英、日英等）、問答系統、聊天生成、摘要生成。

◆ 6. Transformer

- 技術核心

完全基於注意力的序列模型，移除循環，並行計算效率高。

- 結構組成

- (1) Self-Attention：學習每個 token 與其他 token 的關聯權重。
- (2) Multi-Head Attention：從多個語意角度學習表示。
- (3) Positional Encoding：補充序列順序資訊。
- (4) 殘差連接 + LayerNorm 穩定訓練。

- 缺點

對長序列計算成本高（ $O(n^2)$ ），容易導致記憶體爆炸。

- 應用

是所有大型語言模型的核心骨幹（BERT、GPT、T5）。

◆ 7. BERT (Bidirectional Encoder Representations from Transformers)

- 技術核心

- (1) 使用 Masked Language Model (MLM) 學習語境表示。
- (2) 雙向 attention，捕捉語境左右依賴。

- 預訓練任務

- (1) MLM：遮蔽輸入中的詞讓模型預測。
- (2) NSP：預測兩句是否連續。

- 衍生模型

- (1) RoBERTa：去除 NSP，加大訓練資料與 batch。
- (2) ALBERT：參數共享 + embedding 分解，效率優化。
- (3) DeBERTa：引入 decoupled attention 與相對位置編碼。

- 應用

文本分類、NER、語義匹配、句子排序、閱讀理解 (SQuAD)。

◆ 8. GPT 系列 (Generative Pretrained Transformer)

- 技術核心

- (1) 自回歸語言模型 (Causal LM)，預測下一個詞。
- (2) 用於文字生成任務。

- GPT 演進

- (1) GPT-2：大規模生成能力。
- (2) GPT-3：few-shot learning, zero-shot prompting。
- (3) GPT-3.5 / GPT-4：多模態、可控生成、長上下文 (16K~128K)。

- 關鍵技巧

- (1) Prompt Engineering：利用自然語言引導模型行為。
- (2) Chain-of-Thought、Instruction Tuning：增強推理能力。

- 應用

聊天機器人、長文生成、程式碼輔助 (Codex)、AI 文章撰寫。

◆ 9. T5 (Text-to-Text Transfer Transformer)

- 技術核心

將所有 NLP 任務轉為「文字輸入 → 文字輸出」格式。

- 預訓練任務

span corruption (隨機遮蔽句中片段)，重建文本。

- 特色

支援多任務訓練 (翻譯、分類、摘要、問答皆可)。

prefix + prompt 指定任務類型。

- 延伸模型

- (1) mT5：支援多語。
- (2) Flan-T5：instruction tuning 適應多樣任務。
- (3) UL2：Universal Language Learning，統一 pretraining 模式。

- 應用

多任務模型、摘要、重寫、機器翻譯、QA。

◆ 10. BART (Bidirectional and Auto-Regressive Transformers)

- 技術核心

- (1) 結合 BERT (編碼) 與 GPT (解碼) 之優點。
- (2) 預訓練目標為：將損壞文本還原為原始輸出。

- 預訓練方式

Token deletion、sentence permutation、text infilling 等擾動。

- 優勢

適合 seq2seq 任務如摘要、QA、句子改寫。

- 應用

自動摘要 (CNN/DM benchmark)、對話生成、資料修復。

◆ 11. Transformer 進階模型 (效率提升與結構強化)

- (1) Transformer-XL：引入 segment recurrence 機制保留跨段上下文。
- (2) Longformer：使用窗口式 self-attention，降低長文處理成本。
- (3) Reformer：LSH attention：使用 locality-sensitive hashing 減少 attention 計算。
- (4) BigBird：將 full + random + local attention 結合處理百萬字級長文。
- (5) Performer：使用 kernel-based attention，使 self-attention 計算降為線性。

◆ 12. 檢索強化生成 (Retrieval-Augmented Generation, RAG)

- 技術核心

- (1) 結合知識檢索與生成式模型。
- (2) 「檢索 → 融合 → 生成」三階段。

● 結構

- (1) 檢索器：使用 DPR/BM25 將相關文件找出。
- (2) 生成器：如 BART，將上下文與問題生成答案。

● 應用

- (1) 開放領域問答 (open-domain QA)
- (2) 金融與醫療領域長文查詢生成
- (3) 法條與知識文件問答

◆ 13. 模型微調技術 (Parameter-Efficient Fine-Tuning, PEFT)

- (1) LoRA (Low-Rank Adaptation)：將大模型權重矩陣分解為低秩矩陣微調，節省參數與資源。
- (2) Adapter：在每層 Transformer 中插入可訓練模塊，主模型權重凍結。
- (3) Prefix Tuning：僅訓練一組 task-specific prefix 向量接在原輸入前。
- (4) Prompt Tuning：將任務知識轉為 embedding 層輸入，不修改模型本身。

三、代表應用場景與模型配適

應用任務	技術需求	常用模型
文本分類	特徵提取 + 表示學習 + 分類器	TF-IDF + SVM / BERT + FCN
情感分析	情緒詞彙 + 序列建模	LSTM / GRU / BERT
問答系統	上下文建模 + 回答生成	BERT / GPT / RAG
摘要生成	序列到序列生成	Seq2Seq / Transformer
關鍵詞提取	TF-IDF / TextRank / LDA	jieba.analyse / HanLP

四、語言模型與生成式應用

◆ (一)傳統語言模型 (Statistical Language Models)

1. N-gram 模型

基本概念：以機率方式建模句子中第 n 個詞出現的條件機率， $P(w_t | w_{t-1}, \dots, w_{t-n+1})$ 。
須滿足馬可夫假設 (即詞僅依賴前 $n-1$ 個詞)。

平滑技巧：

- (1) Add-one / Laplace smoothing：避免出現 0 機率。
- (2) Kneser-Ney smoothing：強化低頻詞的語境影響力。

優點：訓練快速、解釋性佳。

缺點：維度爆炸、無法捕捉長距依賴、不支持 OOV 詞。

2. Neural Language Model (NNLM)

Bengio 等人提出，使用神經網路學習詞嵌入與語境關聯。

架構：

- (1) 輸入層：前 $n-1$ 個詞的詞向量。
- (2) 隱藏層：通常為 ReLU 或 tanh。
- (3) 輸出層：Softmax 預測下一詞機率分布。

引入了「詞嵌入 (word embedding)」與「語境學習」兩大概念。

◆ (二) 上下文語言模型 (Contextual Language Models)

1. ELMo (Embeddings from Language Models)

- (1) 使用雙向 LSTM 生成上下文敏感的詞嵌入。
- (2) 每個詞的表示由不同層組合而成 (低層捕捉語法，高層捕捉語意)。

特點：可插入任意模型架構中作為額外特徵。

2. GPT 系列 (Causal Language Model)

- (1) GPT-1：左到右單向 Transformer，next token prediction。
- (2) GPT-2：大規模語料訓練，生成連貫段落能力強。
- (3) GPT-3 / 3.5 / 4：引入多層 transformer decoder。
- (4) 支援 zero-shot / few-shot / chain-of-thought prompting。
- (5) 使用 RLHF (Reinforcement Learning from Human Feedback) 強化人類可讀性。
- (6) GPT-4 引入多模態能力與更長上下文處理 (可達 128K tokens)。

3. BERT (Masked Language Model)

雙向 Transformer Encoder。

預訓練任務：

MLM：隨機遮蔽句中 15% token，預測其原始值。

NSP：預測第二句是否為第一句的續句。

非自回歸，適合分類、抽取、理解類任務。

◆ (三) 編碼-解碼生成模型 (Seq2Seq + Transformer)

1. T5 (Text-to-Text Transfer Transformer)

核心理念：所有 NLP 任務皆轉換為文本到文本。

預訓練：Span Corruption，隨機遮蔽片段並還原。

多任務設定：使用 prefix 指定任務 (如翻譯、分類、摘要)。

強化版本：

- (1) mT5：多語預訓練版本。
- (2) FLAN-T5：instruction tuning 加強跨任務能力。
- (3) UL2：融合 denoising 與 causal 預訓練策略。

2. BART (Bidirectional and Auto-Regressive Transformers)

結合 BERT 編碼能力與 GPT 解碼能力。

預訓練策略：

token deletion、sentence permutation、span masking

適合文本修復、摘要生成、自然對話。

3. Pegasus (Google, 2020)

為生成摘要任務設計，使用 gap sentence generation (GSG) 預訓練任務。

遮蔽句子後訓練模型重建該句，強化關鍵資訊擷取能力。

◆(四)增強語言模型 (強化、指令、檢索導向)

1. RAG (Retrieval-Augmented Generation)

混合生成與檢索：

- (1) 檢索器：DPR、BM25、Faiss
- (2) 生成器：BART、T5

任務流程：

- (1) 給定 query 檢索相關文本
- (2) 將文本餵入生成模型作為 context
- (3) 生成自然語言回答

用於 open-domain QA、企業資料導入應答。

2. Prompt Tuning / Instruction Tuning

- (1) Prompt Tuning：調整輸入 prompt (如 "Summarize:") 即可引導模型完成任務。
- (2) Instruction Tuning：微調模型以理解自然語言任務描述，如 FLAN。
- (3) In-Context Learning：使用 few-shot example 提示模型學習任務邏輯。

3. RLHF (Reinforcement Learning from Human Feedback)

- (1) 模型先以 supervised fine-tuning 訓練，再由人類評分器給出 ranking。
- (2) PPO 演算法根據 ranking 微調模型。

用於 ChatGPT、Bard 等符合人類偏好的生成對話。

◆ (五)多模態語言生成模型 (Multimodal Generative Models)

1. Flamingo (DeepMind)

- (1) 接收圖像 + 文本作為輸入，產生連貫敘述。
- (2) 使用 Cross-Attention 機制將圖像特徵融合進文字。

2. GPT-4 (OpenAI)

- (1) 支援圖像 + 文本輸入 (Vision Transformer 模組)，可用於視覺問答。
- (2) 已成功應用於圖像解讀、流程圖分析等多模態任務。

◆ (六)控制式生成模型 (Controllable Generation)

1. CTRL (Conditional Transformer Language Model)

在輸入加入控制碼 (control code)，如「 [Wikipedia] 」、「 [News] 」，生成對應風格文本。

2. PPLM (Plug and Play Language Model)

利用梯度微調 latent representation，引導 GPT 生成特定主題文本，而不需重新訓練整體模型。

3. DialoGPT

微調 GPT-2 用於多輪對話，針對 response diversity 與流暢度優化。

◆ (七)語言模型應用範疇深入展開

A. 自動寫作與內容生成

使用模型：GPT-3.5、Claude、Command R

應用：部落格、技術文、腳本撰寫、自動校稿、改寫

技術強化：

- (1) 文體指令控制 (Formal, Friendly)
- (2) 結構提示 (Introduction, Body, Conclusion)

B. 問答系統 (QA)

類型：

- (1) Closed-domain：使用 BERT、ALBERT 微調於 SQuAD。
- (2) Open-domain：RAG, FiD, REALM 結合檢索與生成。

強化方法：

- (1) Dense Passage Retriever (DPR)

(2) Fusion-in-Decoder (FiD): 將多段輸入融合後再解碼生成。

C. 摘要生成 (Summarization)

抽取式模型：

TextRank、LexRank、BERTSum

生成式模型：

T5、BART、Pegasus

長文摘要：

Longformer Encoder-Decoder (LED)、BigBird-Pegasus、ChatGPT-API + sliding window

D. 翻譯 (Machine Translation)

模型：

(1) Transformer (Vaswani et al.)

(2) mBART：多語種 seq2seq

(3) mT5：多語多任務

技術關鍵：

Shared vocabulary、語言標籤 tokens、語言轉換特化層

E. 對話系統

開放型：ChatGPT、BlenderBot、LaMDA

任務導向：Schema-Guided Dialogue、Rasa、Task-Oriented BERT

技術策略：

(1) Dialogue State Tracking (DST)

(2) Dialogue Policy Learning (DQN, PPO)

(3) Knowledge-grounded Dialogue (利用知識檢索生成回應)

◆ 應用案例 (以 LLM 為基礎)

應用場景	代表數據集與實作範例	效能比較 (速度/精度/資源)	最佳模型選擇建議
基礎語言模型、輸入法、拼字修正	Penn Treebank、KenLM	快 / 低 / 極省	輕量級 NLP 建模、小型設備部署
簡易語意建模、詞嵌入初始化	WikiText-2、One Billion Word	中 / 低 / 省	學術實驗、詞嵌入預訓練

上下文詞嵌入、命名實體識別、句法標註	CoNLL-2003、 OntoNotes	中 / 中 / 一般	上下文敏感任務首選
自然語言生成、詩歌與短文創作	OpenWebText、 Poetry dataset	快 / 中 / 一般	小型創作型任務起步
Chatbot、寫作助手、code generation、多輪對話	CodeX, HumanEval, OpenAI API	慢 / 高 / 高	高端生成、多任務智慧助理
文本分類、NER、閱讀理解、情感分析	GLUE、SQuAD、 CoNLL-03	中 / 高 / 一般	語義理解基礎任務最佳
通用 NLP 任務 (分類、摘要、翻譯、QA)	SuperGLUE、 CNN/DM、XSum	中 / 高 / 高	跨任務整合優選
摘要生成、語意改寫、內容濃縮	CNN/DM、Gigaword	中 / 高 / 中	文風修飾、摘要應用
新聞與長文摘要生成	XSum、Newsroom	中 / 高 / 中	高質新聞摘要或研究文章
開放領域問答系統、知識型聊天機器人	NaturalQuestions、 MS MARCO、KILT	慢 / 高 / 高	複雜 QA 與企業內查詢系統
任務控制、格式化輸出、Few-shot prompting	T5 + PromptSource、 多樣化指令資料集	快 / 中 / 極省	低資源場景與快速實驗
ChatGPT, Claude 等人類偏好對齊生成任務	Anthropic HH-RLHF、 OpenAI DPO	慢 / 極高 / 高	強人類偏好控制型對話
圖片 + 文字生成、多模態描述	LAION、COCO Caption	慢 / 高 / 極高	影像-語言任務、內容生成
視覺問答、多模態搜尋、多模態指令執行	MMBench、VQAv2	慢 / 極高 / 極高	多模態助理、AI Agent 模型
風格控制、語調轉換、主題限定生成	CTRL corpus、Reddit- conditioned prompt	快 / 中 / 中	可控風格生成任務
主題引導式對話、主題生成	OpenWebText + Topic classifier	中 / 中 / 一般	無需大模型微調的主題控制

多輪對話生成、社群聊天、即時對應	PersonaChat、DailyDialog	中 / 中 / 一般	輕量聊天模型
關鍵詞抽取、抽取式摘要、SEO 建議詞	DUC2004、ZHIHU Extractive	快 / 中 / 極省	結構化內容抽取
抽取式摘要、文件壓縮、段落濃縮	CNN/DM Extractive、XSUM Hard Extract	中 / 中 / 中	長文本濃縮
雙語翻譯、多語翻譯、跨語理解	WMT14、IWSLT	中 / 高 / 高	高質翻譯與教學訓練
多語任務處理、跨語翻譯、摘要與問答	XTREME、FLORES-101	中 / 高 / 高	跨語大模型應用
聊天助理、任務型對話、內容生成工具	OpenAI Playground、ChatML API	慢 / 極高 / 高	商用 AI 對話入口
任務導向型對話、訂餐、訂票、客服流程	SGD Dataset、MultiWOZ	中 / 中 / 一般	特定任務導向 (銀行、旅遊)

L21102 電腦視覺技術與應用

一、電腦視覺核心任務分類

(一) 📌 影像分類 (Image Classification)

📄 定義：

將整張圖片分類為一個或多個已知類別 (單標籤或多標籤任務)。

🔍 關鍵技術：

- (1) 卷積神經網路 (CNN) 為核心架構
- (2) 運用池化層 (Pooling)、激活函數 (ReLU)、BatchNorm
- (3) 使用 Softmax 輸出概率分布 + Cross Entropy 損失函數

🧪 模型與演算法：

LeNet-5 (1998)

由 Yann LeCun 提出，專為手寫數字識別

結構：Conv → Pool → Conv → Pool → FC

搭配 sigmoid/tanh，已不常用但為經典基礎模型

AlexNet (2012)

引爆深度學習熱潮的經典模型，使用 GPU 訓練

採用 ReLU 激活與 Dropout，結合多通道卷積

VGG (2014)

模型統一為 3×3 卷積 + 2×2 Max Pool，深度高達 19 層 (VGG-19)

GoogLeNet / Inception (2014)

透過 Inception 模組實現多尺度提取，減少參數

ResNet (2015)

引入「殘差學習」(Residual Learning)，使用 skip connections 解決梯度消失，可達 152 層深度 (ResNet-152)

DenseNet (2017)

所有層都相互連接，提升資訊流與梯度傳遞效率

EfficientNet (2019)

使用 NAS (神經架構搜尋) 自動找出高效模型

採用 Compound Scaling 同時調整深度、寬度、解析度

(二) 📦 目標檢測 (Object Detection)

📖 定義：

從圖像中找到所有目標位置並為其指定類別 (提供邊界框 + 類別標籤)。

🔍 任務流程：

- (1) 特徵提取 (CNN)
- (2) 區域提議 (Region Proposal)
- (3) 邊界框回歸 (Bounding Box Regression)
- (4) NMS (Non-Maximum Suppression) 去除重複框

🧪 模型與技術脈絡：

A. Two-Stage 架構 (精度高、速度較慢)：

R-CNN (2014)：

使用 Selective Search 提出候選框 → 對每個框單獨提取 CNN 特徵

缺點：運算耗時，非端到端訓練

Fast R-CNN (2015)：

改為整張圖提取 CNN 特徵，候選框用 ROI Pooling 進行分類與回歸

Faster R-CNN (2015):

使用 Region Proposal Network (RPN) 產生候選框
真正實現端到端訓練 · 準確率高

Libra R-CNN 、 Cascade R-CNN (2018~)

多層級候選區域回歸提升難例偵測效果

B. One-Stage 架構 (即時應用為主):

YOLO 系列

- (1) YOLOv1 : 將圖像劃分為網格 → 每個網格預測物件框
- (2) YOLOv3 : Darknet-53 backbone + 多尺度預測 + 殘差結構
- (3) YOLOv5~YOLOv8 : 引入 mosaic augmentation 、 CSPNet 、 Anchor-Free 機制

SSD (Single Shot MultiBox Detector)

每一層特徵圖預測不同大小的目標 → 特別適合中小物件
搭配多樣的預設框 (Default Boxes)

RetinaNet

使用 Focal Loss 解決前景與背景樣本不平衡問題

DETR (2020)

引入 Transformer 架構
擺脫 anchor 與 NMS · 直接回歸物件位置與類別 (End-to-End)

(三) 🧠 語意分割 (Semantic Segmentation)

📄 定義:

為每一個像素指定語意類別 → 圖像到語意圖的映射

🔍 技術特點:

網路輸出需與輸入同尺寸 (透過轉置卷積、上採樣實現)

評估指標: Pixel Accuracy 、 mIoU 、 Frequency Weighted IoU

🧪 代表模型:

FCN (Fully Convolutional Network)

- (1) 將分類 CNN 改為逐像素輸出
- (2) 使用 skip connection 恢復解析度

U-Net (醫療圖像經典)

編碼器 (下採樣) + 解碼器 (上採樣) + 跳躍連接 (skip connections)

SegNet

解碼端使用編碼時記錄的 pooling index 來進行上採樣

DeepLab 系列

- (1) DeepLabv1 : 引入空洞卷積 (dilated convolution)
- (2) DeepLabv3+ : 加入 ASPP (Atrous Spatial Pyramid Pooling) 和解碼器

PSPNet (Pyramid Scene Parsing Net)

使用 Pyramid Pooling Module (PPM) 整合多尺度上下文

HRNet (High-Resolution Net)

維持多解析度輸出支線同時運作 → 保留高解析度細節

(四) 🎯 實例分割 (Instance Segmentation)

📄 定義 :

不僅需標記每個像素的語意類別 , 還要區分實體 → eg. 圖中兩隻狗要分開標記

🔍 關鍵挑戰 :

融合語意分割與物件偵測資訊

增加遮擋下實體區分能力

🧪 代表模型 :

- (1) Mask R-CNN : 在 Faster R-CNN 基礎上添加分割 mask 分支
- (2) ROIAlign 替代 ROI Pooling → 解決量化誤差
- (3) YOLACT (You Only Look At CoefficientTs) : 實時運作、速度快 → 使用 prototype + mask coefficient 合成 mask
- (4) BlendMask、SOLOv2、CenterMask : Anchor-Free 方法、中心點感知 (Center-aware) , 結合語意掩碼與定位資訊

(五) 🧑 人體姿勢估計 (Pose Estimation)

📄 定義 :

輸出入體各關鍵點 (如手肘、膝蓋、脖子) 之 2D/3D 座標位置

🧪 模型 :

- (1) OpenPose : 預測 Part Affinity Field (PAF) 來連結關鍵點對多人姿態辨識經典

- (2) HRNet：各分支保持高解析度並融合低解析度特徵 → 適合精細姿勢識別
- (3) PoseNet：Google 提出的單目相機姿勢估計，適合 AR 應用
- (4) AlphaPose / HigherHRNet：多人姿勢估計效果顯著，支援實時分析

(六) 🖼️ 單張深度估計 (Monocular Depth Estimation)

📖 定義：

僅由單張 RGB 影像預測出場景的深度圖 (每像素深度)

🔍 代表模型：

- (1) MiDaS (by Intel)：使用 Transformer backbone，泛化性極高，適合多場景環境
- (2) DPT (Dense Prediction Transformer)：使用視覺 Transformer 進行像素級預測
- (3) MegaDepth / Monodepth2：利用結構光、立體學資料訓練 → 改進建模精度與遠近層次感

二、關鍵卷積神經網路架構與演算法演進

(一) CNN 基礎與核心概念

CNN (Convolutional Neural Network) 是電腦視覺的支柱技術，模仿生物視覺皮層的感受野 (Receptive Field)，其核心概念如下：

- 卷積層 (Convolution Layer)：使用可訓練濾波器提取局部特徵
- 激活函數 (ReLU)：引入非線性，使模型更具表達力
- 池化層 (Pooling)：降維與位置容忍 (Max Pooling / Avg Pooling)
- Batch Normalization：穩定梯度、加快收斂
- Dropout：防止過擬合
- 全連接層 (FC Layer)：特徵轉換為類別分布

🎓 電腦視覺六大核心技術與模型細部技術總解構

(一) 卷積層 (Convolution Layer)

💡 原理：

模仿視覺皮層的感受野機制，透過核心的濾波器 (kernel/filter) 提取圖像的局部特徵。

📐 計算方式：

對輸入特徵圖 XX 使用一組卷積核 K 執行滑動內積 (Dot Product)，輸出為特徵圖 Y ：

$$Y[i, j] = \sum_m \sum_n X[i + m, j + n] \cdot K[m, n]$$

✂️ 核心超參數：

Kernel size (常見為 3x3, 5x5)

Stride (步長，控制輸出尺寸)

Padding (邊界補 0 · 避免尺寸收縮)

Dilation rate (空洞卷積用)

 擴展類型：

- Depthwise Convolution：只對每個通道各自卷積 (MobileNet)
- Grouped Convolution：特徵圖分組 (ResNeXt, ShuffleNet)
- Dilated Convolution：擴張感受野 (DeepLab)

 應用模型：

- 標準卷積：LeNet, AlexNet
- 空洞卷積：DeepLab 系列 (語意分割)
- 可分離卷積：MobileNet, Xception
- 多尺度卷積：Inception 系列

(二) 啟動函數 (Activation Function)

 目的：

在每一層加入非線性映射，賦予模型學習複雜特徵的能力。

 常見啟動函數：

函數	公式與圖像特性	優點	缺點
ReLU	$f(x) = \max(0, x)$	快速收斂、計算簡單	對負值梯度為 0 (Dying ReLU)
Leaky ReLU	$f(x) = \max(\alpha x, x), \alpha \approx 0.01$	負值區有微梯度	參數需手動調整
PReLU	可學習的斜率參數	更靈活	增加參數量
ELU / SELU	指數平滑負值區域	更穩定梯度	計算複雜
Swish	$f(x) = x \cdot \sigma(x)$	Google 提出，性能佳	複雜度高
GELU	Transformer 常用	保留梯度連續性	非常耗資源

 應用模型：

ReLU：幾乎所有 CNN

GELU / Swish：Vision Transformer, BERT, EfficientNet

(三) 池化層 (Pooling Layer)

 原理：

將鄰近區域的資訊做降維總結，降低特徵圖尺寸並增強位置容忍性。

 類型：

類型	描述	適用場景
Max Pooling	取局部最大值，保留邊緣與紋理	通用（較具鑑別力）
Average Pooling	取區域平均值，平滑特徵	與 ResNet 最終分類配合使用
Global Average Pooling (GAP)	對整個特徵圖平均，壓成 $1 \times 1 \times C$	MobileNet, ResNet, SqueezeNet
Stochastic Pooling	根據值大小機率性抽樣	Regularization

🔄 演進方向：

Adaptive Pooling (PyTorch)：可以輸出固定大小

Spatial Pyramid Pooling (SPPNet)：多尺度特徵融合 (YOLOv3)

🧪 應用模型：

- VGGNet (Max Pooling)
- ResNet (GAP)
- YOLOv3 (SPP Layer)

(四) 批次正規化 (Batch Normalization, BN)

💡 目的：

解決「內部協變數偏移 (Internal Covariate Shift)」問題，使每層輸入分布穩定，促進梯度傳遞、加速收斂。

📌 流程：

對每一個 mini-batch 特徵做標準化：

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, y = \gamma \hat{x} + \beta$$

γ, β ：可學參數，恢復表達能力

🎯 優點：

- 增加學習穩定性
- 可使用更大學習率
- 降低對 Dropout 的依賴

🔄 擴展版本：

變體	描述
LayerNorm	每一個樣本內標準化 (RNN 常用)
InstanceNorm	每通道標準化 (風格遷移)

變體	描述
GroupNorm	小 batch 環境的替代 (GN)
SwitchNorm	動態切換多種正規化方式

 應用模型：

- ResNet (BN + ReLU)
- Transformer (LayerNorm)
- StyleGAN (InstanceNorm)

(五) Dropout (隨機失活)

 原理：

在訓練過程中以機率 pp 隨機「關閉」神經元，防止過度依賴特定特徵 (即過擬合)。

 操作：

- 訓練階段：隨機選定節點不參與計算
- 測試階段：按 pp 等比例縮放權重輸出

 優點：

- 抑制過擬合 (尤其在小資料集)
- 增加模型的隨機性與魯棒性

 延伸：

- DropConnect：隨機關閉權重連接
- Spatial Dropout：針對整個 feature map 做 Dropout
- Monte Carlo Dropout：測試階段保持 dropout，進行不確定性估計 (Bayesian)

 應用模型：

- AlexNet (原始使用 Dropout)
- VGG (後段 Dropout)
- WideResNet (Dropout between blocks)

(六) 全連接層 (Fully Connected Layer, FC)

 定義：

將多維特徵展平為一維向量後與權重矩陣相乘產生輸出，具備最高密度參數連接，作為分類器末端輸出層。

 特性：

所有節點兩兩相連

輸出形狀： $y = Wx + b$

權重數量 = 輸入特徵 × 輸出類別 + bias

 優化策略：

GAP 取代 FC：減少參數 (MobileNet、ResNet)

Attention/Transformer 替代 FC：能動態學習特徵權重

 應用模型：

- VGG：大量 FC 層 (4096→4096→1000)

- ResNet：僅最後一層 FC (GAP→1000)

Transformer：Self-Attention + FFN (全是 FC 架構)

 總結技術演進圖譜

卷積層 → 激活函數 → 池化層 → 批次正規化 → Dropout → 全連接層

↓ ↓ ↓ ↓ ↓ ↓

多尺度卷積 ReLU/Swish GAP/SPP BN/GN/SN Spatial Dropout Transformer 替代

三、經典卷積神經網路架構與技術演進路徑

(一) LeNet-5 (1998)【早期範式】

應用場景：手寫字體辨識 (MNIST)

結構：C1 → S2 → C3 → S4 → FC5 → FC6 → Softmax

創新技術：引入卷積與池化層、tanh 激活

限制：訓練數據小 (無法處理複雜場景)、缺乏深度與表達能力

(二) AlexNet (2012)【深度學習時代開端】

ImageNet 錦標賽冠軍 · Top-5 Error: 15.3%

結構：5 層卷積 + 3 層全連接

創新技術：

(1) 使用 GPU 訓練 (CUDA + CuDNN)

(2) ReLU 取代 sigmoid/tanh (解決梯度消失)

(3) Dropout 防止過擬合

(4) Data Augmentation (隨機裁剪、翻轉)

限制：

參數量大、模型龐大、後續模型朝向更深、更窄、更輕量發展

(三) VGGNet (2014)

Top-5 Error: 7.3% (VGG-19)

結構設計哲學：所有卷積核均為 3x3，堆疊形成深層模型 (VGG-16 / VGG-19)

創新技術：

- (1) 模型簡潔統一，便於複製與修改
- (2) 使用 1x1 convolution 增強非線性

限制：

參數極多 (VGG-19 $\approx$ 143M)、運算與記憶體成本高

(四) GoogLeNet / Inception (2014)

Top-5 Error: 6.7%

核心模組：Inception Module

並行使用 1x1, 3x3, 5x5 卷積 + Max Pooling、使用 1x1 卷積作為維度壓縮 (bottleneck)

創新技術：

- (1) 模型寬度與深度動態配置
- (2) 模型計算效率提升 (較 VGG 快 2.5 倍，參數僅 1/12)

(五) ResNet (2015) 【殘差革命】

Top-5 Error: 3.57% · ImageNet 冠軍

核心技術：Residual Block

引入 Skip Connection： $y = F(x) + x$ ，解決深層模型梯度消失/退化問題

變種版本：

ResNet-50 / ResNet-101 / ResNet-152

Bottleneck 設計 (1x1 $\rightarrow$ 3x3 $\rightarrow$ 1x1)

深度突破：

支援 100+ 層網路訓練，仍保持優異精度

(六) DenseNet (2017)

Dense Connectivity：每層輸出皆與後續所有層相連接

特徵融合方式：

特徵疊加而非相加 $\rightarrow$ 增加資訊流通與梯度傳遞

結構組件：

Dense Block + Transition Layer

使用 Growth Rate 控制特徵通道增長

優點：

參數少 (ResNet 的一半)、訓練快、表現穩定

應用：語意分割、分類、醫療圖像診斷

(七) MobileNet 系列 (V1~V3) 【輕量神經網路】

針對 邊緣運算、移動裝置 設計的輕量級 CNN 架構

✦ MobileNet V1 (2017)

使用 Depthwise Separable Convolution

相比標準卷積可減少 8-9 倍 FLOPs

結構：Depthwise $\rightarrow$ Pointwise (1x1)

控制因子：Width Multiplier (α) + Resolution Multiplier (ρ)

✦ MobileNet V2 (2018)

Inverted Residual + Linear Bottleneck

倒置殘差：從低維升到高維再投影回低維

更適用於低記憶體場景

✦ MobileNet V3 (2019)

使用 NAS (Neural Architecture Search) 自動調參

引入注意力模組 SE (Squeeze-and-Excitation)

分為：

MobileNetV3-Large (效能優先)

MobileNetV3-Small (資源節省)

(八) EfficientNet 系列 (2019~2021)

AutoML 搜索出的模型架構

Compound Scaling：同時調整網路深度 (depth)、寬度 (width)、解析度 (resolution)

效能：EfficientNet-B7 在 ImageNet 上達到 SOTA

後續進化：EfficientNetV2：混合卷積 + 訓練加速 (progressive learning)

(九) RegNet (Facebook AI · 2020)

提出網路設計的 正則化規律性 (Regularity)

目標：尋找參數與 FLOPs 最佳性價比模型

設計原則：根據 depth, width, group convolution 等參數自動生成結構

四、延伸卷積結構與強化模組

(一) 空洞卷積 (Atrous / Dilated Convolution)

📖 定義：

透過在卷積核間插入間距 (hole rate 或 dilation rate) 來擴張感受野，無需增加參數數量。

📐 計算方式：

標準卷積是連續取樣，空洞卷積為跳躍式取樣，例如 dilation=2 時，相當於用 3×3 卷積看 5×5 的範圍。

 優點：

- 增大感受野
- 無需下採樣
- 適用於高精度語意預測

 代表應用：

- DeepLab 系列 (v2/v3/v3+)：語意分割中擴展感受野以捕捉上下文
- WaveNet (聲音生成)：生成長期依賴序列

(二) 可分離卷積 (Separable Convolution)

將標準卷積拆分為兩個步驟：

- Depthwise Convolution：每個輸入通道單獨卷積
- Pointwise Convolution (1×1)：整合不同通道資訊

 優點：

計算量減少近 9 倍

精度下降有限

 代表模型：

- MobileNet 系列
- Xception
- EfficientNet

(三) 組卷積 (Grouped Convolution)

 定義：

將輸入與輸出通道分成多組，僅在每組內進行卷積。

 代表模型：

- AlexNet：最早使用組卷積來分配到多 GPU
- ResNeXt：提出 Cardinality (組數) 作為提升效能新維度
- ShuffleNet：進一步提出 Channel Shuffle 解決信息孤立問題

 優化版本：

Depthwise 卷積 = Grouped Convolution 的極限情況 (每組=1)

(四) 注意力模組 (Attention Modules)

 任務：

提升網路的「重點感知能力」，根據特徵重要性重新分配資源。

 模型分類：

模組	描述	應用模型
SE (Squeeze-and-Excitation)	利用全局平均池化取得通道關注權重	ResNet-SE, SENet
CBAM (Convolutional Block Attention Module)	結合通道注意力 + 空間注意力	ResNet, YOLOv5
ECA (Efficient Channel Attention)	使用卷積代替 FC 更輕量	EfficientNet
BAM (Bottleneck Attention Module)	引入通道與空間關注於 bottleneck 處	ResNet
Triplet Attention	引入三維交互注意力提升精度	輕量分類與分割網路

(五) Inverted Residual & Linear Bottleneck (MobileNetV2)

 概念：

將傳統殘差塊倒轉：從低維 → 高維 → 非線性操作 (ReLU6) → 再壓縮為低維 (Linear)

 原因：

在低維空間使用非線性操作會造成資訊損失，因此壓縮階段不加激活函數。

 應用：

- MobileNetV2
- MobileNetV3

(六) 多尺度模組 (Multi-scale Feature Extraction)

 概念：

不同物件大小需不同感受野進行有效辨識，常見手法如下：

 方法與模型：

技術	描述	代表模型
Inception 模組	並行多種卷積核 ($1 \times 1, 3 \times 3, 5 \times 5$)	GoogLeNet
FPN (Feature Pyramid Network)	多層特徵自底向上融合	RetinaNet, Mask R-CNN
SPP (Spatial Pyramid Pooling)	多尺寸池化堆疊	YOLOv3
ASPP (Atrous Spatial Pyramid Pooling)	多種 dilation rate 的空洞卷積併行	DeepLabv3+

(七) Normalization 增強模組

📖 類型對照：

正規化技術	運作方式	適用場景
BatchNorm	對 mini-batch 做標準化	CNN 訓練加速
LayerNorm	對單一樣本中每一層進行正規化	Transformer
InstanceNorm	對每個通道獨立標準化	Style Transfer
GroupNorm	對通道分組進行正規化	小 Batch 訓練
SwitchNorm	混合多種正規化	多場景兼容性

(八) 現代模組：ConvNeXt 模型

📖 核心理念：

- 結合 Transformer 設計與 CNN 計算效率
- 使用 LayerNorm、GELU、Depthwise Convolution、Large Kernel

🔧 技術特點：

- (1) 替換 BatchNorm → LayerNorm
- (2) 使用 7x7 depthwise 卷積模擬全局注意力
- (3) 垂直模仿 ViT 結構（倒殼設計）

📱 應用：

- 作為 ImageNet SOTA 模型骨幹
- 替代 ResNet 成為分類/偵測主流骨架

(九) 模組整合應用實例

架構	使用模組	運用場景
YOLOv5	C3 模組、SPP、CBAM	即時物件偵測
EfficientNet	MBConv、SE、Swish	資源受限下的高效分類
ResNeSt	Split-Attention (類似 CBAM)	分類與分割競賽 SOTA
ConvNeXt	GELU, LayerNorm, DWConv	新一代分類架構

五、物件檢測與辨識技術細節

🔍 (一) 什麼是物件檢測？

任務定義：在圖像中定位所有目標物件的位置（Bounding Boxes）並辨識其類別（Classification）

輸入：單張 RGB 圖像

輸出：每個物件的 (x, y, w, h) + 類別標籤

任務難度高於單純分類，需同時兼顧空間定位 + 語意辨識

🔄 (二)核心流程架構

輸入圖像 → 特徵提取 (CNN) → 候選區域產生 (Proposal / Anchor) → 分類 + 邊界框回歸 → 後處理 (NMS) → 輸出結果 (boxes + labels)

✂️ (三)物件檢測模型分類 (依架構分類)

類型	描述	優點	缺點
Two-stage	先產生候選區域，再進行分類與回歸	精度高	較慢，非即時
One-stage	直接預測物件位置與類別	快速	小物體檢測弱、精度略低
Anchor-free	不依賴預設框，直接預測中心點	訓練簡單、泛化佳	過去精度略低 (近年已克服)

🔧 (四)Two-Stage 系列代表模型 (高精度路線)

1. R-CNN (2014)

- Region Proposal + CNN 特徵提取 + SVM 分類器
 - 使用 Selective Search 找候選框
- 特點：特徵提取與分類分開，非端對端

2. Fast R-CNN (2015)

- 改為整張圖像提取 CNN 特徵
- 使用 ROI Pooling 處理每個候選區域
- 結合分類與邊界框回歸

3. Faster R-CNN (2015)

- 引入 RPN (Region Proposal Network)
- 小卷積網路滑動整個特徵圖，自動產生候選區域
- 完整端對端訓練，速度與精度大幅提升

4. Mask R-CNN (2017)

- 在 Faster R-CNN 架構上加入 segmentation 分支
- 使用 ROI Align (比 ROI Pooling 精度高)

🧠 (五)One-Stage 系列代表模型 (即時應用)

1. YOLO 系列 (You Only Look Once)

版本	特點
YOLOv1 (2016)	將圖像分為網格，對每個格子預測固定數量的框
YOLOv2 (YOLO9000)	使用 Anchor box、BatchNorm、HiRes
YOLOv3	多尺度預測、Darknet-53 backbone、FPN
YOLOv4	引入 SPP、Mish 激活、Cross-Stage-Partial
YOLOv5	PyTorch 重寫，支援多種變種 (n/s/m/l/x)
YOLOv6/7	工業化優化、支援 DNN 推理引擎
YOLOv8	改用 Anchor-Free，支援 segmentation、pose

2. SSD (Single Shot MultiBox Detector)

- 在多個層級上輸出不同尺度的特徵圖 (Feature Maps)
- 適合偵測多大小不一物體
- 損失函數包含 Smooth L1 + Cross Entropy

3. RetinaNet

- 引入 Focal Loss 解決前景/背景樣本不均問題
- 使用 FPN + ResNet backbone
- 精度與速度兼具

💡 (六)Anchor-Free 檢測模型 (新世代)

1. CenterNet

每個物體中心點預測一個高斯分佈，回歸寬度與高度，不需 Anchor，架構簡單，定位準確

2. FCOS (Fully Convolutional One-Stage)

直接在特徵圖上做像素級回歸，引入 Center-ness 評分，提升預測準確度

3. YOLOv8

開始轉向 Anchor-free 設計，精度與速度比前代 YOLO 更進一步

🔧 (七)損失函數與訓練技巧

🔧 損失函數類型

類型	目的	代表
分類損失	判斷正確類別	Cross Entropy, Focal Loss

類型	目的	代表
邊界框損失	回歸框與真實框差距	L1, Smooth L1, GloU, CloU
結合損失	混合定位與分類	YOLO loss = obj + cls + bbox

⚙️ 訓練技巧

- Hard Negative Mining：解決背景過多問題
- Data Augmentation：如 Mosaic、MixUp、CutMix (YOLOv5 常用)
- 多尺度訓練：隨機改變輸入解析度

📊 (八)模型效能評估指標

指標	說明
mAP (mean Average Precision)	平均準確率：衡量分類與定位雙重正確性
IoU (Intersection over Union)	預測框與真實框的重疊比例
FPS (Frame Per Second)	推理速度
AP@50 / AP@75	不同 IoU 門檻下的準確率

🔧 (九)應用場景

應用	說明
自駕車感知	偵測行人、車輛、交通標誌
安防監控	識別異常動作與可疑目標
醫療影像	病變區域定位與分類 (如肺結節)
製造檢測	檢查瑕疵、零件組裝錯誤
零售分析	商品追蹤、顧客動線分析

🔍 (十)架構比較總覽

模型	精度	速度	是否 Anchor-free	特點
Faster R-CNN	★★★★★	★★	否	高精度，訓練成本高
YOLOv5	★★★★	★★★★★	否	工業標準，PyTorch 開源版本
YOLOv8	★★★★★	★★★★★	☑	Anchor-free，支援 segmentation
RetinaNet	★★★★	★★★	否	Focal Loss 處理樣本不平衡
FCOS	★★★★	★★★★	☑	輕量、結構簡潔、無 Anchor 設計

六、輔助視覺任務與強化技術

(一) 🚀 Self-Supervised Learning (自監督學習)

💡 定義：

不依賴人工標註資料，而是從數據本身構造學習目標 (proxy task)，讓模型在無監督下學到有效表徵。

🔧 核心方法分類：

類型	說明	代表方法
預測型	預測遮蔽區域、順序、顏色等	Context Prediction、Colorization
對比學習 (Contrastive)	最大化正樣本相似度、最小化負樣本相似度	SimCLR, MoCo, BYOL, DINO
Masking 類	Mask 特徵/圖像，要求模型重建	MAE (Masked AutoEncoder)、BEiT
排序或排序學習	學習圖像變換前後順序	Jigsaw Puzzle, Temporal Order

(二) 🚀 注意力模組 (Attention Modules)

🔍 此類模組為卷積架構 (CNN) 引入動態選擇特徵通道或區域的能力

🌱 1. SE 模組 (Squeeze-and-Excitation)

對每個 feature map 計算 global average pooling → 得到 channel descriptor

通過 FC→ReLU→Sigmoid 生成 channel attention weights

將權重應用至原始通道進行 re-weighting

🚀 公式簡述：

$$Output_c = \sigma(W_2 \cdot ReLU(W_1 \cdot GAP(X_c))) \cdot X_c$$

📺 應用：SENet、EfficientNet、MobileNetV3

🌱 2. CBAM (Convolutional Block Attention Module)

Channel Attention + Spatial Attention

通道注意力：MaxPool + AvgPool → FC

空間注意力：Channel 通道最大值/平均值 → 送入 7x7 conv → Sigmoid

📺 應用：ResNet-CBAM、YOLOv5 的 C3CBAM 結構

🌱 3. ECA (Efficient Channel Attention)

不使用全連接層，改為 1D 卷積進行局部交互

效率更高，適用於移動端部署

📺 應用：EfficientNetV2、ShuffleNetV2

(三) Feature Fusion Modules (特徵融合模組)

1. FPN (Feature Pyramid Network)

對 backbone 中不同層的輸出進行 top-down 加 lateral connection 合併
可處理多尺度物件 (小物體)

 應用：RetinaNet、Mask R-CNN、YOLOv3 FPN

2. PANet (Path Aggregation Network)

在 FPN 基礎上增加 bottom-up pass
增強底層語意資訊往高層回流

 應用：YOLOv4、PP-YOLO

3. BiFPN (Bidirectional FPN)

可學權重的融合層 (Weight Feature Fusion)
支援上下雙向訊息流

 應用：EfficientDet

(四) 正規化與活化函數強化 (訓練穩定化)

1. LayerNorm (Transformer 使用)

對每個樣本的特徵維度做標準化
常配合於自注意力架構

2. GroupNorm

對通道分組標準化，適合小 batch 訓練

3. GELU (Gaussian Error Linear Unit)

替代 ReLU 的平滑激活函數
增強表達能力

 應用：Transformer、ViT、ConvNeXt

(五) 多任務學習 (Multi-task Learning, MTL)

以共享特徵學習同時解決多個任務，提高效率與泛化能力

 搭建方式：

Shared Backbone → 多個 Head

Head 1 : Classification

Head 2 : Object Detection

Head 3 : Segmentation

 損失設計：

$$L_{total} = \sum_i \lambda_i \cdot L_i$$

可使用 Uncertainty Weighting 或 Learnable Weighting 調整任務損失貢獻

 應用：YOLOv8-Seg, MTL-Net (關鍵點+分類)、Face++ (臉部多任務)

(六) 模型壓縮與推理優化技術 (強化部署)

 1. 知識蒸餾 (Knowledge Distillation, KD)

使用大型模型 (teacher) 指導小模型 (student)

輸出 soft label 作為訓練目標 (可提供隱性知識)

 應用：DeiT (ViT 精度強化)

 2. 模型剪枝 (Pruning)

移除權重較小或冗餘神經元

保持推論效能下降最小化

 3. 量化 (Quantization)

將模型參數從 32-bit 浮點轉為 8-bit / 4-bit

節省運算與記憶體

 應用：Edge AI (嵌入式部署)

(七) Self-Training (半監督學習的一種)

透過預測未標資料的 pseudo label，再加入訓練迴圈中

搭配信心閾值過濾不可靠樣本

 應用：半監督偵測 (Semi-supervised detection)、PseudoSeg

(八) 動態路徑選擇 (Dynamic Routing)

根據輸入樣本特徵，自適應選擇路徑 (而非固定網路結構)

SkipNet：動態選擇是否略過部分層

CondConv / SwitchableNet：依條件選擇不同權重分支

 應用：自適應推論、邊緣裝置運算優化

 小結：輔助與強化模組的角色與價值

技術類型	核心價值	代表技術	模型應用
自監督學習	提高表徵學習能力	SimCLR, MAE	MAE, DINO
注意力機制	提升特徵選擇性	SE, CBAM	EfficientNet, YOLOv5
特徵融合	多尺度特徵聚合	FPN, BiFPN	RetinaNet, EfficientDet
多任務學習	效率提升、共享學習	MTL Head, KD	YOLOv8-seg
正規化/活化	穩定訓練與收斂	GELU, LayerNorm	ViT, ConvNeXt
模型壓縮	邊緣運行與部署	Quantization, Pruning	MobileNet, TinyYOLO
自適應推理	調整模型路徑	CondConv	SwitchNet

L21103 生成式 AI 技術與應用

一、生成式 AI 概論

(一) 定義與範疇

- GenAI 是深度學習 (Deep Learning) 的一個子領域，強調從大量資料中學習結構與規律，以產生全新內容 (文本、圖像、音樂等)。
- 與判別式模型 (Discriminative Models) 不同，GenAI 是生成式模型，關注「生成數據的機率分佈」。

(二) 演化歷程簡述

年代	技術里程碑	說明
1906	馬可夫鏈	最早的統計語言生成模型
2014	生成對抗網路 (GAN)	引入對抗機制，使生成更擬真
2015	Seq2Seq + Attention	改善翻譯、摘要等序列任務品質
2017	Transformer 架構	改變 NLP 發展格局，引入自注意力
2018+	GPT, BERT, T5	引領 LLM 熱潮，邁入萬億參數規模
2022+	Diffusion Models	成為圖像生成主流模型 (如 Stable Diffusion)

二、生成式 AI 核心技術

(一) 核心生成模型

◆ 1. GPT (Generative Pre-trained Transformer)

■ 核心理念

GPT 是一種基於 Transformer Decoder 架構的自回歸語言模型，利用預訓練學習大量語料的語言規則，並透過上下文預測下個詞，達成文字生成、摘要、問答等任務。

■ 三階段流程

1. Unsupervised Pretraining：用大量語料做語言建模 (Language Modeling, LM)
2. Supervised Fine-tuning (如有)
3. In-Context Learning / Prompt Learning

■ 模型架構 (GPT-2 版本為例)

- 多層 Transformer Decoder (12 層、24 層、96 層等)
- 每層包含：
 - Masked Multi-head Self-Attention
 - Feed Forward Network (FFN)
 - Layer Norm + Residual Connection

■ 數學公式

語言模型的目標是最小化：

$$L = - \sum_{t=1}^T \log P(x_t | x_1, \dots, x_{t-1})$$

應用實例

- ChatGPT、GPT-4、GPT-4 Turbo
- Code generation (GitHub Copilot)
- 文章生成、對話系統、客服自動回應

◆ 2. BERT (Bidirectional Encoder Representations from Transformers)

核心理念

BERT 是基於 Transformer Encoder-only 架構，主要透過「雙向上下文理解」強化詞語的語意表示，是語言理解型模型 (Discriminative)，而非生成模型。

關鍵訓練任務

1. Masked Language Modeling (MLM):

- 隨機遮蔽 15% 詞彙，模型預測該位置的詞：

$$L_{MLM} = - \sum_{i \in \text{masked}} \log P(x_i | \text{context})$$

2. Next Sentence Prediction (NSP):

- 判斷句子 B 是否接續句子 A (0 or 1)

優勢

- 雙向語境建模，語意捕捉能力強
- 適合遷移學習 (Fine-tune on QA, NER, Text Classification)

典型應用

- 自然語言理解 (NLU)
- 情感分析、問答 (SQuAD)、命名實體識別 (NER)
- Sentence Embedding (如 SBERT)

◆ 3. T5 (Text-to-Text Transfer Transformer)

核心理念

T5 將所有 NLP 任務轉換為文字到文字 (Text-to-Text) 格式，例如：

輸入：translate English to German: How are you?

輸出：Wie geht es dir?

訓練方法

- 目標：最大化轉換任務的輸出機率

$$L = - \sum_{t=1}^T \log P(y_t | y_{<t}, x)$$

架構：Transformer Encoder-Decoder (完整版)

- 使用 全雙向 Encoder 與 自回歸 Decoder

- 大型版本參數可達 11B+

訓練語料

- C4 corpus : Colossal Clean Crawled Corpus · 750GB+ 清理後網頁文本

任務示例

任務類型	輸入格式	輸出例子
翻譯	translate English to German: ...	...
文本摘要	summarize: ...	...
問答	question: ... context: ...	answer
文本分類	classify sentiment: ...	positive

優勢

- 統一處理框架、易於擴展多任務
- 成為後續多任務大模型如 FLAN-T5 的基礎

◆ 4. Diffusion Models (擴散模型)

核心理念

Diffusion 模型透過學習將高斯噪音逐步轉換為資料分佈，特別適合用於圖像生成。

原理

① 正向過程 (Forward Process)

- 持續加入雜訊至資料樣本 x_0 ，形成 $x_1, \dots, x_T$ ：

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

② 反向過程 (Reverse Process)

- 學習如何從雜訊還原為資料：

$$p_\theta(x_{t-1} | x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

架構設計

- 基於 U-Net 作為主要生成網路
- 加入時間嵌入 (Timestep Embedding)

代表模型

- DDPM (Denoising Diffusion Probabilistic Model)
- Stable Diffusion：文本到圖像 (Text-to-Image)
- Imagen / DALL-E 2：Google、OpenAI 的擴散模型

特點

- ☒ 高品質圖像生成
- ☒ 可控制性強 (條件生成)
- ☐ 推論速度較慢 (可用 DDIM 改進)

◆ 5. GAN (Generative Adversarial Networks)

┃ 核心理念

GAN 是透過對抗學習的機制，讓一個「生成器 (Generator)」學會如何騙過一個「判別器 (Discriminator)」的結構。

┃ 損失函數

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

┃ 代表變種

變種名稱	特點
DCGAN	卷積架構，更適合圖像生成
WGAN	使用 Wasserstein 距離，提高穩定性
StyleGAN	控制圖像風格、生成高品質臉部
CycleGAN	無需配對資料，圖像轉換 (如馬 ↔ 斑馬)

┃ 應用場景

- 人臉生成、圖像超分辨率
- 圖像轉換 (黑白 → 彩色、白天 → 夜晚)
- 時尚設計、音樂、藝術創作

END 總結比較表

模型	架構	任務類型	特點	代表應用
GPT	Transformer Decoder	自回歸生成	高連貫語言建模	ChatGPT, Copilot
BERT	Transformer Encoder	語言理解	雙向語境、微調效率高	QA, NER, 檢索
T5	Encoder-Decoder	文本轉文本	統一任務格式	多任務 NLP
Diffusion	U-Net + 時間條件化	生成式	高質感圖像、控制強	Text-to-Image
GAN	雙網對抗結構	生成式	對抗學習，極擬真圖像	StyleGAN, CycleGAN

🧠 生成式 AI 四大重要機制

◆ (一)Self-Attention 與 Transformer 架構

🌱 Self-Attention (自注意力) 機制

┃ 核心理念：

每一個輸入 token 可以根據序列中其他位置的資訊，動態加權地建構自身的表示。

┃ 數學運算流程：

給定輸入向量 $X \in R^{n \times d}$ ，我們將其轉換為三個矩陣：

- Queries $Q = XW^Q$
- Keys $K = XW^K$
- Values $V = XW^V$

接著計算注意力分數：

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

- $\frac{QK^T}{\sqrt{d_k}}$ ：計算 query 與 key 的相似度
- softmax：轉為權重分布
- 加權 Value 值取得每個 token 的「加權上下文表示」

🧠 Transformer 架構

組成模組 (以 Encoder 為例)：

1. Multi-head Self-Attention
2. Add & Layer Norm
3. Feed Forward Layer
4. Add & Layer Norm

Multi-Head 設計意圖：

- 並行執行多個注意力機制，以捕捉不同語意子空間的關聯。
- 例如：一個 head 專注主詞-動詞關係；另一個專注時間序列。

位置編碼 (Positional Encoding)：

Transformer 無法處理順序 → 使用 sinusoidal 編碼：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

☑ 優點

- 可擴展性強 (並行運算)
- 長距離依賴建模優秀
- 上下文捕捉靈活

◆ (二) RLHF (Reinforcement Learning from Human Feedback)

■ 背景說明

原始語言模型如 GPT 僅優化預測下個詞的機率，不保證「輸出合理、邏輯一致、符合人性喜好」。

RLHF 解決的是：「讓 AI 更像人，且對話更安全」。

🧠 三階段訓練流程

階段	說明	技術
① Supervised Fine-tuning (SFT)	人類標註高品質對話 → 作為教師訊號	監督式學習
② Reward Model (RM)	讓模型判斷哪個回應更好 (偏好建模)	Pairwise ranking

階段	說明	技術
訓練		
③ Reinforcement Learning	使用 PPO (Proximal Policy Optimization) 讓生成回應符合 RM 評分	Policy Gradient

PPO 演算法簡介

優化策略函數 $\pi_{\theta}(a | s)$ 最大化：

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

其中：

- $r_t(\theta)$ 是新舊策略比值
- $\hat{A}_t$ 是 Advantage (好壞的衡量)
- Clip 避免步伐太大，確保穩定性

關鍵成果與應用

- GPT-3.5 / GPT-4 對話體驗巨大提升
- ChatGPT、Claude、Gemini 均採用 RLHF
- 為生成內容導入倫理、安全、可控性

◆ (三) LoRA / PEFT (參數高效微調)

為何需要 PEFT？

全參數微調	PEFT
訓練所有參數	只調整部分輕量參數
儲存空間大 (GB 級)	小 (MB 級)
資源需求高	可用筆電等低資源設備

◆ LoRA (Low-Rank Adaptation)

┆ 核心理想：

將全連接層 WW 的更新拆解為：

$$W' = W + \Delta W = W + BA$$

其中：

- $A \in R^{r \times d}$
- $B \in R^{d \times r}$
- $r \ll dr$ ：即「低秩矩陣」

只訓練 A 與 B ，不動原始參數 W 。

◆ Adapter、Prefix Tuning 其他 PEFT 方法

方法	說明	優勢
Adapter	在原始層中插入小模組	輕量、模組化

方法	說明	優勢
Prompt Tuning	僅學習固定向量 prompt	快速切換任務
Prefix Tuning	為每層 attention 加 prefix 向量	強化序列控制性

✦ 實際應用

- 微調 LLaMA2、ChatGLM 等大型模型 (多語、醫療、教育)
- HuggingFace peft 支援多種 PEFT 方法

◆ (四)Prompt Engineering (提示詞工程)

💡 定義：

精心設計輸入提示，引導生成模型產生預期輸出，是使用 LLM 的核心技術之一。

🔍 類型分類

類型	說明	範例
Zero-shot	無範例，直接提問	Translate "apple" to French
Few-shot	提供數個範例	Q: 1+1 → A: 2 ... Q: 2+2 → A:
Chain-of-Thought (CoT)	引導模型一步步推理	Let' s think step by step.
Role Prompting	設定模型人格/角色	You are a math tutor...
Instruction Prompt	明確給任務指令	Summarize the text below.

🧰 技術策略

▮ Prompt Templates 建構

template = "請以{tone}語氣幫我寫一封道歉信，主題是：{topic}"

▮ 指令詞搭配技巧

- 「你是...」 → Role Playing
- 「請分點列出」 → 格式控制
- 「使用繁體中文」 → 語言約束
- 「給我 3 種風格...」 → 多樣性提升

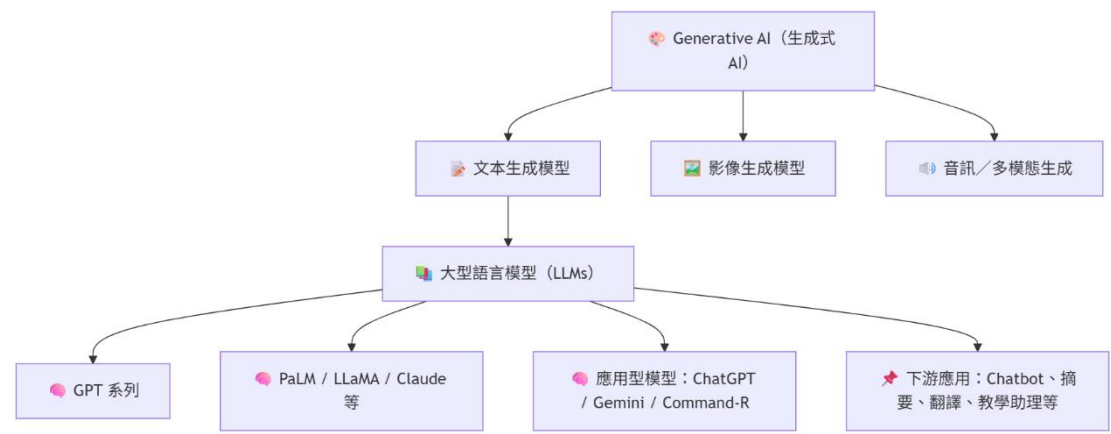
📄 整體總結比較

機制名稱	類型	核心功能	使用模型
Self-Attention	架構設計	建立全序列依賴關係	GPT, BERT, T5
Transformer	通用框架	多任務編碼/解碼	幾乎所有 LLM
RLHF	訓練優化	強化人類對話偏好	ChatGPT, Claude
LoRA/PEFT	微調策略	高效任務微調	LLaMA, Alpaca
Prompt Engineering	使用方法	精準引導生成	所有對話式模型

三、大型語言模型與生成式人工智慧的關係

大型語言模型 (LLM, Large Language Model) 與生成式人工智慧 (GenAI, Generative AI) 的關係，從架構、功能、發展與應用層面來看，是一種涵蓋與驅動的核心與體現關係：

🧠 (一)總體架構關係概覽



☑ GenAI 是總領域，LLM 是核心技術實現的子集，特別聚焦在「語言生成」領域。

🦋 (二) LLM 是 GenAI 在語言領域的代表性實踐

▮ 什麼是 LLM？

大型語言模型是指參數量在數十億至萬億級別的深度學習模型，使用 Transformer 架構，透過大規模語料進行自監督訓練，具備以下特性：

- 具備上下文理解與語言生成能力
- 支援多任務 (問答、摘要、翻譯、對話等)
- 可經由 Prompt 引導完成 Zero-shot / Few-shot / Chain-of-Thought 推理

▮ 什麼是 GenAI？

生成式 AI 是指具備創造新資料的能力的人工智慧系統，涵蓋：

- 文本生成：GPT、T5
- 圖像生成：DALL-E、Stable Diffusion
- 語音生成：Whisper、Voicebox
- 多模態生成：Gemini、GPT-4V、Sora

🔴 二者關係的三層面比喻：

比喻	LLM 是什麼	GenAI 是什麼
軟體堆疊	核心演算法庫	整體應用平台
工廠產線	語言製造引擎	各類產出品
建築系統	結構鋼筋與電路系統	整體建築與功能空間

🎨 (三)技術視角：LLM 如何驅動 GenAI？

◆ 1. 能力層面：通用生成能力

能力	說明
語言生成	故事創作、電子郵件撰寫
對話理解	Chatbot、客服 AI
任務執行	撰寫程式碼、資料轉換
多語翻譯	跨語言生成與摘要
語境記憶	對話追蹤與上下文保留

◆ 2. 訓練方式：生成導向訓練

- 目標函數：最小化下個 token 的預測誤差：

$$L = - \sum_{t=1}^T \log P(x_t | x_1, \dots, x_{t-1})$$

- 資料來源：Wikipedia、Common Crawl、Reddit、GitHub 等

◆ 3. 應用範疇：語言生成是 GenAI 的核心實踐場域

GenAI 子領域	代表模型 (多數為 LLM)	生成內容
文本生成	GPT-4, Claude, PaLM2	對話、論文、摘要
程式碼生成	Codex, Code LLaMA	Python, Java, SQL
多語翻譯	mT5, NLLB	多語言句子
多輪對話	ChatGPT, Gemini, LLaMA-2-Chat	人工客服/教練
知識問答	GPT-3.5 + RAG, LLaMA + RAG	答案、建議
教學/生成導師應用	Agent GPT, AutoGPT	任務規劃/執行

🔧 (四)工程視角：GenAI 應用架構 = LLM + 工具鏈整合

📦 GenAI 應用技術堆疊：

層級	技術元件
使用者層	Chat UI / Voice Assistant
應用邏輯層	Prompt Templates / LangChain
知識強化層	RAG (檢索增強生成)
模型核心層	LLM (如 GPT-4, Claude)
基礎架構層	GPU, TPU, 向量資料庫 (如 FAISS, Weaviate)

🌐 (五)產業與應用觀點：LLM 是 GenAI 成功的催化劑

- ◆ LLM 推動 GenAI 應用落地的關鍵因素：

關鍵點	說明
語言能力強	可支持多語、多樣生成任務
可少樣本學習	Few-shot Learning 讓模型應用更泛化
容易 API 化	透過 OpenAI、Anthropic API 快速部署
與工具鏈整合	可與 RAG、Agents、資料庫整合

- ◆ 應用場域實例

領域	GenAI 功能	採用 LLM
教育	自動出題、對話導學	GPT-4、Claude
法律	判例摘要、條文解釋	LLaMA + 法律語料
醫療	問診助理、知識導航	Med-PaLM 2
電商	產品文案生成、客服	GPT-3.5 + 對話強化
軟體開發	自動程式補全	Codex、Code LLaMA

- ☑ (六)總結：LLM 是 GenAI 在語言領域的「引擎與腦」

面向	LLM 角色	對 GenAI 貢獻
技術	核心模型	提供生成能力
應用	通用基礎層	供應各種應用
工程	模組化核心	易於整合與擴展
發展趨勢	AI Agent、Tool Use	延伸更廣泛生成應用

四、生成式 AI 應用實例與架構設計

應用場景

領域	案例	技術核心
教育	自動出題、學習路徑建議	GPT + 自然語言生成
法律	法條摘要、案件生成模擬	T5 + RAG
醫療	ChatDoctor 模型診斷建議	LLaMA 微調 + 醫療資料集
電商	產品描述自動撰寫、客服回應	多模態生成 + RLHF
金融	報告生成、投資摘要	RAG + 微調模型
多模態應用	影片生成 (文生圖→影片)、語音生成	Diffusion + VAE + TTS 模組

五、進階話題：RAG、MoE、LLM Agents

🧠 (一)RAG (Retrieval-Augmented Generation) 檢索增強生成

■ 背景說明

大型語言模型 (如 GPT) 在生成內容時，無法即時更新知識、無法接觸外部資料，會出現「幻覺 (hallucination)」現象。

RAG 提供一種將檢索系統結合生成模型的方式，使 LLM 可利用「即時、精準、可控」的外部知識。

📋 流程步驟

1. 查詢編碼 (Embedding):
 - 將使用者輸入轉為語意向量 (使用如 BGE, MiniLM, Instructor)
2. 相似度檢索 (k-NN):
 - 在知識庫中找出語意最接近的數筆文件 (如 FAISS, Weaviate, Milvus)
3. 拼接提示 (Contextual Prompt):
 - 把檢索內容 + 原查詢組成 LLM 的提示詞
4. 生成回答：
 - 使用 GPT、Claude、LLaMA 等語言模型生成回答

🔧 技術組件

模組	技術
Retriever	向量資料庫 + 檢索器 (BM25、ANN)
Ranker	rerank 預測關聯性 (可用 CrossEncoder)
Reader / Generator	LLM (如 ChatGPT、LLaMA-2-Chat)
Chunking & Embedding	OpenAI Embeddings / Sentence Transformers

🔥 應用實例

領域	應用	工具/模型
企業內部搜尋	問文件 / SOP	GPT + FAISS + LangChain
法律問答	判例查詢 + 問答	GPT + BM25 + LexisNexis
醫療支援	根據醫療知識回答患者問題	BioGPT + PubMed
資料密集摘要	整合多文獻資訊摘要	Claude + RAG

⚠️ 挑戰與對策

問題	解法
片段長度限制	Sliding window / Condense summary

問題	解法
檢索誤差傳遞	多階段檢索 + Ranker
記憶遺失	加入對話歷史 (RAG + Memory)

🧠 (二)MoE (Mixture of Experts) 專家混合模型

■ 技術背景

大型模型 (如 GPT-3、GPT-4) 參數數量龐大 → 訓練成本、推理成本高昂。

MoE 提出只啟用部分模型子網路 (Expert) 參與計算的策略，實現「大而不重」。

📊 數學描述

輸出為：

$$y = \sum_{i \in Top-k} G(x)_i \cdot E_i(x)$$

其中：

- $G(x)_i$: Gating network 的 softmax 分數
- $E_i(x)$: 第 i 個 Expert 的輸出

🔥 代表性 MoE 模型

模型名稱	發布單位	特點
GShard (2020)	Google	支援萬億參數訓練
Switch Transformer (2021)	Google	使用 1 個 Expert，簡化訓練
M6-T (華為)	華為諾亞	中文 MoE LLM，解耦稀疏訓練
DeepSpeed-MoE	微軟	支援彈性專家數與高效率分布式

🔴 優勢與挑戰

優點	挑戰
參數多但推理計算少 (僅部分激活)	負載不均 (部分 Expert 負擔過重)
可彈性擴展專家數	門控不穩定、收斂難度較高
模型容量大但推理成本低	難以部署在低資源邊緣設備上

🧠 (三)LLM Agents (大型語言模型智能體)

■ 技術背景

雖然 LLM 可以回答問題，但它本身不具備「目標導向的行動與推理計劃能力」。

LLM Agent 結合 LLM、推理框架與工具操作模組，使 AI 能自主執行任務、調用外部工具、

維持記憶與上下文。

✿ 核心架構元件

組件	功能
Planning	擬定任務行動計畫
Tool Use	調用工具 (Web Search / Calculator / API)
Memory	長期記憶 / 對話上下文追蹤
Reflection	反思結果並調整行動策略
Executor	根據 LLM 產生的指令，具體執行行動

🔧 常見技術框架

技術名稱	描述
LangChain	建立多階段任務型 LLM agent
AutoGPT	自主 AI 任務執行與資源使用
AgentGPT	可瀏覽、思考、操作的智能體
ReAct (Reason + Act)	推理 + 工具調用的混合框架
Toolformer	自我學習何時調用工具的 Transformer

🔥 實際應用場景

應用場景	說明
自動研究助理	自主查詢 → 擬摘要 → 提煉觀點
財經分析智能體	擷取資料 → 製表 → 推論趨勢
軟體代理工程師	自動寫程式 + 測試 + 修正錯誤
工作流程自動化	排程 + 信件回覆 + 報表輸出

🧠 特徵總覽

模組	說明
環境互動	可透過 API 操作資料庫、Web、程式
多輪控制	有上下文記憶與結果反饋
規劃能力	可自訂子任務與順序
工具選擇	判斷使用哪個工具進行任務執行

L21104 多模態人工智慧應用

一、多模態人工智慧的定義與背景

多模態人工智慧 (Multimodal AI) 指的是能夠處理、理解與生成來自兩種以上模態 (例如文字、圖像、語音、影片、3D 資料等) 的資料與任務。其核心目標是模擬人類的感官融合與認知決策能力，使機器能夠在多種感知資料下進行語義推理、任務判斷與互動回應。

二、核心架構與模型組件 (以 MM-LLMs 為例)

多模態 LLM 主要架構可拆解為以下五大模組：

以下為您更細膩、逐層展開說明「多模態 LLM 架構五大模組」的子系統結構、對應模型、底層機制與方法原理，以碩士以上的深度理解層級撰寫，並標示應用鏈結。

✦ 模組 1：模態編碼器 (Modality Encoders, ME)

任務：將不同模態 (圖像、語音、3D、影片) 轉換為統一的語意向量表徵 (embedding)，便於與文字模態融合。

◆ 圖像模型

模型	架構與原理	技術亮點
CLIP-ViT	ViT (Vision Transformer) + Contrastive Learning	使用圖像與文字對比學習 (Contrastive Loss) 對齊語意空間
NFNet-F6	Normalizer-Free Network	去除 BatchNorm，強化數值穩定性與大模型訓練能力
Eva-CLIP	ViT + 自監督擴充	引入 masked modeling 與 image-text contrastive 雙任務自監督預訓練

◆ 語音模型

模型	架構	方法
HuBERT	CNN + Transformer	採用 Masked Prediction 搭配無標註語音分段學習
Whisper	CNN + Decoder-only Transformer	大規模監督式語音到文字訓練，支援多語種與雜訊環境
BEATs	Pre-trained Transformer	使用聲學單元編碼來提升表示學習效果 (類似 BERT for audio)

◆ 3D 模型

模型	機制	說明
PointBERT	Masked Point Prediction	將 3D 點雲表示為 token，再用 BERT-

模型	機制	說明
		style 預訓練恢復遮罩點
ULIP-2	多模態對比學習	同時對齊圖像、語音、3D 到語意空間，實現跨模態任務學習

◆ 統一模型

模型	支援模態	技術特徵
ImageBind	影像、文字、音訊、熱圖、IMU、深度圖等	將各模態投影到共同嵌入空間，實現統一語意對齊與檢索

✦ 模組 2：輸入投影器 (Input Projector)

任務：將來自不同模態編碼器的向量特徵對齊並轉換為 LLM 可理解的文字嵌入向量 (如 GPT token embedding)。

常見方法與技術細節：

方法	機制描述	適用場景
MLP (多層感知器)	線性投影 + 非線性激活 (如 GELU) 組合層，用於映射模態特徵到 LLM 嵌入維度	簡單高效，適用特徵長度固定模態 (如影像)
Cross-Attention 機制	使用模態向量作為 Query，LLM token 作為 Key/Value 或反之，以學習跨模態對齊	適用於時間性或互動性強的輸入 (語音、影片)
Q-Former / P-Former	特殊 Query token (Learnable) 學習模態與 LLM 溝通方式，具備更強上下文融合能力	應用於 BLIP-2、MiniGPT-4 等進階架構

✦ Q-Former = Query-enhanced Transformer ; P-Former = Prefix-enhanced Transformer。

✦ 模組 3：LLM 主體 (LLM Core)

任務：負責語意推理、知識整合與任務指令生成。

功能特性說明：

技術	說明
Zero-shot / Few-shot ICL	可透過提示直接完成任務，無需或僅需極少範例學習。
Chain-of-Thought (CoT)	模型透過邏輯思維鏈條產生多步推理結果 (如圖問答、文本推論)。

技術	說明
Tool Use / API Call	搭配外部工具（如 Calculator, Web Search）進行增強反應生成。

微調方法（PEFT 高效微調）：

方法	原理	特點
Prefix Tuning	在輸入前插入一串可學習參數 Prefix，用來調控注意力分布	記憶小、調整快
Adapter	在 LLM 各層插入小模塊（bottleneck networks），訓練時只更新 Adapter 參數	模塊化插拔靈活
LoRA (Low-Rank Adaptation)	對 LLM 的權重更新施加秩限制（ $W \approx A \times B$ ），僅更新小矩陣 A、B	最少參數更新、 高效遷移能力

✿ 模組 4：輸出投影器（Output Projector）

任務：將 LLM 產生的文字嵌入或語意輸出，轉換為下游模態生成器可讀的特徵向量。

技術內容：

方法	對應任務	原理描述
MLP / Linear Head	圖像生成（text2img）、 語音控制	將文字語意向量投影成潛變分模型（LDMs） 所需條件特徵
Instructional Vectorization	多步任務生成	將 LLM 的多輪指令轉換為引導 latent 的序列條件向量（Condition Vector）

✿ 模組 5：模態生成器（Modality Generator）

任務：根據 LLM 輸出語意條件生成圖像、影片、語音等模態資料。

常見架構：潛變分模型（Latent Diffusion Models, LDM）

架構	模型範例	原理
Stable Diffusion	Text-to-Image	在壓縮潛空間中學習條件擾動反向擴散過程
Make-A-Video	Text-to-Video	延伸 Stable Diffusion，使用時序一致性機制建構動態畫面
AudioLDM	Text-to-Audio	利用語音潛空間與語意向量對齊進行聲音生成

補充技術：

☒ Classifier-Free Guidance (CFG)

☒ 目標：在**不依賴額外分類器（Classifier）**的前提下，強化生成模型遵循語意提示（Prompt）的能力。

🔴 問題背景：

在條件式生成模型 (如 text-to-image) 中，模型需依據輸入文字生成符合語意的圖像。但實務上：

- 模型若太「自由」，會忽略 prompt 生成不相關圖像。
- 傳統的解法是使用一個額外的類別分類器來引導生成方向 (Classifier Guidance)，但這樣會增加系統負擔與不穩定性。

💡 CFG 的核心思路：

1. 使用兩個預測方向：
 - 有條件預測 (Conditioned)：模型以 prompt 輸入進行反向擾動步驟。
 - 無條件預測 (Unconditioned)：使用空白 prompt (如 "") 輸入模型。
2. 將兩者加權組合，公式如下：

$$\epsilon_{CFG}^{\wedge} = \epsilon_{uncond}^{\wedge} + s \cdot (\epsilon_{cond}^{\wedge} - \epsilon_{uncond}^{\wedge})$$

- $\epsilon_{cond}^{\wedge}$ ：條件噪聲預測
- $\epsilon_{uncond}^{\wedge}$ ：無條件噪聲預測
- s ：引導強度係數 guidance scale (常設為 5~15)

🔍 效果與優勢：

- 無需額外訓練分類器，僅透過一個模型完成雙向推理。
- 可靈活調整 prompt 遵守程度， s 越大，生成圖像越「貼近」語意，但也越容易出現 artifacts。
- 廣泛應用於 Stable Diffusion、Imagen、DALL·E 2 等模型中。

☑ DDIM Sampling (Denoising Diffusion Implicit Models)

☑ 目標：在不改變原始模型訓練方式的前提下，以非馬可夫的方式更快速、高品質地執行反向擾動抽樣。

🔴 傳統 Diffusion Sampling 問題：

在 Denoising Diffusion Probabilistic Models (DDPM) 中，抽樣採用多階段 (通常 1000 步) 馬可夫鏈反向擾動：

- 步驟慢 (需要上百次推理步驟)
- 計算量大，生成時間長

🔍 DDIM 的核心機制：

DDIM 透過可控的 deterministic 映射函數取代隨機擾動步驟，達成「可預測且可重建」的去噪抽樣過程。

◆ 主要差異：

- DDPM 是隨機取樣：每一步皆引入高斯噪聲。

- DDIM 是 deterministic (非隨機) 抽樣：無須加入高斯噪聲，使用特定推理方程計算樣本。

◆ 反向映射公式 (簡化):

$$x_{t-1} = \sqrt{\alpha_{t-1}} \cdot x^{\wedge}_0 + \sqrt{1 - \alpha_{t-1}} \cdot \epsilon^{\wedge}$$

- $x^{\wedge}_0$ ：根據 $\epsilon \backslash \epsilon$ 預測的乾淨圖像
- α_{t-1} ：不同時間步的累積權重

🔧 特性與優點：

- 大幅降低生成步數：可從原本的 1000 步降到 25~50 步，速度提高數十倍。
- 保留畫質：生成圖片清晰度與細節幾乎不受影響。
- 更可控：因為是 deterministic，可以方便做 latent 插值與圖像反演等任務。

🔗 技術比較總覽

技術名稱	問題解決方向	技術關鍵	應用範例
Classifier-Free Guidance	強化語意條件控制	雙向噪聲預測組合 + scale 調控	Stable Diffusion, Imagen
DDIM Sampling	高效生成 & 控制性提升	非隨機反向推理過程，顯著減少步驟	LDMs, DALLE2, MusicLM

三、模型訓練階段與優化方式

🌱 第一階段：多模態預訓練 (Multimodal Pretraining, MM-PT)

🎯 目標：

學習不同模態 (文字、圖像、語音、3D 等) 之間的共享語意表示空間 (shared semantic space)，建立跨模態理解基礎。

📁 訓練模組與技術詳解：

☑ 1.1 雙塔編碼 (Two-Tower Encoding)

- 架構：分別使用圖像編碼器與文字編碼器 (如 ViT + BERT)，將圖文對映入共享向量空間。
- 典型模型：CLIP, ALIGN
- 損失函數：

$$L_{contrastive} = -\log \frac{\exp(\text{sim}(v_i, t_i)/\tau)}{\sum_j \exp(\text{sim}(v_i, t_j)/\tau)}$$

- v_i ：圖像 embedding
- t_i ：文字 embedding
- τ ：溫度係數

☑ 1.2 Masked Prediction 預訓任務

- 文字：如 BERT 的 Masked Language Modeling (MLM)
- 圖像：如 MAE 的 Masked Patch Prediction
- 語音：如 HuBERT 的 Masked Segment Reconstruction

☑ 1.3 模態對齊學習 (Modality Alignment)

- 透過 對比學習、重建學習 或 遷移投影學習 方式將多模態向量映射到共享語意空間。
- 模型實例：
 - ImageBind (支援 6 模態)
 - Flamingo (使用 gated cross-attention 融合文字與視覺)

✚ 第二階段：多模態指令微調 (Multimodal Instruction Tuning, MM-IT)

🎯 目標：

讓模型在跨模態任務中具備指令理解能力、任務導向反應能力與生成能力，對齊人類偏好。

📁 微調模組與訓練技術：

☑ 2.1 SFT (Supervised Fine-Tuning) 監督微調

- 利用 <instruction, input, output> 形式的資料進行有標註任務微調。
- 特別針對：
 - 圖像問答 (Visual Question Answering, VQA)
 - 圖像描述 (Captioning)
 - 多輪多模態對話 (如 LLaVA, MiniGPT-4)

☑ 2.2 RLHF (Reinforcement Learning with Human Feedback)

將「人類偏好」引入訓練迴圈，強化生成內容符合人類價值與語境預期。

三階段流程：

1. 初始預訓 (SFT)：以人類示範對話進行監督式訓練。
 2. 偏好模型訓練 (Reward Model)：訓練一個模型學習人類對輸出排序的偏好。
 3. 策略微調 (PPO)：使用近端策略優化 (Proximal Policy Optimization) 強化回應品質。
- 公式目標：

$$\max_{\theta} E_{x \sim \pi_{\theta}} [r(x) - \beta \cdot D_{KL}(\pi_{\theta} \parallel \pi_{ref})]$$

☑ 2.3 PEFT (Parameter Efficient Fine-Tuning) 參數高效微調

- 避免重新訓練整個 LLM 主幹，只針對小模組進行訓練與插拔。

方法	技術核心	優勢
LoRA	使用低秩分解 $W \approx A \cdot B$	訓練參數量小、可快速切換任務

方法	技術核心	優勢
Adapter	插入 bottleneck 模組	模型穩定性高、支援多任務微調
Prefix Tuning	在輸入 token 前插入 trainable prompts	記憶效率高、適合條件式生成任務

📦 多模態 LLM 訓練任務對應表

任務類型	對應技術	模型實例
圖像問答 VQA	Cross-Attn + SFT	MiniGPT-4, LLaVA
圖像敘述/生成	Decoder + Prefix	GIT, BLIP-2, Flamingo
多模態對話	SFT + RLHF	mPLUG-Owl, ChatGPT-V
模態聯合表示	對比學習 + LoRA	CLIP, ImageBind
語音 / 影片理解	Masked + CoT 微調	Whisper+GPT, Flamingo

四、典型應用場景與技術實例

應用領域	技術實現方法	模型範例
圖像問答 (VQA)	ViT + LLM + CoT	MiniGPT-4, BLIP-2
圖像字幕生成	ViT + Transformer Decoder	Flamingo, GIT
多模態聊天機器人	LLM + 多模態 RAG	Kosmos-1, LLaVA
視頻理解與摘要	動態 ViT + 時序 LLM	VideoGPT, Make-A-Video
聽覺理解	HuBERT + LLM	AudioLM, Whisper + GPT
3D / AR 虛擬互動	Point-BERT + LLM	ULIP-2 + LLaMA

五、新興技術趨勢

☑ 多模態 LLM 高階應用技術

◆ (一) 模態擴展 (Modality Extension)

■ 定義：

將多模態 AI 模型由傳統圖像、語音、文字，進一步擴展至結構化或半結構化資料模態，如表格 (Tabular)、網頁 (HTML DOM)、地圖 (GIS 地理資訊)、熱圖 (Heatmaps) 等。

✿ 關鍵模組：

模態類型	對應編碼器	說明
表格資料	TAPAS, TaBERT	利用表格欄列結構建構位置感知嵌入，結合 Transformer 進行語意對齊
網頁結構	WebGPT Encoder, DOM-T5	將 HTML DOM 結構嵌入為樹狀序列輸

模態類型	對應編碼器	說明
		入模型
地圖/空間圖層	GeoFormer, SpatialBERT	將地理坐標、街道拓撲或 POI 轉為圖結構，結合 GNN 或坐標位置編碼
熱圖（視覺統計）	Vision Transformer + Density Map 編碼器	特徵圖 → 熱區密度 → Attention 強化區域感知性

🔧 關鍵技術：

- Multi-modal Embedding Alignment (語意向量對齊)
- Tree / Graph Encoding (樹狀或圖狀結構編碼)
- Spatial Positional Encoding (地理空間編碼)

📌 應用實例：

- 多模態助理讀取 PDF / Excel 報表問答 (如 Agent-Tables)
- 資料科學輔助建模 (如 TabLLM, OpenAI code interpreter)
- 網頁檢索摘要與頁面 QA (如 WebGPT)

◆ (二) 跨模態 RAG 強化檢索生成流程 (Cross-Modal RAG)

■ 定義：

擴展 檢索式生成 (Retrieval-Augmented Generation, RAG) 機制，讓 LLM 可根據文字指令，動態查詢圖像、語音、影片等非文字資源並融合進推理或生成流程中。

✚ 架構組成：

[Prompt] → [模態檢索器] → [語意轉換器] → [LLM 整合回應]

- 模態檢索器 (Multimodal Retriever)：
 - 圖像 → 使用 CLIP Index + FAISS 快速查詢語義近似圖像
 - 語音 → 使用 Whisper Embedding + vector search
- 語意轉換器：
 - 將 retrieved 圖像/語音特徵 → 用 Input Projector 對齊 LLM token embedding
- LLM 整合回應：
 - 使用 context-window 拼接或 cross-attention 將模態知識輸入主模型

🔧 實作方法：

- FAISS + CLIP / BEATs 向量庫
- 可選搭配 ReAct prompting：Reason + Act 輔助檢索推理融合

📌 應用場景：

- 用戶上傳圖片後問「這是什麼牌子的手錶？」
- 上傳聲音片段，模型回答「這是哪首歌？」
- 搜尋知識庫中的影片片段回答問題（如 ChatVideo）

◆ (三) 專家混合架構 (Mixture of Experts, MoE)

■ 定義：

一種參數路由技術，允許模型在每次推理時只啟用一部分子網路 (expert)，大幅提升參數規模與推理效率之間的平衡。

💡 核心技術：

- 稀疏路由門控器 (Sparse Gating Network)
 - 為每個輸入決定應該路由給哪幾個 expert (Top-k routing)
- Expert Modules 可為：
 - 專精不同語言模態 (Text Expert / Vision Expert)
 - 專精不同任務 (VQA Expert / Caption Expert)

📦 模型實例：

模型	機制	特點
GShard (Google)	分層路由 + expert	支援上千億參數模型
Switch Transformer	單 expert/step	最快且穩定
Kosmos-2 ExpertMix	將多模態任務導向特定 expert 處理	更佳泛化性

📌 應用價值：

- 多任務處理 (對話、描述、推理等)
- 多模態高擴展架構 (支援逐步加入模態子模型)

◆ (四) 模組化與可插拔訓練策略 (Modular & Plug-in Training)

■ 定義：

將整個多模態系統中各模組 (如編碼器、投影器、生成器) 模組化、拆解式設計，可支援快速更新、替換、測試。

✂ 可插拔模組範疇：

模組類型	插拔策略	說明
模態編碼器 (Encoders)	支援多模態訓練接口	CLIP-ViT 可替換為 EVA, SAM 等
投影轉換層 (Projectors)	支援 LoRA/Prefix 調整	任務快速遷移

模組類型	插拔策略	說明
模態生成器 (Generators)	LDM 可插換不同解碼器	Image-to-Image / Text-to-Sound

✦ 應用特性：

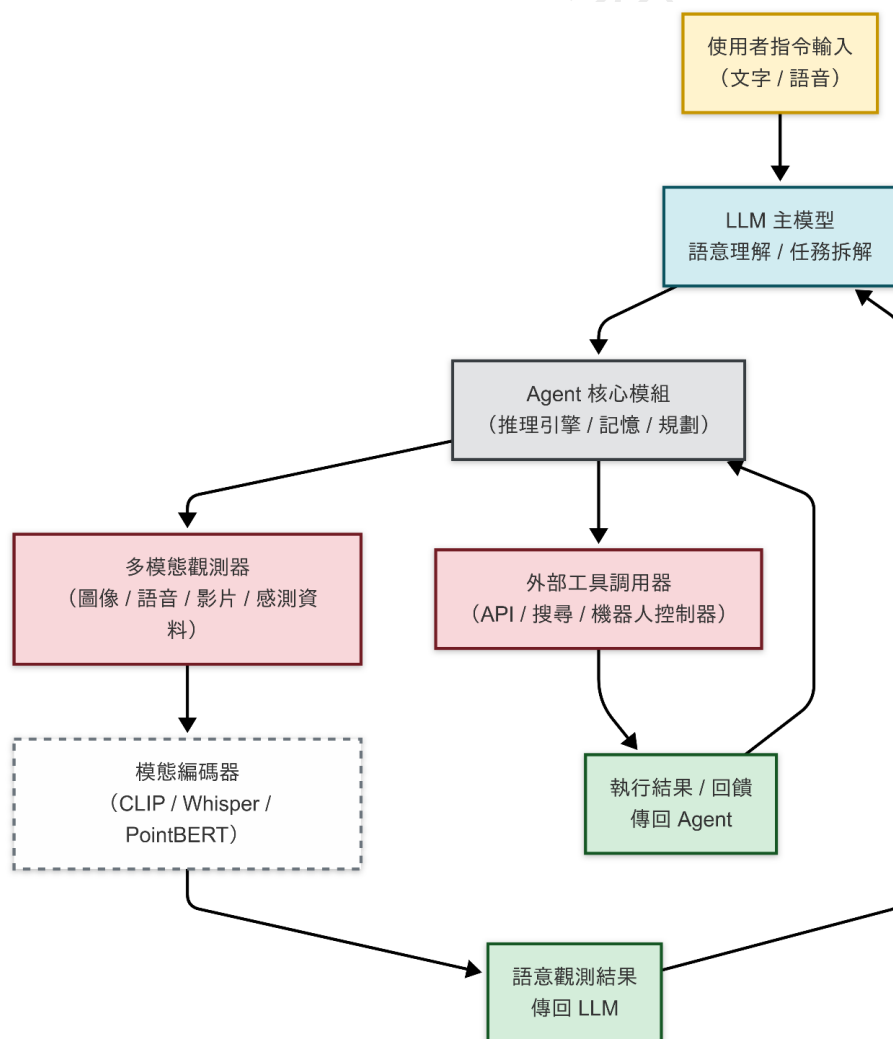
- 模型熱插拔、維運簡單
- 支援逐模態訓練 / 冷啟動 / 多任務遷移
- 類似於 HuggingFace Trainer 模組化設計

◆ (五) LLM + Agent 架構

■ 定義：

將 LLM 搭配**推理決策模組 (Agent) **與外部工具 (工具鏈、API、網頁、感測模組) 互動，使其具備「觀察-思考-行動-回饋」的封閉學習與決策迴圈。

- Agent 模組：具備長期記憶、任務規劃、上下文分析
- 多模態觀測器 (Multimodal Observers)：讀取圖片、聲音、影片，並傳回感知內容給 LLM
- Actuator 工具：執行 API 呼叫、網頁點擊、問卷填寫等



✦ 應用範例：

- Visual Chat Agents：讀懂網頁 + 回答用戶問題（如 WebArena）
- Auto-GPT + Visual + Speech：從語音、影像中解析任務 → 工具操作 → 回報結果
- RoboAgent：讓機器人透過攝像機理解環境並完成任務

L21201 AI 導入評估

一、AI 的商業價值與可行性分析

🔍 (一) 財務價值評估 (Financial Impact)

☑ 1.1 成本效益分析 (Cost-Benefit Analysis)

- 目的：評估 AI 專案產生的直接收益與支出間的比例。
- 常用指標：

- ROI (投資報酬率) : $ROI = \frac{\text{淨收益}}{\text{投資總額}}$

- NPV (淨現值)、IRR (內部報酬率)

- 注意事項：
 - 考量一次性支出 (導入、設備、外包)
 - 長期持續支出 (維運、監控、模型更新)

☑ 1.2 營收提升潛力 (Revenue Uplift)

- 建立 AI 帶來的新收入模式，如：
 - 精準推薦提升轉換率 (電商)
 - 風險模型降低違約率 (金融)
 - 自動化客服減少流失率 (客服中心)

✦ (二) 流程效率優化 (Process Efficiency)

☑ 2.1 自動化節省人力成本

- 導入 RPA + AI (如智能分類器 + 表單辨識)
- 使用技術：OCR + BERT + Workflow Orchestration
- 減少處理時間 (TAT) 與人為錯誤率

☑ 2.2 工作流程最佳化 (Process Optimization)

- AI 分析歷史作業瓶頸，提出流程再造建議 (BPR)
- 工具與技術：
 - Process Mining 工具 (如 Celonis)
 - Task Mining 與 AI 工單指派模型 (Reinforcement Learning)

🌐 (三) 顧客價值提升 (Customer-Centric Value)

☑ 3.1 顧客體驗強化 (CX)

- 客戶互動強化技術：

- 智能客服 (Dialogflow 、 RASA)
- 情緒分析 (Voice-to-Emotion)
- 評估指標：NPS (淨推薦值)、CES (努力指數)、CSAT (滿意度)

☑ 3.2 精準行銷與預測性洞察

- 使用 AI 對 RFM、LTV 模型做深度分群
- AI 驅動推薦系統 (如基於 Transformer 的 Next-Best-Action)

📊 (四) 策略對齊性 (Strategic Alignment)

☑ 4.1 是否與企業核心策略一致

- 若企業正在推動「數位轉型」或「綠色營運」，AI 專案是否能支援此方向？
- 評估矩陣：專案-策略相容矩陣 (Strategy-AI Fit Matrix)

☑ 4.2 是否創造差異化

- 是否能建立護城河 (如：自研演算法 + 專屬數據集)
- 是否能突破市場門檻 (如：自動化報價、客製化生產)

🔵 (五) 風險與合規性成本 (Risk & Regulatory Perspective)

☑ 5.1 導入風險與替代成本

- 法規合規 (如 GDPR、金融 AI 指引)
- AI 模型偏誤或黑箱風險
- 資料外洩、資安風險

☑ 5.2 機會成本評估

- 若不導入 AI，是否會被市場淘汰？
- 是否競爭者已經導入？市場先行者紅利是否正在流失？

🧠 (六) 組織文化與知識資本影響 (Human Capital)

☑ 6.1 員工工作價值轉移

- AI 取代重複工，釋放人力進行創新、洞察、設計任務
- 機會：AI 作為「決策輔助」而非「取代」

☑ 6.2 知識累積與再利用

- 建立內部 AI 模型知識庫 (Model Zoo)
- 模型可重用性與可遷移性 (Transfer Learning、少樣本學習)

📈 (七) 量化模型與工具 (工具與方法)

分析方法	說明	工具/技術
商業案例畫布 (Business Model Canvas)	梳理 AI 專案如何創造、傳遞與擷取價值	Miro、Strategyzer
VSM (Value Stream	視覺化流程價值鏈，找出無效或重	Lucidchart、LeanTools

分析方法	說明	工具/技術
Mapping)	複流程	
AI Value Pyramid	分層分析 AI 如何由數據 → 洞察 → 行動 → 商業價值	自定義 Canvas
預測模型影響分析 (What-If Tool)	分析模型不同輸入情境下，可能的 商業效益/風險差異	TensorBoard、AI Explainability 360

二、 可行性分析技術

(一) 📦 資料成熟度評估 (Data Readiness Assessment)

1.1 定義與目的

在 AI 專案導入初期，資料的可用性與質量直接決定了模型訓練的成功率與業務轉換價值。資料成熟度評估是對資料整備狀況的全面審核，涵蓋質與量、合規與治理四大層面。

1.2 評估維度

維度	說明	評估指標範例
 可取得性	資料是否存在？是否可從系統中擷取？是否具備存取權限？	Data Inventory, ETL 可用性
 質量	資料是否準確、無缺漏、無重複？是否即時更新？	缺值率、錯誤率、更新頻率
 一致性	不同系統資料定義是否一致？欄位格式與單位是否一致？	Schema 比對、一致性校驗報告
 安全合規	是否符合 GDPR、個資法、AI 導入指引等資料處理規範？	加密、匿名化、存取審核機制設計

1.3 工具與方法

◆ Data Readiness Level (DRL) 模型

由 ODI (Open Data Institute) 提出，類似 NASA 技術成熟度等級 (TRL)，將資料成熟度分為 1 至 9 級。

DRL 等級	資料狀態
DRL 1-3	資料尚未可用，需進行辨識與存取
DRL 4-6	資料可取得，開始清洗與標準化
DRL 7-9	資料已準備好可用於分析或模型訓練

◆ Data Quality Scorecard (資料品質評分卡)

建構各欄位的品質評分維度 (例如完整性、唯一性、正確性)，再加權整合為整體資料品質指數

(Data Quality Index, DQI)。

(二) 🏢 組織成熟度模型 (AI Maturity Model)

2.1 定義與目標

評估組織內部是否具備推動 AI 的條件與能力，包括文化、基礎設施、人才、策略與治理架構。用以決定是試點導入、部分推展還是整體數位轉型。

2.2 成熟度五層架構

階段層級	描述
Awareness (覺察)	組織意識到 AI 潛力，但缺乏實際行動，無 AI 項目與策略
Active (啟動)	零星導入 AI，專案多為探索性質，缺乏標準化與成效評估流程
Operational (運作)	有明確的 AI 專案與成果，導入工作流程，但多集中於特定部門
Systemic (系統化)	整體組織建構 MLOps 架構、內部人才培育、導入標準、流程化落地
Transformational (轉型)	AI 成為業務核心競爭力，形成智慧決策平台，具備自我調整與持續創新能力

2.3 搭配架構與方法

- CMMI (能力成熟度模型整合) 作為技術開發流程成熟度參考
- Gartner AI Maturity Model 將 AI 能力分為「初階使用」到「智慧型決策引擎」

2.4 實用指標

領域	具體觀察指標
策略與治理	是否成立 AI 專責小組、是否有導入治理原則
基礎建設	是否部署 GPU / 雲端平台 / 數據湖
人才能力	資料科學家、MLOps 工程師人數占比
成效追蹤	是否具備專案 KPI (如縮短時間、提升精度等)

(三) 📊 風險-收益圖譜 (Risk-Return Mapping)

3.1 目的

協助決策者在多個潛在 AI 應用中，依據技術成熟度與商業效益不確定性進行分層排序與可視化，明確找出最佳導入切入點。

3.2 評估維度

維度	說明
📈 預期收益	可量化的潛在商業貢獻 (提升營收、降低成本等)

維度	說明
 技術成熟度	所需技術的成熟度：是否有現成模組？訓練是否複雜？
 實作風險	資料取得困難度、模型落地可行性、組織接受度
 回收週期	導入時程長短與可回收時間點 (Time to Value)

三、AI 技術選型與解決方案規劃

模型選型與比較 (基於任務特性)

任務類型	建議模型	關鍵演算法/技術
分類任務	XGBoost, SVM, DNN	Boosting, Kernel Trick, Dropout
時序預測	ARIMA, LSTM, Transformer	Sliding Window, Attention
文本生成	GPT, T5, BART	Decoder-only, Seq2Seq, Copy mechanism
圖像分析	CNN, ResNet, Vision Transformer	卷積層設計、殘差模組、Self-Attention
決策優化	強化學習 (DQN 、 PPO)	Value Iteration, Policy Gradient

 AI 任務類型 × 模型 × 關鍵演算法與技術

☒ (一) 分類任務 (Classification Task)

 建議模型：

- XGBoost (Extreme Gradient Boosting)
- SVM (Support Vector Machine)
- DNN (Deep Neural Network)

 關鍵演算法 / 技術詳解：

◆ Boosting (提升學習)

- 將多個弱分類器 (如淺層決策樹) 逐步結合成強分類器
- 每一輪加強錯誤預測樣本的權重
- XGBoost 特點：

- 使用二階梯度 (Hessian) 提升收斂速度
- 支援特徵並行、列取樣 (column sampling)

◆ Kernel Trick (核函數技巧) 【SVM 特色】

- 將非線性特徵映射到高維空間，使其可線性可分
- 常見核函數：
 - 線性核：Linear
 - 高斯核：RBF (Radial Basis Function)
 - 多項式核：Polynomial

◆ Dropout (隨機神經元屏蔽) 【DNN 過擬合解法】

- 訓練時隨機刪除部分神經元，避免神經元之間依賴過強

- 減少 co-adaptation · 提高泛化能力

☑ (二) 時序預測任務 (Time Series Forecasting)

💡 建議模型：

- ARIMA
- LSTM
- Transformer (Informer, Autoformer)

🔍 關鍵演算法 / 技術詳解：

- ◆ Sliding Window (滑動窗口機制)
 - 將連續的時間序列切分成重疊的小片段進行預測
 - 用於將序列資料轉化為監督學習格式
- ◆ ARIMA (自回歸整合移動平均)
 - AR (p)：利用過去值進行預測
 - MA (q)：利用預測誤差修正結果
 - I (d)：透過差分處理非平穩資料
- ◆ LSTM (長短期記憶網路)
 - 引入「記憶單元」與「門控機制」：
 - 忘記門、輸入門、輸出門
 - 解決傳統 RNN 的梯度消失問題
- ◆ Attention / Transformer (基於注意力的預測)
 - 使用 Self-Attention 捕捉遠距序列關聯
 - Informer / Autoformer 為高效長序列預測變種
 - Sparse Attention + 分解預測趨勢/週期

☑ (三) 文本生成任務 (Text Generation)

💡 建議模型：

- GPT (Generative Pretrained Transformer)
- T5 (Text-to-Text Transfer Transformer)
- BART (Bidirectional and Auto-Regressive Transformer)

🔍 關鍵演算法 / 技術詳解：

- ◆ Decoder-only (GPT 架構)
 - 單向 Transformer Decoder · 僅使用前文 (causal masking)
 - 可用於自動寫作、續寫、問答生成
- ◆ Seq2Seq (Sequence to Sequence) 【T5/BART 結構】
 - Encoder：提取輸入語意資訊

- Decoder：逐步生成目標序列
- 搭配 Attention 模組提高長距依賴語意建模能力
- ◆ Copy Mechanism (複製機制)
 - 增強 Decoder 的指向能力
 - 適用於複製關鍵詞、專有名詞 (如摘要任務)
 - 常與 Pointer Generator Networks 結合使用

☑ (四) 圖像分析任務 (Image Analysis)

💡 建議模型：

- CNN (Convolutional Neural Network)
- ResNet (Residual Network)
- Vision Transformer (ViT)

🔍 關鍵演算法 / 技術詳解：

- ◆ 卷積層設計 (Convolution)
 - 透過局部感受野 (Receptive Field) 提取空間特徵
 - 採用共享權重大幅降低參數
- ◆ 殘差模組 (Residual Block) 【ResNet 創新】
 - Shortcut (跳接) 設計解決深層網路梯度消失問題
 - $y = F(x) + x$ ，保留原始輸入資訊，加快訓練穩定性
- ◆ Self-Attention (自注意力) 【Vision Transformer】
 - 非卷積方式提取長距像素之間的語意關聯
 - 直接處理影像 patch (如 ViT: patch+flatten+position embedding)
 - 近期如 Swin Transformer、ConvNeXt 結合卷積與 Attention 技術

☑ (五) 決策優化任務 (Decision Optimization)

💡 建議模型：

- 強化學習 (DQN、PPO)

🔍 關鍵演算法 / 技術詳解：

- ◆ Value Iteration (價值迭代)
 - 使用 Bellman Equation 遞迴計算每個狀態的期望報酬
 - 核心公式：

$$V(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s')]$$

- ◆ Policy Gradient (策略梯度)
 - 直接學習動作機率分佈 (不依賴值函數)

- 代表演算法：
 - REINFORCE
 - Actor-Critic (A2C, A3C)
 - PPO (Proximal Policy Optimization)
 - 引入懲罰項防止策略過大更新・穩定訓練
- ◆ DQN (Deep Q-Network)
 - 將 Q-Value 函數以 CNN 近似
 - 使用 Experience Replay + Target Network 提升穩定性
 - 適用於視覺輸入 + 離散行動空間 (如 Atari 遊戲)

四、解決方案模組化規劃

(一)資料處理模組 (DataOps)

 任務流程：

- ◆ 1. 清洗 (Data Cleaning)
 - 處理缺值、異常值、重複資料
 - 標準化資料格式、單位轉換
 - 工具：pandas、Great Expectations、Databricks Quality Rule
- ◆ 2. 標註 (Data Labeling)
 - 應用於監督式學習資料集建構
 - 方法：
 - 人工標註 (Label Studio、Scale AI)
 - 半監督擴充 (Pseudo-labeling)
 - 弱標註 (Snorkel)
- ◆ 3. 擴增 (Data Augmentation)
 - 提升資料多樣性與模型魯棒性
 - 圖像資料：旋轉、翻轉、裁切、混合 (Mixup, CutMix)
 - NLP：同義詞替換、EASY DATA AUGMENTATION (EDA)
 - 時序資料：窗口擴充、時間偏移、合成序列
- ◆ 4. 特徵工程 (Feature Engineering)
 - 特徵選擇：如 Recursive Feature Elimination (RFE)
 - 特徵轉換：Normalization、Embedding、PCA
 - 自動特徵工程工具：
 - FeatureTools (自動產生跨表欄位)
 - Kats (Meta Facebook 時序分析工具)

工具與平台：

☒ Feature Store (特徵倉儲)

- 中央特徵版本管理平台，可跨專案共用
- 工具：Feast, Tecton, Amazon SageMaker Feature Store
- 優點：
 - 同步線上與離線特徵
 - 特徵自動更新與統一調用接口

☒ Data Drift Detection (資料偏移偵測)

- 比較生產資料 vs 訓練資料分布差異
- 方法：
 - KS 檢定 (Kolmogorov-Smirnov)
 - PSI (Population Stability Index)
 - Embedding Drift (使用 Encoder 比較表示空間變化)

(二) 模型開發與訓練模組 (MLOps)

核心流程：

◆ 自動化訓練流程 (AutoML)

- 自動選擇模型、超參數調整、特徵處理
- 框架：
 - AutoKeras
 - Google Vertex AI AutoML
 - H2O AutoML
 - AutoGluon / FLAML / TPOT

◆ Neural Architecture Search (NAS)

- 自動搜尋最優神經網路架構 (非人工設計)
- 技術路徑：
 - Reinforcement Learning (如 ENAS)
 - 演化演算法 (如 DARTS)
 - Differentiable Search (Gradient-based)

實驗追蹤與模型管理

☒ MLflow

- 功能模組：
 - Tracking (記錄訓練參數、指標)
 - Projects (定義可重複運行訓練流程)
 - Models (模型版本管理)
 - Registry (部署與監控)

☑ Weights & Biases (W&B)

- 實驗儀表板：可即時可視化 loss、accuracy、weights 等
- 提供 Sweeps 功能做自動化超參數搜索
- 支援 PyTorch / Keras / HuggingFace 等框架整合

(三) 部署模組 (AI DevOps)

🎯 任務流程：

◆ 模型容器化 (Containerization)

- 目的：解耦部署環境與開發環境
- 使用技術：
 - Docker：封裝模型 + 預測服務
 - TorchServe：PyTorch 專用部署框架
 - NVIDIA Triton Inference Server：
 - 支援 ONNX / TensorRT / PyTorch / TensorFlow
 - 支援多模型同時上線 (multi-model ensemble)

◆ 模型服務與佈署

- 架設推論 API (Flask、FastAPI、gRPC)
- 多版本模型併行 (Shadow Model / A/B Testing)
- 整合 CI/CD 工具鏈 (Jenkins、GitHub Actions)

◆ 邊緣部署選型 (Edge AI)

- ONNX (Open Neural Network Exchange)
 - 中立格式，模型轉換工具鏈
 - 可從 PyTorch、TensorFlow 匯出至 Edge 應用
- TensorRT (NVIDIA)
 - 針對 GPU 優化的推論引擎
 - 用於自駕車、機器人、智慧攝影頭的邊緣部署

(四) 監控與回饋模組 (Monitoring & Feedback Loop)

🎯 核心任務：

◆ Concept Drift Detection (概念漂移偵測)

- 模型訓練時的輸入資料分布和實際使用時出現偏移
- 類型：
 - Virtual Drift：X 改變，Y 不變
 - Real Drift：Y 改變 (真實標籤轉移)
- 技術工具：
 - Evidently AI
 - Alibi Detect

- River (Incremental Drift Detection)
- ◆ 回饋學習策略 (Feedback Learning)
- ☑ Online Learning (線上學習)
 - 模型持續接收新資料更新 (如 SGDClassifier)
 - 適合資料分批流動 (如 IoT 裝置)
 - 框架 : River, Vowpal Wabbit
- ☑ Active Learning (主動學習)
 - 模型判斷哪些樣本「不確定性高」, 主動請人標註
 - 節省標註成本, 提升效能
 - 策略 :
 - Least Confidence
 - Entropy Sampling
 - Margin Sampling

五、技術選型考量因素

☑(一) 執行效率與推理延遲 (Execution Efficiency & Inference Latency)

◆ 為什麼重要 ?

- 部署於即時應用 (如自動駕駛、智慧客服、邊緣設備) 時, 推理時間延遲 (Latency) 和 運算量 (FLOPs) 成為關鍵瓶頸。
- 模型越大 ≠ 越好, 需在精度與效能間取得平衡。

🔍 關鍵評估指標

指標	說明
FLOPs	Floating Point Operations。單次推論所需的浮點運算數量 (越少越快)
Latency	從輸入到模型輸出所需的總耗時 (ms), 可區分為 Cold Start / Warm Start
Throughput	每秒可處理的請求量 (QPS)
Model Size	模型佔用空間大小 (MB)

🔧 技術工具

- Benchmark Suite :
 - MLPerf (NVIDIA, Google 等主導的標準)
 - Hugging Face Inference API (可比較各模型延遲)
- Profiler 工具 :
 - PyTorch Profiler、TensorBoard Trace Viewer
 - NVIDIA Nsight / TensorRT Profiler

☑(二) 資料類型與資料量 (Data Type & Volume)

◆ 為什麼重要 ?

- 選擇模型必須與資料結構與更新頻率相匹配，否則導致訓練困難、資料不匹配或無法即時預測。

🔍 關鍵分類與對應技術

📁 資料型態

類型	說明	常用模型
結構化 (表格)	行列資料，如金融報表、銷售記錄	XGBoost、TabNet、LightGBM
非結構化 (文字)	自然語言、社群評論、網頁內容	BERT、T5、GPT、TextCNN
非結構化 (圖像)	相片、影像資料	CNN、ResNet、ViT
時序資料	日誌、感測器數據、時間標記系列	LSTM、TFT、ARIMA、Informer

🕒 資料頻率與處理方式

模式	說明	適用架構
Batch	批次處理，定期收集大量資料一次訓練	Spark、Kubeflow、SageMaker
Streaming	即時資料流 (如即時交易、IoT 資料)	Kafka + Flink + Online Model

☑️ (三) 可擴展性與彈性架構 (Scalability & Flexibility)

🔥 為什麼重要？

- 模型需可適應不同資料量、佈署環境與硬體限制，才能實現「從訓練到生產」的延展性。
- 架構需具備模組化、動態擴充、彈性選擇的能力。

🔧 核心技術展開

🔹 模型壓縮技術 (Model Compression)

方法	說明	案例
Knowledge Distillation	教師模型訓練學生模型 (小型)，學生學習 soft-label 概率分布	DistilBERT、TinyBERT
Quantization	將參數從 FP32 轉為 INT8/INT4 減少記憶體與加速推論	TensorRT、TFLite Quantization
Pruning	將神經網路中不重要的連接刪除，減少參數	L1-Norm, Lottery Ticket Hypothesis
Weight Sharing	不同層之間共用參數，壓縮模型體積	MobileNet

🔹 多模型系統 (Multi-Model Systems)

架構	說明	應用場景
Model Ensemble	結合多模型輸出結果 (平均 / 投票 /	Kaggle、金融風控

架構	說明	應用場景
	stacking)	
MoE (Mixture of Experts)	多個子模型動態選擇執行，根據輸入選出最適模型子集合	GShard, Switch Transformer

L21202 AI 導入規劃

一、需求分析 (需求識別與問題建模)

(一) 問題定義與業務流程對應

☒ 技術方法

(a) 使用 BPMN 釐清流程節點

- BPMN (Business Process Model and Notation) 是企業流程建模的標準語言，用於明確定義業務邏輯。
- 在 AI 導入中，BPMN 有助於：
 - 視覺化流程瓶頸或高頻人工作業節點 (automation opportunity)
 - 確認需 AI 支援的決策點或預測任務 (如：客服分類、風險評分)

例子：

使用者進入電商平台 → 搜尋商品 → 瀏覽詳情頁 → 加入購物車 → 結帳 → 訂單完成

↳ 可導入 AI 預測「使用者轉換率」或「商品推薦」

(b) 映射為 ML 問題類型

流程場景	ML 類型	可能模型
使用者是否會購買？	分類任務	Logistic Regression, XGBoost
預測訂單金額	回歸任務	Random Forest Regressor
需求量預測	時序序列預測	LSTM, TFT
自動生成客服回答	生成式任務	GPT, T5, BART

技巧：分析任務的「輸出型態」與「決策邏輯」，即可推斷所屬的 ML 任務類型。

工具與建模輔助框架

工具/框架	功能說明
BPMN 2.0	流程標準符號，支援多角色/多實體操作分析
UML Use Case	用例圖聚焦「使用者-系統互動」，對需求拆解有利
Celonis / Apromore (Process Mining)	由 log 分析流程實際執行軌跡、發現瓶頸與變異流程

(二) 資料現況盤點與質量分析

☒ 技術檢查面向

(a) 缺失值分析 Missingness Mechanism

類型	解釋	解法建議
MCAR	資料完全隨機缺失 (Missing Completely At Random)	可刪除或平均補值
MAR	缺失與其他觀測變數相關 (Missing At Random)	多重插補 (MICE)
NMAR	缺失與其自身值相關 (Not Missing At Random)	使用模型估計缺失 (如自回歸補值)

(b) 特徵重要性排序

- Mutual Information : 衡量特徵與目標變數之間的互資訊 (非線性相關也有效)
- SHAP Value (SHapley Additive exPlanations) :
 - 適用於黑盒模型 (如樹模型、DL)
 - 可解釋個別樣本對預測的貢獻程度

工具應用

工具	功能說明
Pandas Profiling	一鍵生成資料概覽報告、包含分佈、缺失、常見值等
Great Expectations	設定資料驗證規則，自動追蹤資料品質
OpenDataDiscovery	元數據盤點、資料源血緣追溯，可視化整體數據流程架構

(三) 關鍵變數與 KPI 定義

☒ 與業務端共構技巧

- SMART 原則 :
 - Specific (具體) : 定義業務指標 (如「轉換率」非「使用者活躍度」)
 - Measurable (可量化) : 有明確的數據源與計算方式
 - Achievable (可實現) : 不超出當前資料與模型可行範圍
 - Relevant (具相關性) : 對業務價值直接影響 (如營收、流失)
 - Time-bound (有時限) : 設定完成時間與觀察周期

例子 :

- 不佳 KPI 定義 : 「希望模型更準」
- 佳 KPI 定義 : 「於 3 個月內將推播點擊率從 5% 提升至 8%」

☑ KPI 與模型選型連動策略

KPI 項目	對應模型類型	演算法建議與特化模型
Click-Through Rate (CTR)	排名式預測、排序學習	DeepFM, DCN, DIN
客戶流失率 (Churn Rate)	分類任務	XGBoost, CatBoost, LSTM
訂單金額預測	回歸預測	LightGBM Regressor, Prophet
客服應答自動化滿意度	序列到序列任務 (Seq2Seq)	GPT, T5 + RLHF + Feedback
詐欺風險等級分數	分類 + 解釋性需求	TreeSHAP + RuleFit

模型選型需根據 KPI 特性進行專屬優化，例如 Ranking 模型需處理曝光偏差與排序損失。

二、技術應用方案設計 (模型與演算法選型)

(一) 技術應用方案設計 (模型與演算法選型)

1. 任務類型對應的模型選型架構

任務類型	傳統模型	深度學習架構	代表應用範例
分類任務	Logistic Regression, SVM, XGBoost	MLP, CNN, BERT Classifier, TabTransformer	客戶流失預測、信用風險、醫療診斷分類
時序預測	ARIMA, SARIMA, Facebook Prophet	LSTM, GRU, Transformer-TST, TFT	銷售預測、能源負載預測、IoT 故障偵測
結構生成	無 (需仰賴深度架構)	Seq2Seq, T5, BART, Transformer, Diffusion Model	自動摘要、報表生成、SQL 查詢生成
多模態判別	無	CLIP, BLIP, Flamingo, LLaVA, ImageBind	圖文搜尋、影片理解、商品多模態推薦

🔍 模型簡述補充：

- BERT Classifier：預訓練語言模型，常用於句子分類、情感分析、文本匹配。
- TFT (Temporal Fusion Transformer)：
 - 可同時處理靜態與動態時間序列變數，支援多變數預測，廣用於金融與供應鏈預測。
- Diffusion Models：
 - 原用於圖像生成 (如 Stable Diffusion)，現延伸至結構化輸出如程式碼補全與表格填寫。
- CLIP/BLIP/Flamingo：
 - 具備圖像-文字嵌入聯合學習能力，可用於圖片說明生成、圖文語意比對、多模態檢索等。

2.模型選型考量因素

⚙ 資料類型與資料量

分類	說明
結構化資料	表格型資料 (如：客戶資料、銷售紀錄)，建議採用樹模型 (XGBoost、LightGBM)
非結構化資料	文字、圖像、音訊，需用 CNN (影像)、RNN/BERT (文字) 等深度模型
半結構資料	JSON、Log、表格中的嵌套欄位，可用 TabTransformer、AutoGluon 處理
資料量級指導建議	< 1 萬筆：簡化模型 (SVM、LogReg) >10 萬筆：建議使用 GPU + DNN

🔥 即時資料與延遲考量：

- 應用情境：金融風控 (<100ms)、電商推薦 (<50ms)
- 部署建議：
 - 使用 INT8 量化模型 (ONNX、TensorRT)
 - TensorFlow Lite / TorchScript for Edge
 - 將推論移至邊緣設備 (IoT Device / 移動端)

🕒 推理效率與延遲

■ 核心效能指標：

指標	說明	工具
FLOPs	計算量指標，浮點運算數 (越低越快)	Netron, DeepSpeed profiler
Latency (ms)	推論時間延遲	ONNX Benchmark, Triton
Throughput	每秒推論次數 (高流量推薦系統須關注)	TensorRT / TVM

■ 常見模型壓縮方法：

方法	技術描述
Knowledge Distillation	教師模型 → 學生模型，保留精度降低參數 (如 TinyBERT)
Quantization (INT8)	權重從 float32 降為 int8，提升效能約 2-4 倍
Pruning	移除重要性低的神經元，常與量化/蒸餾合併使用

■ 實務建議：

- 使用 HuggingFace optimum-intel or onnxruntime 將 BERT/BLOOM 模型進行 INT8 優化部署。

可擴展性與部署策略

■ 架構設計

架構方式	說明	工具
微服務架構	每一模型或任務為獨立 container，利於 CI/CD	Kubernetes + FastAPI
Serverless	僅按需執行，省資源、彈性大	AWS Lambda / GCP Cloud Function
Edge AI	模型部署在 IoT 裝置或手機端，如影像辨識、AR 應用	TensorFlow Lite, CoreML

■ 多模型系統設計

類型	描述
Model Ensemble	多模型平均/加權投票，常用於 Kaggle/金融風控
Mixture of Experts (MoE)	部分參數激活策略，擴大模型容量但節省推理資源，如 GShard、Switch Transformer

✦ MoE 特點：

- 適用大語言模型與多任務學習
- 推理僅使用部分 expert，節省計算資源
- 有效提升泛化能力

總結建議表：任務、模型與選型關聯一覽

任務類型	資料特性	推薦模型	推理優化策略
客戶流失預測	結構化	XGBoost + SHAP	Pruning, CPU batch
銷售預測	時序 + 數值	TFT, LSTM, Prophet	GPU 加速 + TFLite
圖文推薦	多模態	CLIP + KNN / FAISS Index	ONNX + INT8 + Triton
自動摘要	非結構文字	BART / T5 / GPT2	Distillation + GPU
IoT 故障預測	邊緣即時	GRU / LightGBM	TFLite Edge Deployment

(二) 目標設置與成效評估

1. 多層級目標拆解法 (OKR + TTF 技術導向目標)

☑ 多層級目標設計邏輯：OKR + TTF

模型	說明
OKR (Objectives & Key Results)	設定整體願景與可衡量成果

模型	說明
TTF (Technical Task Framing)	將技術導向目標細分為可執行的開發任務

目標拆解流程 (技術導向範例)

Objective (目標) : 提升模型性能以支援推薦系統業務擴展

KR1 (關鍵成果) : 模型 F1-score 提升至 0.90 以上

→ TTF 子目標對應如下 :

- 處理資料不平衡 (class imbalance)
 - 使用 SMOTE, ADASYN 或加權損失函數
- 模型超參調整 (Hyperparameter Tuning)
 - 使用 Optuna / GridSearchCV
- 特徵工程增強
 - 特徵交叉、嵌入式選擇 (L1 Regularization)
- 模型集成實驗 (如 Voting, Stacking)

☒ 工具整合與實務建議

工具	功能用途
JIRA	技術任務拆解與 Sprint 排程
GitHub Action	自動化執行模型訓練、測試、部署流程
MLflow	實驗追蹤、模型版本與參數記錄管理
Wandb / Notion	團隊可視化實驗進度與決策歷程

實作建議 :

- 每次模型調整應同步產生 MLflow Run，記錄參數、評估指標與備註。
- 用 GitHub PR 標籤 (如 f1>0.9, drift-fixed) 標示實驗目標。

2. 成效評估指標技術對應【維度化展開】

分類模型評估 (模型精度與覆蓋率)

指標類型	技術方法	說明與應用場景	工具
分類能力指標	ROC-AUC, PR-AUC	評估模型排序能力 (排序任務、風控尤重要)	sklearn.metrics, wandb
準確/召回指標	Precision, Recall, F1-score	精確率與召回的平衡 (不平衡資料時尤需關注)	scikit-learn, MLflow, wandb
Confusion Matrix	類別分布視覺化	檢查 FP / FN 分佈是否偏重	seaborn heatmap, wandb logs

☑ 注意事項：

- 避免只看 accuracy，應搭配 Recall（尤其在醫療、風控領域）
- PR-AUC 通常優於 ROC-AUC 在類別極不平衡時提供資訊

📊 商業成效指標（LTV / ROI / 成本）

商業衡量指標	描述與用法	工具建議
LTV (Lifetime Value)	用戶終身貢獻價值，常用於推薦、行銷投資預估	Tableau, PowerBI
ROI (投資報酬率)	模型改善後帶來的收益 / 成本比	Excel, Python Calc
成本效益分析	比較 AI 導入前後的人工/錯誤成本降低	PowerBI 敏感性分析工具

🔴 應用實例：

- AI 客服導入 → 每月節省人工成本 30%
- 精準行銷模型提升 CTR 3%，增加營收 15%

📊 模型穩定性指標（資料與概念漂移）

項目	說明	工具 / 技術建議
Data Drift Detection	特徵分布在部署後是否偏離訓練資料（統計分布檢測）	KS-test, Kolmogorov test
Concept Drift	標籤對應特徵關係變化，如詐欺手法、使用者習慣改變	NannyML、Alibi-detect
特徵重要度變化	比較 SHAP value 或 Gini importance 是否變動	SHAP Tracker、Evidently AI

☑ 監控建議：

- 每週自動追蹤 drift 情況，並設定警戒閾值（如 $\text{psi} > 0.2$ ）
- 可自動產出報告通知 DevOps 或 Retrain pipeline 觸發

三、資源分配（人力、資安、計算資源與 MLOps）

1. 人力組織與角色分工

- AI 專案常見角色：
 - Data Engineer（資料管線建置）
 - ML Engineer（模型開發與部署）
 - MLOps Engineer（持續集成、版本控管、監控）
 - Domain Expert（業務流程轉譯與需求釐清）

2. 資源配置技術工具

- 計算資源排程：KubeFlow, Airflow, Ray, Horovod

- 資料版本控管：DVC (Data Version Control)、LakeFS
- 模型版本與部署：MLflow、Triton Inference Server、Seldon Core

3. 合規與風險控制

- 技術應用：
 - 差分隱私 (Differential Privacy)：保護使用者資料
 - 聯邦學習 (Federated Learning)：跨裝置學習且不共享原始數據
 - XAI (Explainable AI)：強化監管與問責 (如 SHAP、LIME)

L21203 AI 風險管理

◆ AI 風險管理核心框架

核心範疇	說明	關聯技術
風險識別	從資料、模型、應用、供應鏈辨識風險來源	Risk Taxonomy, Threat Modeling
安全性與合規性	包含資訊安全、法規遵循 (如 GDPR、AI Act)	DP, FL, RBAC, Audit Trail
AI 倫理	公平性、偏誤消除、人類尊嚴、歧視風險管理	Fairness-Aware ML, Causal Inference
負責任 AI 治理	包含可解釋性、可追溯性、問責與持續監測	XAI, MLOps, NIST AI RMF

一、風險識別技術

◆ (一) AI 風險類型與來源分類 (AI Risk Taxonomy)

☒ 資料風險 (Data Risks)

風險來源：

- 偏誤資料 (Historical Bias)：如訓練集中女性薪資普遍低 → 模型預測女性適合低薪職。
- 不平衡樣本 (Class Imbalance)：導致過擬合少數類別，尤其在醫療/詐騙檢測中常見。
- 機敏資訊洩露 (PII)：個資可能透過反推模型輸出重建，違反 GDPR/CCPA。

關聯演算法與防範技術：

-  Fairness-Aware ML (如 Reweighting, Adversarial Debiasing)
-  Data Balancing：SMOTE、ADASYN、Random Undersampling
-  Privacy Preservation：差分隱私 (DP-SGD)、去識別化 (De-identification)

☒ 模型風險 (Model Risks)

風險來源：

- 過擬合 (Overfitting)：模型在訓練集上表現極佳但泛化差

- 黑箱模型 (Black-Box Behavior) : 難以理解/解釋其決策邏輯
- 魯棒性不足 (Lack of Robustness) : 容易被 adversarial input 攻擊 (如加入雜訊使模型誤判)

技術與演算法應對 :

- ✦ 可解釋 AI (XAI) : LIME 、 SHAP 、 Integrated Gradients 、 Counterfactual Explanation
- ✦ 魯棒性訓練 (RobustML) : Adversarial Training (FGSM 、 PGD) 、 Certified Robustness (如 AutoLiRPA)

☒ 應用風險 (Deployment Risk)

風險來源 :

- 使用情境錯誤 (如醫療 AI 被用於非臨床場景)
- 過度信任 AI : 用戶可能忽略模型潛在偏誤
- 模型操縱 : Prompt Injection 、 Prompt Hacking 、 Social Engineering

技術風險場景 :

- ✦ 幻覺現象 (Hallucination) : 生成式 AI 編造資訊 (如 ChatGPT 生成虛構資料)
- ✦ RAG Failure (Retrieval Augmented Generation 失效) : 外部知識檢索落差導致錯誤回應

對策 :

- Prompt Guardrails (Prompt Layer 審核)
- 自動事實檢查 (Fact-Checking API 、 LLM-based QA Assurance)
- RAG 驗證模型 (使用 Embedding Similarity + Groundedness Score)

☒ 供應鏈風險 (Supply Chain Risk)

風險來源 :

- 開源模型汙染 (Supply Chain Attack) : 如開源 repo 植入惡意後門
- Model Dependency Risks : 使用來自第三方不明品質模型或 dataset

解決技術與治理工具 :

- ✦ SBOM (Software Bill of Materials) : 紀錄 AI 模型建構過程中用到的套件、data、權限、依賴鏈
- ✦ AI Software Supply Chain Security Tools :
 - Model Card++ (模型說明)
 - Provenance Tracker (如 MLflow Tracking + Data Lineage)

◆ (二)威脅建模技術 (Threat Modeling for AI)

☒ STRIDE 威脅分類分析 (Microsoft 定義)

類型	說明與 AI 範例
Spoofing	模仿使用者 → Deepfake 生成身份欺騙
Tampering	攻擊訓練資料 → Data Poisoning, Label Flip
Repudiation	無法追溯決策來源 → 缺乏 Audit Trail
Information Disclosure	模型洩漏資料 → Model Inversion, Membership Inference
Denial of Service	模型無法處理過量請求 → Prompt Flooding
Elevation of Privilege	對 AI prompt 控制權提升 → Jailbreaking LLMs

✦ 補充工具：Microsoft Threat Modeling Tool + AI-specific STRIDE 模板

☒ AI Attack Tree 建模 (AI-Specific Threat Tree)

建構原則：

1. 明確標出攻擊者目標 (如操縱預測結果)
2. 使用樹狀結構展開攻擊手法
3. 結合資安攻擊鏈如 Kill Chain：Recon → Delivery → Exploit → Impact

常見攻擊類型：

- ▲ 對抗樣本生成 (Adversarial Examples)：FGSM、Carlini-Wagner
- ▲ 模型逆向 (Model Extraction)：用 API 回應建模
- ▲ 模型中毒 (Data Poisoning)：控制訓練資料造成分類錯誤

☒ CVSS + AI 擴展評分系統

CVSS (Common Vulnerability Scoring System) 基本架構：

- 基本分數 (Base) + 時效性分數 (Temporal) + 環境分數 (Environmental)

AI 擴展建議：

- 引入 AI-specific 欄位：
 - ✦ Sensitivity of AI Output (如生成有攻擊性語句)
 - ✦ Model Exposure Level (API 開放 vs. 本地封裝)
 - ✦ Input Sensitivity Range (極小輸入改變是否會造成極大錯誤)

工具例子：

- MITRE ATLAS (Adversarial Threat Landscape for AI Systems)
- NIST AI RMF Threat Characterization

◆ (三) 模型風險評估指標 (Model Risk Evaluation Metrics)

指標 詳細說明與技術解析

☒ Accuracy Parity (準確率公平)

- 衡量不同子群 (如男性/女性) 的分類準確率是否一致。
- 例如：女性樣本預測準確率 89%、男性 72% → 存在偏誤。

解法：

- 重加權 (Reweighting)、公平性正則化項 (如 Fairness Constraint)

☒ Demographic Parity (人口統計平衡)

- 預測結果的機率應該與群體比例無關。
- 例如：若全體女性佔 50%，則模型預測為“適任”的女性也應該佔 50%。

技術實作：

- Adversarial Debiasing：利用對抗訓練防止模型學會人口特徵
- Prejudice Remover Regularizer：直接修改 loss function 結構

☒ Calibration (模型校準性)

- 預測為 $P=0.8$ 的樣本實際有 80% 為正例 → 模型具備良好校準。
- Poor calibration → 高信心錯誤預測 (風險大)

工具與方法：

- Reliability Diagram (信心 vs. 準確率)
- Platt Scaling、Isotonic Regression (後處理校準法)

☒ Robustness to Perturbation (對擾動的穩定性)

- 輕微輸入 (如圖片加 0.01 雜訊) 是否會導致大幅誤判？

技術對策：

- 對抗訓練 (如 PGD)
- Certified Robustness Toolkits (如 CROWN、CLEVER)

二、AI 安全性與合規技術

◆ (一) 資料隱私保護 (Privacy Protection)

☒ 差分隱私 (Differential Privacy, DP)

定義：差分隱私是一種統計技術，確保資料集中某一筆個資是否存在不會顯著影響最終模型輸出，藉此防止逆推攻擊 (如 membership inference)。

◆ 兩大機制：

機制名稱	說明與應用
Laplace Mechanism	用於標量輸出 (如統計報表)，基於 L1 敏感度加入拉普拉斯雜訊。公式： $output + Lap(\Delta f/\epsilon)$
Gaussian Mechanism	用於實值向量輸出，基於 L2 敏感度加入高斯雜訊，允許 (ϵ, δ) -差分隱私，適用於機器學習梯度。

◆ 應用技術：

- DP-SGD (Differentially Private Stochastic Gradient Descent)

- 對每個 mini-batch 的梯度：
 1. 梯度裁剪 (Clip gradients)
 2. 加入高斯雜訊
- 常用於私密醫療資料、金融資料的模型訓練

✦ 框架與工具：

- Google's TensorFlow Privacy
- Opacus (PyTorch DP 套件)

☑ 聯邦學習 (Federated Learning, FL)

定義：聯邦學習允許在不集中數據的情況下，於多個設備端進行本地訓練，透過中心伺服器進行模型聚合，保護原始資料隱私。

✦ 類型：

類型	說明	應用
Horizontal FL	特徵空間相同，樣本不同 (如多醫院)	聯合醫療診斷模型
Vertical FL	特徵不同但樣本重疊 (如銀行 + 電商)	信用風控模型
Transfer FL	特徵和樣本均不同	跨國通用模型訓練

✦ 演算法比較：

名稱	核心邏輯	特點
FedAvg	取平均權重更新	標準方法，效率高
FedProx	增加正則項對抗異質資料	解決非 IID 資料問題
FedNova	解耦本地計算差異與學習率	適應多節點環境

✦ 工具框架：

- TensorFlow Federated (TFF)
- PySyft (OpenMined)

☑ 同態加密 (Homomorphic Encryption)

定義：讓資料在加密狀態下仍能執行加法與乘法等運算，最終解密得出與原始結果相同的運算結果。

✦ 常見加密類型：

型別	說明	底層算法
CKKS	支援近似加法與乘法，適用於深度學習浮點計算	支援複數/浮點數
BFV	精確整數運算	適合精確金融模型

✦ 應用場景：

- 加密推薦系統

- 醫療影像辨識 (雲端推論)
- 金融 AI 服務 (保密模型評分)

◆ (二) 安全性設計與模型強化技術

☒ 對抗訓練 (Adversarial Training)

定義：主動生成敵意樣本加入訓練集，以提高模型抵禦 adversarial attacks 的能力。

✦ 常見對抗演算法：

名稱	原理	特點
FGSM (Fast Gradient Sign Method)	利用 loss 對輸入求導，加入微小方向擾動	計算快，攻擊效果強
PGD (Projected Gradient Descent)	多次 FGSM 攻擊並投影回合法區間	攻擊力更強，訓練更穩定
TRADES	損失函數加入 adversarial risk trade-off	強化對抗與標準精度間平衡

✦ 框架：

- Adversarial Robustness Toolbox (IBM)
- CleverHans (TensorFlow)

☒ 模型驗證與形式化分析

目的：對深度學習模型進行安全性或邏輯正確性的形式證明。

✦ 工具與方法：

工具	原理	特點
Reluplex	符號求解器，針對 ReLU 網絡進行 SAT/SMT 驗證	證明分類器在特定區域不變
Verivital	形式驗證 + 抽象解釋，針對自駕車控制系統	通過訓練時直接導入限制約束

☒ 模型水印與來源追溯 (Model Watermarking)

目標：防止模型被盜用或提供溯源能力

✦ 技術方法：

名稱	原理	功能
DeepSigns	在權重/激活中嵌入訊號	可檢驗權限模型是否遭竊
MIP (Model Inversion-based Watermarking)	重建輸入訊號辨識模型出處	用於供應鏈驗證與版權維權

◆ (三) 合規技術與監理工具 (Compliance & Regulatory Mapping)

☒ GDPR / CCPA (資料保護法)

-  Data Lineage (資料沿革追蹤)
 - 追蹤資料源頭、轉換流程與模型使用關聯
 - 實作工具：Apache Atlas, Amundsen
-  刪除請求自動化 (Right to Erasure)
 - 應用技術：Machine Unlearning (如 SISA, Sharded SGD)
 - Model Update without Retraining (撐控成本)

☒ EU AI Act (人工智慧法)

-  High-Risk AI Documentation Kit
 - 包含：
 - 模型風險說明文件 (Model Card++)
 - 目標用戶、使用情境與監督流程
 - Fairness / Bias audit report
 - 對應：Annex III 高風險應用 (醫療、教育、司法、信用評估等)

☒ 金融監管 (例如巴塞爾 AI 模型風險、台灣金管會指引)

-  Explainable Risk Models
 - 對模型結果提供局部解釋 (SHAP)、群體公平分析 (Equalized Odds)
 - 避免模型不具可解釋性造成法規風險
-  Model Registry (模型註冊系統)
 - 管理模型版本、訓練數據來源、訓練參數與使用單位
 - 可用平台：MLflow Registry, Amazon SageMaker Model Registry

三、AI 倫理技術

◆ (一) 公平性演算法 (Fairness-Aware Machine Learning)

■ 公平性演算法 (Fairness-Aware Learning) 完整技術展開

◆ 公平性處理三大類別分類概觀

類別	定義	演算法代表	適用場景
Pre-processing	在模型訓練前處理資料或特徵	Reweighting, Disparate Impact Remover	不可修改模型結構或使用黑箱模型時
In-processing	修改模型訓練過程與目標函數	Adversarial Debiasing, Prejudice Remover	可控制模型訓練流程
Post-processing	在模型訓練完成後調整預測輸出	Equalized Odds, Calibrated Equalized Odds	不可重訓模型或資料不公開

◆ 1. Pre-processing 公平性演算法

☒ Reweighting

核心目標：對不同群體的樣本重新加權，降低資料中的不平等表示 (representation bias)

技術邏輯：

令樣本屬性：

- \$A\$: 敏感變數 (如性別)
- \$Y\$: 標籤 (如是否錄取)

Reweighting factor 為：

$$w(A = a, Y = y) = \frac{P(Y = y) \cdot P(A = a)}{P(Y = y, A = a)}$$

步驟：

1. 根據樣本的群體屬性計算聯合機率與邊際機率
2. 為每筆資料分配一個補償權重
3. 在模型訓練時根據權重最小化 loss

優點：

- 可與任何機器學習模型結合
- 不改變特徵空間與模型架構

缺點：

- 仍可能留下結構性偏誤
- 對小樣本群體效果有限

☒ Disparate Impact Remover

目標：修改資料特徵，使其與敏感變數的相關性下降，但仍保留可預測性

方法：

- 對數值特徵進行正規化與映射
- 儘可能讓資料分布在不同群體間趨近一致

數學定義：

$$\text{Disparate Impact (DI)} = \frac{P(\hat{Y} = 1 | A = 0)}{P(\hat{Y} = 1 | A = 1)}$$

→ 若 $DI < 0.8$ ，視為存在不公平

使用案例：

- 金融信貸評分：讓不同種族背景的信用特徵不導致明顯差異化評分
- 員工甄選：避免性別導致評分差異

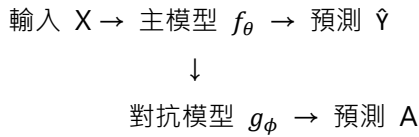
◆ 2. In-processing 公平性演算法

☒ Adversarial Debiasing

核心概念：結合 adversarial learning 架構，引入對抗分類器來預測敏感屬性，迫使主模型

去學習與敏感屬性無關的特徵表示。

架構圖：



損失函數：

$$\min_{\theta} \max_{\phi} L_{pred}(f_{\theta}(x), y) - \lambda L_{adv}(g_{\phi}(f_{\theta}(x)), a)$$

- L_{pred} ：主任務損失（如交叉熵）
- L_{adv} ：對抗任務損失（如預測性別）
- λ ：權重調整公平性與精度的權衡

實作建議：

- 在神經網絡中加入梯度反轉層（GRL）實作 adversarial branch
- 使用 PyTorch / TensorFlow 皆可實現

☒ Prejudice Remover Regularizer

邏輯：修改原始損失函數，加入一項正則化項來最小化敏感變數與預測結果的關聯性。

公式：

$$L_{total} = L_{original} + \beta \cdot \text{Prejudice Index}$$

Prejudice Index (PI) 定義：以 *mutual information* ($\hat{Y}; A$) 作為衡量

應用：

- 適用於 logistic regression、neural network
- 特別適合監督式分類問題

優點：

- 模型內部公平性學習，適應性高

缺點：

- 需要能修改訓練過程，無法應用於封閉模型

◆ 3. Post-processing 公平性演算法

☒ Equalized Odds Postprocessing

公平性定義：讓不同群體在相同真實標籤下，模型預測為正的機率一致。

$$P(\hat{Y} = 1 | Y = y, A = 0) = P(\hat{Y} = 1 | Y = y, A = 1), \forall y \in \{0,1\}$$

實作方法：

- 根據混淆矩陣調整每個群體的預測機率
- 解線性規劃問題以找到最佳後處理策略

使用情境：

- 應用在不允許更動資料或模型的情況下（如部署後調整）
- 模型為黑箱 API（無法 retrain）

☒ Calibrated Equalized Odds

目標：

在維持 Equalized Odds 的同時保留原始模型的機率校準性 (calibration)

技術邏輯：

- 使用 calibrated score 取代硬分類
- 將每個群體的 ROC 曲線作轉換，保持 precision-recall 平衡

應用場景：

- 需要機率預測做風險排序 (如保險費率、貸款排序)
- 不希望破壞模型可信度與置信度

☒ 演算法比較表

演算法	分類	是否需 retrain	適用模型	優勢	限制
Reweighting	Pre-processing	否	任意	不需模型改動	無法處理模型內部偏見
Disparate Impact Remover	Pre-processing	否	任意	特徵調整自動化	可能影響模型精度
Adversarial Debiasing	In-processing	是	NN	深層表示公平性學習	訓練複雜、效能需權衡
Prejudice Remover	In-processing	是	多種	可調參公平性強度	不支援預訓練黑箱模型
Equalized Odds	Post-processing	否	任意	不需 retrain、法遵友善	損失原始校準、僅能調整輸出
Calibrated Eq. Odds	Post-processing	否	任意	平衡公平與置信度	較高實作門檻、效果依賴數據

☒ 補充：實作資源推薦

工具 / 套件	說明
IBM AIF360	支援超過 20 種公平性演算法與指標
Fairlearn (MS)	Python 套件，易整合 scikit-learn
What-If Tool (Google)	TensorBoard 外掛，視覺化公平性分析
AI Fairness 360 Notebooks	現成示例與數據集，支援 Reweighting、Equalized Odds 等

☒ 總結：公平性處理路徑建議

- 若只能修改資料：使用 Reweighting / Disparate Impact Remover
- 若能修改訓練流程：使用 Adversarial Debiasing / Prejudice Remover
- 模型已部署但需調整公平性：使用 Equalized Odds / Calibrated EO

- 需高階對因果公平性建模：結合 Causal Graph 與 Counterfactual Estimation (見上一講義)

四、負責任 AI 技術

◆ (一)可解釋性技術 (Explainable AI, XAI)

☑ 核心目標：

- 使 AI 系統的決策過程「可被理解、可被質疑、可被回溯」
- 滿足 法規遵循 (如 GDPR "right to explanation") 與商業透明度要求

☑ A. 黑箱模型的可解釋技術

這類技術可應用於任意預測模型 (CNN、Transformer、XGBoost 等)，不需修改其內部結構，屬於**模型不可知 (model-agnostic) **類型。

✦ 1. LIME (Local Interpretable Model-agnostic Explanations)

原理：

- 對輸入進行隨機擾動，觀察模型預測結果如何變化
- 建立一個局部線性模型來解釋黑箱模型在這個點附近的行為

步驟：

1. 擾動原始輸入生成新樣本
2. 計算這些樣本對應的模型預測
3. 擬合一個稀疏線性模型
4. 解釋變數對結果的貢獻

✦ 2. SHAP (SHapley Additive exPlanations)

數學基礎：賽局理論中的 Shapley Value

每個特徵對模型預測貢獻的平均邊際值

特性：

- Additivity：各特徵貢獻加總等於模型預測值
- Consistency：增加一個特徵的影響力不會減少其 SHAP 值

應用：

- 較適用於 tree-based models (如 XGBoost, LightGBM)
- SHAP Interaction Values 可解釋特徵交互作用

☑ B. 結構模型內建可解釋性

✦ 1. Rule-based Models (決策規則模型)

- Decision Tree、Rule List、Bayesian Rule Set
- 輸出可視為「if-then」規則集，適合醫療/法律等強需求場景

✦ 2. Prototype Networks (原型學習)

- 利用距離最近的已知實例解釋模型決策 (如圖像分類中呈現類似圖片)
- 代表模型根據哪些 “記憶” 進行推理

☒ C. 視覺模型專用 XAI 技術

技術	概念	使用說明
Grad-CAM	計算最後卷積層對預測的梯度	生成熱圖 (highlight) 關鍵區域
Guided Backpropagation	將輸出訊號反向傳回輸入	可視化像素對結果的貢獻度

◆ (二) 可追溯性與問責技術 (Traceability & Accountability)

☒ 目標：

- 追蹤模型從資料、訓練、評估到部署的每一個環節
- 確保模型責任可界定、變更可控、行為可審查

☒ A. Audit Trail for ML Pipelines

實踐重點：

- 建立完整模型生命週期 (ML Lifecycle) 記錄
- 包含資料版本、模型參數、訓練配置、測試集表現、部署紀錄

代表工具：

工具	功能	特點
MLflow Tracking	追蹤實驗參數、評估指標、artifact	支援自動化與版本控制
DVC (Data Version Control)	資料、模型與代碼的 git-style 管理	支援 CI/CD 整合
Neptune.ai	圖形化實驗儀表板 + 多人協作追溯	可接入 PyTorch、Keras 等主流框架

☒ B. Model Registry (模型註冊中心)

功能特點：

- 模型元數據登記 (如版本、開發者、用途)
- 部署版本記錄 + 撤銷管理
- 支援模型簽名與驗證

實作平台：

- Amazon SageMaker Model Registry
- MLflow Registry
- Azure ML Model Management

- ☒ C. 使用者互動與回溯機制 (End-user Feedback Logging)
- 使用者可標記錯誤預測、提出申訴或修正建議
 - 系統自動記錄錯誤回饋與模型輸出
 - 支援錯誤熱點回報、自動再訓練 (如 Active Learning Loop)

◆ (三) MLOps 與部署監控 (Responsible MLOps)

☒ A. Drift Detection (資料偏移監測)

類型：

- Data Drift：資料分布與訓練資料不同
- Concept Drift：標籤的含義或關聯發生改變 (如詐騙樣態演進)

監測工具：

工具	功能特色
Evidently AI	自動比較訓練與生產資料的分布、統計指標
NannyML	支援無標籤資料監控模型穩定性與效能衰退
WhyLogs	高效能 streaming logs + 分布比較 + alert system

☒ B. 部署安全性 (Deployment Controls)

技術策略	功能
Shadow Deployment	不干擾用戶的情況下收集回饋
Canary Release	小比例用戶先用新模型
Rollback Support	支援版本快速回滾

☒ C. AI 事件回報與紀錄機制 (Incident Reporting)

代表制度與實務規範：

- NIST AI RMF - AI Incident Handling Submodule
- PARTA (Public AI Risk Transparency Architecture)
- 企業內部通報 (如 AI Escalation Protocol)

 應用場景整合對照表

元件類型	技術實作	應用場景
XAI 解釋技術	SHAP, LIME, Grad-CAM	醫療決策、金融信用評估
追溯機制	MLflow + DVC + Registry	金融模型監理、政府治理
偏移監測	Evidently + NannyML	詐騙行為偵測、氣候預測系統
問責處理	Incident Logging + 人機對話記錄	聊天機器人誤導、Deepfake 檢舉

五、可信 AI 安全標準與實作框架

◆ (一) NIST AI Risk Management Framework (NIST AI RMF 1.0)

✳ 出處與目的：

美國國家標準與技術研究院 (NIST) 於 2023 年發佈，目的是協助企業與組織減緩 AI 系統導致的風險，並增進 AI 的可信任性、安全性、公平性與問責性。

☑ 核心功能解析

功能	說明	工具 / 方法
MAP	發現與識別 AI 風險	AI 項目風險評估矩陣、威脅建模
MEASURE	評估風險影響與可信度屬性	Bias Metric、Fairness Audit、Evidently AI
MANAGE	持續控制風險並實施修正	CI/CD 插入風險閘道、差分隱私、模型監控
GOVERN	設立治理政策、確保組織一致性	AI 治理委員會、模型卡 (Model Card)

✳ 技術實作建議：

可信特性	技術工具 / 實踐方式
有效性 (Accuracy)	測試覆蓋率、模型回溯驗證、Golden dataset
公平性 (Fairness)	AIF360、Fairlearn、差別影響指標
可解釋性 (Explainability)	SHAP, LIME, Captum
隱私 (Privacy)	差分隱私、聯邦學習
安全性 (Robustness)	對抗樣本測試 (FGSM, PGD)、異常偵測
問責性 (Accountability)	模型卡、訓練紀錄追蹤 (MLflow)

◆ (二) ISO/IEC 42001：AI 管理系統標準 (AIMS)

■ 簡介

ISO/IEC 42001 是全球首部 AI 專用管理系統國際標準，由 ISO 在 2023 年底發佈，適用於所有使用、部署或開發 AI 的組織，與 ISO 27001 類似，是可認證的「AI 管理制度」標準。

📁 架構與特色

模組	說明
AIMS (AI Management System)	為 AI 建立治理、風險、合規、品質、倫理等制度
高階管理責任 (Leadership)	要求組織高階層參與 AI 目標與風險策略制定
風險與倫理框架整合	與 NIST RMF、ISO 31000、UNESCO AI Ethics 對齊
可稽核性與文件紀錄要求	對模型開發歷程、使用情境、測試結果皆需有紀錄

☑ ISO 42001 對 AI 系統的具體要求

管理面向	要求示例
資料管理	資料來源、標註品質、匿名化流程記錄
模型生命週期紀錄	模型訓練參數、資料版本、測試準確度報告
部署監控	設置 KPI，持續監控偏差、漂移
AI 風險應變計畫	定期模擬模型誤用、失誤風險應變措施
外部合規對接	GDPR、歐盟 AI Act、國家資訊安全法等對應表

◆ (三) 實務導入建議與對應實作

需求面向	建議工具 / 技術	實施說明
模型治理	MLflow, Model Card Toolkit	建立可回溯模型訓練/評估紀錄
資料治理	Data Catalog (Amundsen, DataHub)	追蹤欄位來源、稽核匿名化紀錄
偏差與風險監控	Fairlearn, NannyML, Presidio	實時分析資料 / 模型行為風險
隱私合規	OpenDP, SmartNoise	差分隱私生成機制落地
文件化與標準驗證	OpenAI Governance, ISO 42001 稽核報告框架	作為外部稽核證據與企業治理報告書基礎

📊 統整對照表：NIST AI RMF × ISO 42001

對應面向	NIST AI RMF 功能	ISO/IEC 42001 要求
風險發現	MAP	風險評估政策、風險庫建置
效能與可信分析	MEASURE	模型驗證報告制度
持續控管	MANAGE	實作措施、應變計畫
組織層治理	GOVERN	AIMS 架構、高階參與證明
監控與稽核	Trust Measures	文件化、記錄儲存、外部認證

六、AI 審計與可責任化機制設計

■ (一) 核心定義與目標

名稱	定義
AI 審計 (AI Audit)	一種系統性過程，用來檢查 AI 系統在開發、部署與使用過程中是否符合預定的倫理、安全、合規與治理標準。
可責任化	建立可追蹤、可驗證、可懲罰 / 可補救的 AI 運作機制，明確

名稱	定義
(Accountability)	「誰」對「什麼」負責。

☑ 國際標準基準來源：NIST AI RMF、ISO/IEC 42001、OECD AI Principles、EU AI Act

(二) AI 審計應包含哪些層面？

面向	審計項目	工具 / 技術建議
資料審計	資料來源與標註記錄、資料偏差檢測、合法性 (GDPR)	datasheets for datasets、Great Expectations
模型審計	訓練與驗證流程記錄、準確度與穩健性、漂移偵測	MLflow, SHAP, LIME, Captum
公平性與偏差	性別 / 種族 / 年齡 / 地域偏見檢測	Fairlearn, AIF360, Disparate Impact
隱私與安全性	差分隱私、模型攻擊測試 (對抗樣本、反向推理)	SmartNoise, ART (Adversarial Robustness Toolbox)
使用情境與風險	是否清楚限定模型使用情境、存在濫用風險？	Model Card, Risk Scenario Matrix
行為紀錄與審計追蹤	推論輸入/輸出記錄、使用者行為記錄、改版日誌	Audit Trail System, Loki, ElasticSearch
人責設計 (人類可介入性)	模型能否接受人工干預？是否存在誤判復原機制？	Human-in-the-Loop 介面設計

(三) 可責任化機制架構設計 (Accountability Architecture)

設計原則

- 行為可追蹤 (Traceability)
 - 所有模型決策均可追溯至資料來源、演算法版本、開發者行為
- 責任分配明確 (Responsibility Attribution)
 - 為每個 AI 行為建立責任矩陣：開發者、部署者、使用者
- 治理程序透明 (Transparency)
 - 模型卡、審計報告、用途說明公開化 (模型開放或白箱解釋)
- 補救與復原流程 (Remediability)
 - 錯誤預測是否能補救？有無申訴 / 人工覆核通道？

(四) AI 審計作業流程建議 (Audit Lifecycle)

階段	作業內容	工具 / 作法
1. 界定範圍	確認審計模型、資料、用途範圍	定義 AI 使用邊界、風險層級
2. 資料與模型蒐證	收集資料來源、訓練紀錄、測試報告	MLflow + Git versioning
3. 執行審查分析	使用偏差偵測、說明性測試、錯誤分析工具	SHAP, LIME, AIF360
4. 建立模型報告	撰寫模型卡、風險卡、行為報告	Model Card Toolkit
5. 問責與通報	異常行為回報、合規風險上報機制	人工覆核 + 風險等級回報表
6. 矯正與回訓	針對問題模型進行 retraining、策略修正	Auto retraining pipeline + CI/CD

(五) 實用工具與範本資源

名稱	說明	連結
Model Card Toolkit	Google 開源，生成模型卡報告	GitHub
OpenAI System Card	用於大型語言模型的責任報告書架構	ChatGPT System Card
Audit-AI	比較模型結果是否存在不公平偏差	GitHub
Microsoft Responsible AI Dashboard	整合解釋性、偏差、因果分析與錯誤分析	Azure SDK 工具組
LLM Auditing Playbook	AI Now / Mozilla 提出，針對 LLM 審計建議	PDF

(六) 實務應用案例參考（實體導入）

組織 / 企業	審計對象	機制亮點
OpenAI	GPT 系列 LLM	公開 system card、說明性限制與偏差分類
Google	Med-PaLM	醫療語言模型進行多層級問責 + 臨床覆核
IBM Watson	NLP Chatbot	模型行為與業務規則審核自動化
歐盟 AI Act	所有高風險系統	要求事前風險報告、用途範圍備查與年審制度

(七) 統整建議：建立 AI 審計與責任化 SOP 清單

類型	審計項目	說明文件或產出物
資料	標註與分布審計	dataset_card.json, DVC metadata

類型	審計項目	說明文件或產出物
模型	訓練歷程與公平性測試報告	model_card.md, SHAP image
使用風險	系統使用邊界與責任者登記	use-case register, responsibility matrix
事後稽核	行為追蹤與問題修正紀錄	audit_log.csv, correction_plan.pdf

L21301 數據準備與模型選擇

一、AI 應用情境分析與技術選擇

🔍 (一) AI 應用需求分析 (需求工程與目標技術對齊)

▶ 需求識別技術

● 問題導向式分析 (Problem-Driven Design, PDD)

定義：

從實際問題出發，透過問題定義 → 根因分析 → 技術對齊，來確認 AI 是否適用與如何適用。

步驟：

1. 問題界定 (Problem Framing)：明確定義痛點或瓶頸。
2. 根因分析 (Root Cause Analysis)：5 Why、Ishikawa Diagram。
3. 可量化需求轉換：將問題轉為可觀測變數 (如「客戶流失」→ 預測 churn rate)。

應用案例：

- 醫療業 → 預測急診再入院率以提升床位調度效率
- 金融業 → 檢測可疑交易模式以防止洗錢

● 任務拆解樹 (Task Tree Formulation)

定義：

透過分層方式將高階任務拆解成子任務，逐步對應合適的 AI 模型與資料需求。

技術結構：

主任務：提升客服回應效率

|

├─ 子任務 A：自動回應常見問題 → 文本分類模型

├─ 子任務 B：識別緊急程度 → 多類分類 + 關鍵詞提取

└─ 子任務 C：自動轉接至適當單位 → 意圖識別模型

工具輔助：

- 標準任務樹建構工具：BPMN、Mermaid.js
- 搭配業務流程管理 (BPM) 框架使用

● 商業目標對應指標 (Business Goal Alignment)

目標與技術連結方式：

商業目標	對應技術	關鍵指標
提升營收	產品推薦系統	CTR、Conversion Rate
降低成本	自動化質檢	缺陷檢出率、False Positive
增加顧客留存	預測客戶流失	Churn Precision / Recall
增強用戶體驗	自然語言生成回應	BLEU、ROUGE、UX 評估

常用指標：

- ROI (Return on Investment) : $\frac{\text{收益提升} - \text{成本}}{\text{投入成本}}$
- LTV (Lifetime Value) : 預測單一客戶在生命週期中的總貢獻
- TCO (Total Cost of Ownership) : 含硬體、模型訓練、維運成本

⚙️ (二) 模型選擇與技術考量 (Model Selection & Alignment)

▶ 模型類型分類 (依任務需求配適)

● 監督式學習 (Supervised Learning)

- 分類 (Classification) :
 - 二元分類 (Binary) : 是否違約、是否癌變
 - 多類分類 (Multiclass) : 郵件類別、圖片標籤
 - 常用模型 : Logistic Regression、Random Forest、XGBoost、BERT (文本)
- 回歸 (Regression) :
 - 連續型數值預測 : 銷售額、股價
 - 常用模型 : Linear Regression、Lasso/Ridge、LightGBM、DNN

● 非監督式學習 (Unsupervised Learning)

- 分群 (Clustering) :
 - 顧客分群、行為型態識別
 - 演算法 : K-Means、DBSCAN、Gaussian Mixture
- 降維 (Dimensionality Reduction) :
 - 用於資料視覺化、噪聲過濾、特徵選擇
 - 技術 : PCA、t-SNE、UMAP、Autoencoder

● 強化學習 (Reinforcement Learning)

- 策略學習 (Policy Learning) :
 - 應用 : 動態定價、自駕車、遊戲 AI
 - 架構 : Policy Gradient, PPO, A3C
- 價值函數估計 (Value Function Approximation) :

- 評估不同動作價值，用於策略提升
- 技術：Q-Learning, DQN (Deep Q Network)

▣ 模型選擇考量因子 (綜合性評估維度)

評估維度	說明	技術影響舉例
資料特性	結構化、非結構化、類別數量、樣本不平衡、噪聲比例	圖像：CNN；文字：Transformer；樣本小：Few-shot/Transfer Learning
成效需求	精度、召回率、可解釋性、推理速度	可解釋性需求高選用 Logistic/Decision Tree；即時應用選用 ONNX、量化模型
實作限制	資源成本、部署平台、維護頻率	雲端 (如 SageMaker) vs 邊緣部署 (需壓縮：Quantization / Distillation)

✳ 補充：模型部署策略對照

部署方式	描述	適用模型與框架
雲端部署	彈性大、運算強	HuggingFace + AWS, TensorFlow Serving
邊緣設備	時延低、即時性高	TFLite, ONNX, NVIDIA TensorRT
混合式	雲端訓練 + 邊緣推論	蒐集行為資料於雲端再回傳模型更新

二、數據收集與資料前處理

DataOps 是資料工程與資料科學交集下的重要模組，目的在於建構可重複、自動化且可持續監控的資料流通與處理流程，支撐 AI 模型生命週期的高品質資料來源。

🔍 (一) 數據收集策略 (Data Acquisition)

☑ 來源分類與技術

資料來源類型	來源實例	技術方法
結構化資料 (Structured)	資料庫 (MySQL、PostgreSQL)、ERP、CRM	JDBC/ODBC API、ETL 工具 (如 Apache NiFi、Airbyte)
非結構化資料 (Unstructured)	文本、圖片、影片、音訊	網路爬蟲 (Scrapy)、OCR、Speech-to-Text
半結構化資料 (Semi-structured)	JSON、XML、log、Sensor data	API 擷取、Log parser (Fluentd、Logstash)
開放資料 / 標準語料	Kaggle、UCI ML、HuggingFace Datasets	CLI 抓取、自動同步腳本 (wget/curl)

☒ 擷取模式 (Streaming vs Batch)

模式	適用場景	工具與技術
批次擷取 (Batch)	銷售歷史、快照分析	Apache Airflow、SSIS
串流擷取 (Streaming)	IoT 資料、即時風控	Apache Kafka、AWS Kinesis、Spark Streaming

☒ 實務注意事項：

- 權限與合規性審查 (Data Governance)：GDPR、資料共享合約、API 授權
- API 擷取限制控管：Rate Limiting、Retry 機制、Token 更新

💡 (二) 數據清洗與品質管控 (Data Cleaning)

資料清洗約佔整體 AI 專案時間 60–70%，是提升模型表現的核心關鍵之一。

☒ 常見問題與對應處理技術：

問題類型	描述	解決方法與工具
缺失值 (Missing Values)	欄位空值、NULL	插補 (均值、中位數、KNN、MICE)、刪除樣本
異常值 (Outliers)	資料偏離常態	IQR、Z-score、Isolation Forest、Boxplot 分析
重複資料 (Duplicates)	多筆相同記錄	Pandas drop_duplicates()
錯誤格式 (Incorrect Types)	日期格式混亂、錯誤編碼	Regex 修正、型別轉換 (astype())
資料不一致 (Inconsistency)	地區拼音不一、標籤重疊	Mapping 標準化、轉換函數
噪聲資料 (Noise)	訊號內無意義擾動	Smoothing (滑動平均、加權平均)、Wavelet Filtering

✂ (三) 資料轉換與正規化 (Data Transformation)

資料需轉換為 AI 模型能處理的格式，例如數值化、標準化等。

☒ 常見轉換技術

方法	適用資料	工具函式
數值標準化 (Z-score, Min-Max)	數值資料、神經網路訓練	Scikit-learn：StandardScaler, MinMaxScaler
類別編碼 (One-hot, Label, Target)	類別型欄位	pd.get_dummies()、LabelEncoder, CategoryEncoders

方法	適用資料	工具函式
日期時間拆解	時間欄位	年、月、週、是否假日等欄位
數據型態轉換	Text → Int、Float	Numpy, Pandas

☒ 特殊類型轉換

- 文本資料：
 - Tokenization → Vectorization (TF-IDF、Word2Vec、BERT Embedding)
- 圖像資料：
 - Resize → Normalize (像素值標準化) → Tensor 化 (PIL → Tensor)
- 時間序列：
 - 移動平均、差分 (Differencing)、窗口擴展 (Rolling features)

■ (四) 資料標註與校驗 (Annotation & Label Validation)

適用於監督式學習，標註品質直接決定模型學習效能。

☒ 標註方式與技術

類型	描述	工具/框架
人工標註	人工介面輸入	Label Studio, doccano
半自動標註	弱監督 / 規則 / 模型預標	Snorkel, Prodigy
眾包平台	大量樣本快速標註	Amazon MTurk, Scale AI
主動學習 (Active Learning)	優先挑選最具不確定樣本	modAL, ALiPy (使用不確定性選樣)

☒ 資料驗證機制

- 標註一致性分析 (Inter-annotator Agreement) → Cohen's Kappa
- Label Leakage 檢查：目標變數是否在特徵欄位中
- 資料版本控管：DVC, Weights & Biases (W&B Artifacts)

🔄 (五) 自動化與資料工作流程管理

☒ 工具鏈組合

功能	工具
數據 workflow 排程	Apache Airflow, Prefect
資料追蹤與版本控制	DVC, LakeFS, DeltaLake
數據異常檢測	Great Expectations, SodaSQL
監控與紀錄	DataDog, MLflow, W&B

💡 實務應用案例

🚀 案例：智慧製造缺陷檢測 AI 專案

步驟	技術應用
感測資料串流	OPC-UA + Kafka
異常點位清洗	Z-score 異常移除
特徵擷取	Rolling Average, FFT
標註樣本	半自動標註 + QA 工程師審核
自動化流程	Airflow 每日排程擷取與模型更新

三、特徵工程

🔍 (一) 什麼是特徵工程？

特徵工程是指將原始資料轉換為機器學習模型能有效學習的資訊表示方式，其涵蓋：

- 特徵轉換 (Transformation)
- 特徵選擇 (Selection)
- 特徵構建 (Construction)
- 特徵擴展與壓縮 (Expansion & Reduction)

好的特徵 $\approx$ 一半的模型效能

"Better data beats fancier algorithms" — Peter Norvig

✿ (二) 特徵轉換 (Feature Transformation)

將原始資料標準化、正規化或轉換成模型能處理的格式。

◆ 數值型轉換

技術	目的	技術細節
Z-score 標準化	均值為 0、標準差為 1 (對稱分布)	StandardScaler()
Min-Max 正規化	映射至 [0, 1] 區間	MinMaxScaler()
Robust Scaling	對異常值不敏感	使用 IQR · RobustScaler()
Log / Box-Cox 轉換	使偏態數據更接近常態分布	np.log1p(x)、 scipy.stats.boxcox()

◆ 分類特徵編碼

技術	描述	適用模型
One-Hot Encoding	為每個類別新增二元欄位	線性模型、樹模型
Label Encoding	將類別轉為數字編碼	樹模型 (會自動處理順序)
Target Encoding	類別 $\rightarrow$ 該類別的目標平均	有過擬合風險，需交叉驗證

Embedding Encoding	用低維向量表達語意	用於深度學習模型 (如 Embedding Layer)
-----------------------	-----------	--------------------------------

◆ 時間與日期特徵拆解

將 timestamp 拆解成：

- 年 / 月 / 日 / 小時 / 星期
- 是否假日 / 假日距離
- 時間週期 (如上班時段、季節)

🔗 (三) 特徵選擇 (Feature Selection)

目的：保留有用資訊、移除冗餘、減少維度、提升學習效能與泛化能力。

◆ 方法類別對照表

方法類別	範例	特點
Filter	Chi-square、Mutual Info、Variance Threshold	快速計算、模型無關
Wrapper	RFE (遞迴式特徵消除)、Forward/Backward Selection	需訓練模型、運算成本高
Embedded	Lasso (L1)、Tree Importance、XGBoost Gain	內嵌在模型訓練中，效率高

◆ 常用評分指標

指標	說明
Information Gain (IG)	特徵提供多少資訊來分類目標
Gini Importance	決策樹中特徵對純度提升的貢獻
SHAP Value	模型解釋技術，用於排名重要特徵

◆ 實作套件：

- sklearn.feature_selection
- BorutaPy (適用於 Random Forest)
- XGBoost/LightGBM → .feature_importances_

🔗 (四) 特徵構建 (Feature Construction)

將多個原始特徵進行結合、變換、計算，創造更有代表性的變數。

◆ 數學轉換與交互特徵

類型	方法	實例
加總 / 比例 / 交互	$x_1 + x_2$, x_1/x_3 , $x_1 * x_4$	客戶支出總額 / 平均購買
非線性變換	log, sqrt, exp	資料偏態修正

類型	方法	實例
多項式特徵	x, x^2, x^3	多項式回歸擴展 (PolynomialFeatures)

- ◆ 分群後標籤作為新特徵
 - 使用 KMeans / GMM 分群後，將群編號作為類別特徵輸入模型。
- ◆ 頻率與排名特徵
 - 使用者出現次數、某個類別的排名位置 (如 RFM 模型)

(五) 特徵擴展與壓縮 (特徵空間控制)

- ◆ 擴展：對高維度資料進行擴張提升模型表現 (例如交互特徵、多項式)
- ◆ 壓縮：當特徵過多，考慮降維以避免維度詛咒

技術	方法說明	適用
PCA	主成分分析 (保留變異數)	結構化連續數值
t-SNE / UMAP	非線性降維，保留分布關係	用於可視化
Autoencoder	神經網路降維	用於圖像、語音、時間序列等高維資料

(六) 特徵工程對不同模型的關鍵性

模型類型	是否需特徵工程	原因
線性模型 (Logistic, SVM)	☒ 非常需要	對線性組合與標準化敏感
決策樹 (RF, XGBoost)	◆ 適度處理即可	對數值尺度與缺值較不敏感
DNN / CNN / RNN	☒ 重要但可嵌入	可自動學習特徵，但資料質量與嵌入表達仍為關鍵
BERT / Transformer	 可少處理	已有內建嵌入與語境理解能力，僅需斷詞與特殊 token 處理

實務案例

範例：銀行信貸違約預測模型

原始資料	特徵工程處理
年收入	Log 轉換後 Z-score
教育程度 (低、中、高)	One-Hot 編碼
信用卡張數	製造「卡片總額 / 月收入」交互特徵
銀行帳戶建立日期	拆分成「年資」欄位

總結要點

- 特徵工程是模型效能提升的核心手段，需根據資料類型、任務與模型特性設計。
- 包含轉換、選擇、構建、擴展與壓縮等五大面向。
- 搭配可視化（如 SHAP）、模型訓練評估流程（如交叉驗證）調整特徵策略。

四、模型比較與選擇

（一）模型比較面向總覽

維度	說明
 資料類型適配	結構化 vs 非結構化、文本、圖像、時間序列等
 模型表現能力	線性 / 非線性、擬合能力、可擴展性
 可解釋性	特徵影響度、可視化、符合法規要求
 訓練效率	訓練時間、計算需求、資源耗費
 實作便利性	實作工具可用性、調參複雜度、部署彈性

（二）監督式學習模型總覽與比較（分類與回歸通用）

模型	特性說明	優點	缺點	適用場景
Logistic / Linear Regression	線性模型，分類/回歸	解釋性強、快、簡單	難處理非線性、多共線性敏感	金融、醫療等強調可解釋領域
Decision Tree	根據資料特徵建立樹狀結構	易解釋、不須特徵轉換	易過擬合、表現受限	教育應用、低資源設備
Random Forest	多顆決策樹投票平均	穩健、抗過擬合、重要特徵排名	不易解釋、資源中高	標準分類 / 回歸任務
XGBoost / LightGBM	提升法（Boosting）樹模型	準確率高、速度快、處理缺值	複雜調參、可解釋性差	資料結構化競賽、Kaggle
Support Vector Machine (SVM)	使用超平面區分邊界	能處理高維稀疏資料	對大資料不易擴展	小樣本文字分類
k-NN (K 最近鄰)	基於距離的非參數法	不需訓練、直觀	運算慢、無法泛化	小型影像辨識、基準實驗
Naive Bayes	條件獨立的機率分類	計算快速、記憶體少	假設過強、難處理複雜關係	文字分類（垃圾郵件偵測）
MLP / DNN (多層感知機)	多層神經網路	表達力強、自動學特徵	訓練需大資料、不易解釋	結構化深度任務、語音、信號

模型	特性說明	優點	缺點	適用場景
CNN (卷積神經網路)	對空間資訊敏感	最強圖像辨識模型	訓練時間長、需大量資料	影像分類、醫學影像
RNN / LSTM / GRU	處理序列資料	可捕捉時間依賴	訓練困難、梯度爆炸問題	時間序列預測、文字生成
Transformer (BERT, GPT)	自注意力機制，適用多模態	大語言模型基礎、高彈性	訓練成本極高	NLP、跨模態 AI 任務

(三) 模型選擇建議對照 (依照任務 / 資料特性)

▶ 1. 結構化資料任務 (表格型)

任務類型	建議模型	理由
二元分類	Logistic (易解釋)、XGBoost (高表現)	兼顧準確與可解釋性
多類分類	LightGBM、Random Forest	支援多類且泛化穩定
回歸	Linear Regression、LGBMRegressor	適合大規模特徵數據

▶ 2. 非結構化資料任務 (文本、影像、語音)

資料型態	任務	建議模型	補充技術
文本	分類	TF-IDF + Logistic / BERT	Tokenizer、embedding
影像	分類	CNN / ResNet	Data Augmentation
聲音	辨識	RNN / Wav2Vec	MFCC 特徵、語音切割

▶ 3. 小樣本或特徵稀疏資料

情境	模型選擇建議
樣本 < 1000	SVM、KNN、Bayes
特徵極多 (> 1000)	Lasso、XGBoost、Embedding Network
類別不平衡	使用 Focal Loss、樣本加權、SMOTE + XGBoost

(四) 模型組合與增強技術 (Model Ensembling)

方法	描述	常見工具
Bagging	多模型平均 (如 Random Forest)	sklearn.ensemble.BaggingClassifier
Boosting	逐步糾正前一模型誤差 (如 XGBoost)	XGBoost, LightGBM
Stacking	多模型輸出 → Meta Learner 再預測	mlxtend.stacking, H2O Ensemble

(五) 工具對照與框架支援

工具平台	適合模型	支援功能
scikit-learn	Logistic, Tree, SVM, KNN	全套前處理 + 模型比較
XGBoost / LightGBM	提升法樹模型	自動 Early Stopping、特徵重要性
PyTorch / TensorFlow / Keras	CNN, RNN, Transformer	深度學習模型建構
AutoML 工具	多模型混合	自動選型 + 超參調整

(六) 模型選擇總結建議表 (供決策時快速參考)

模型	適合資料量	可解釋性	訓練資源	適合場景
Logistic / Linear	小-中	☑ 高	● 低	金融、醫療、問卷分析
Random Forest	中-大	◆ 中	● 中	結構化分類、回歸
XGBoost	中-大	✗ 較低	● 中高	比賽、商業模型部署
DNN / CNN	中-大	✗ 低	● 高	深度學習任務 (圖像、文本)
Transformer	大	✗ 非常低	●● 高	跨模態、生成式 AI

◆ 補充：選擇建議依據目標與條件

1. 商業可落地 / 法規合規重視：Logistic / Explainable Boosting Machine
2. 需要高精度競賽模型：XGBoost / Ensemble Model
3. 影像或自然語言任務：CNN / Transformer
4. 資源受限：Random Forest / LGBM / 小型 MLP
5. 需持續學習或線上更新：Online Learning (如 Vowpal Wabbit)

五、模型評估指標

(一) 📁 分類任務指標 (Classification Metrics)

適用於離散目標變數 (如 0/1、Yes/No、多類別)

◆ Confusion Matrix (混淆矩陣)

	預測為正類	預測為負類
實際為正類 (P)	TP (真正)	FN (假負)
實際為負類 (N)	FP (假正)	TN (真負)

◆ Accuracy (準確率)

- 定義：所有預測中正確的比例

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- 適用：當正負類比例接近時
- 風險：資料不平衡時會失真

◆ Precision (精確率)

- 定義：預測為正類中，實際為正類的比例

$$Precision = \frac{TP}{TP + FP}$$

- 應用：醫療陽性判定 (希望假正低)

◆ Recall (召回率 / 敏感度)

- 定義：所有實際正類中，被正確預測出的比例

$$Recall = \frac{TP}{TP + FN}$$

- 應用：錯過代價高的場景 (如癌症偵測)

◆ F1-score (加權調和平均)

- 定義：Precision 與 Recall 的調和平均

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- 適用：資料不平衡，需平衡 Precision 與 Recall

◆ AUC-ROC (曲線下面積)

- 定義：ROC 曲線下的面積，反映模型整體區分能力
 - ROC：x 軸為 FPR (假正率)，y 軸為 TPR (真陽率)
 - AUC = 1 表示完美分類器
- 適用：機率輸出模型 (如 Logistic, XGBoost)

(二) 📁 回歸任務指標 (Regression Metrics)

適用於 連續目標變數 (如價格、利潤、溫度)

◆ MSE (均方誤差)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- 特點：放大誤差，對離群值敏感
- 用於：DNN、Lasso 等需平滑損失函數時

◆ MAE (平均絕對誤差)

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- 優點：對異常值較穩定
- 用於：商業決策（誤差以元計算）

◆ R^2 (決定係數)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- 意義：模型解釋資料變異的比例
- 區間： $R^2 \in (-\infty, 1]$ $R^2 \in (-\infty, 1]$ 表示完美預測

(三) 📁 排序任務指標 (Ranking Metrics)

適用於 推薦系統、搜尋引擎、資訊檢索任務

(模型輸出為排序列表)

◆ MAP (Mean Average Precision)

- 意義：多個查詢的平均精確率

$$MAP = \frac{1}{Q} \sum_{q=1}^Q \left(\frac{1}{|P_q|} \sum_{k \in P_q} precision@k \right)$$

- 應用：電商推薦、新聞推播

◆ NDCG (Normalized Discounted Cumulative Gain)

- 定義：考慮排序位置的加權回報

$$NDCG_k = \frac{DCG_k}{IDCG_k}$$

$$DCG_k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$$

- 優點：前面項目重要性更高、具位置權重

(四) 📁 多維評估視覺化 (Multidimensional Evaluation)

◆ Confusion Matrix (延伸)

可拓展至多類別任務：

- 以 $N \times N$ 表格表示每一類的預測與實際交叉
- 評估「哪一類容易混淆」「哪些類型 Recall 較差」

◆ PR Curve (Precision-Recall Curve)

- 定義：橫軸 Recall、縱軸 Precision，曲線呈現不同閾值下的平衡
- 適用：資料極度不平衡時（如欺詐、癌症）
- $PR AUC > ROC AUC$ 在不平衡樣本情境下更能反映真實效能

🔧 模型選擇與指標搭配建議

任務	常用模型	建議指標
二元分類	Logistic, XGBoost	F1, ROC-AUC, PR Curve
多類分類	RF, LightGBM, BERT	Macro F1, Confusion Matrix
回歸	Linear, DNN	MSE, MAE, R ²
排序	FM, DeepFM, Transformer-Rank	NDCG@k, MAP
NLP (QA,摘要)	BERT, T5	ROUGE, BLEU
CV (偵測)	YOLO, Faster-RCNN	mAP (IoU-based)

💡 實務範例：信用卡違約預測

| 模型 | XGBoost | | 指標 | AUC: 0.93、Recall: 0.78、Precision: 0.62 | | 解讀 |

- 高 AUC → 模型可區分違約與非違約用戶
- Recall 較高 → 找出多數違約者 (偏向降低 FN)
- Precision 較低 → 可能誤判一些好用戶 (FP 上升)

🐍 Python 工具與函式

指標類型	套件 / 函式
分類	sklearn.metrics.classification_report, roc_auc_score, confusion_matrix
回歸	mean_squared_error, r2_score, mean_absolute_error
排序	sklearn.metrics.ndcg_score, average_precision_score
多維可視化	seaborn.heatmap(confusion_matrix), precision_recall_curve()

六、AutoML 與模型選擇工具

(一) 什麼是 AutoML ?

AutoML (Automated Machine Learning) 是一種系統性方法，目的是讓使用者不需大量人工干預就能完成從資料輸入到模型部署的整體機器學習流程，其核心是：

“ 讓非專業資料科學家也能訓練出具有競爭力的模型 ”

(二) AutoML 自動化的技術流程

✳️ AutoML 涵蓋的技術模組：

模組階段	自動化技術	技術細節說明
資料處理	自動缺值處理、自動類別編碼	SimpleImputer, OneHotEncoder
特徵工程	特徵選擇、自動構建、多項式轉換	Recursive Feature Elimination, Polynomial Features

模組階段	自動化技術	技術細節說明
模型選擇	多模型搜尋與評估策略	GridSearchCV、Bayesian Search
超參數調整	自動調參 (Hyperparameter Tuning)	Random Search, Bayesian Optimization, BOHB
結果評估	自動指標計算、視覺化	cross_val_score, ROC Curve, SHAP 解釋
模型導出	可部署化 (ONNX、PMML)	joblib、mlflow、Triton

(三) AutoML 探索策略 (Search Space 與策略類型)

類型	描述	技術範例
Grid Search	遍歷所有超參數組合	GridSearchCV (缺點：慢)
Random Search	隨機抽取若干組參數	RandomizedSearchCV
Bayesian Optimization	根據前一輪結果調整搜尋方向	Hyperopt, Optuna, SMAC
Evolutionary Search	基因演算法進化式調整	TPOT
Reinforcement Learning	學習最佳模型搜索策略	Google Vizier (如 Google AutoML)

(四) 主流 AutoML 工具平台比較

工具 / 平台	編程語言	特點與優勢	適用情境
Auto-sklearn	Python / scikit-learn	使用貝葉斯優化與 Ensemble Selection，擅長結構化資料	結構化資料、小中型專案
TPOT	Python / Genetic Programming	使用演化演算法，自動組合 pipeline	建立創新特徵管道、科研探索
H2O AutoML	Java + Python / R	企業級，支援多模型堆疊、XGBoost、GLM、DRF 等	企業部署、大量資料
Google AutoML / Vertex AI	雲端	支援表格、Vision、NLP，整合 GCP 生態系	雲端端到端部署
Microsoft Azure AutoML	雲端	支援 GUI 與 Python SDK，深度整合 Azure ML	MLOps, CI/CD 運作
Amazon SageMaker Autopilot	雲端	結合 S3 + Auto Tuning，支援 Endpoint Deploy	雲端商業應用部署
MLJAR AutoML	Web 介面 / Python	操作簡便，支援	教育、實驗性質快速

工具 / 平台	編程語言	特點與優勢	適用情境
		Explainability	入門

(五) 模型選擇與調參工具 (非完整 AutoML，但輔助選型)

☒ Scikit-learn 模型組合與評估工具

- Pipeline：將前處理 + 模型組合成統一流程
- GridSearchCV / RandomizedSearchCV：內建交叉驗證
- SelectKBest, RFE：特徵選擇器

☒ Hyperparameter Tuning 專用工具

工具	技術機制	特點
Optuna	貝葉斯優化 + early stopping	快速、彈性高，支援 DNN/LightGBM
Ray Tune	分散式調參	適合多機訓練、自動資源配置
MLflow + hyperopt	實驗追蹤 + 超參調整	整合性高，可視化效果好

(六) AutoML 的優勢與限制

☒ 優勢：

- 降低建模門檻，提升效率
- 快速產生高效能 baseline
- 可整合 MLOps 工具鏈自動部署

⚠ 限制：

- 對於特殊任務 (如 NLP 多模態) 仍需人工設計
- 難以全自動處理資料品質與資料偏差問題
- 黑盒特性強，模型可解釋性需額外設計 (如 SHAP)

💡 實務應用案例

🔴 案例：保險詐欺預測專案

項目	技術應用
任務類型	二元分類 (是否詐欺)
資料型態	結構化：保單欄位 + 使用行為
AutoML 工具	Auto-sklearn + MLflow
效果	比人為 GridSearch 模型提升 AUC 9.3%
解釋性處理	使用 SHAP 分析詐欺重要特徵：通報次數、異常索賠金額、理賠頻率

📦 AutoML 與 MLOps 整合建議 (企業部署場景)

階段	工具建議
自動化建模	Auto-sklearn, H2O, Optuna
結果追蹤	MLflow, W&B
模型部署	ONNX, TorchScript, SageMaker Endpoint
模型監控	Evidently AI, Alibi Detect (資料漂移檢測)

七、進階實作與實例分析

📦 案例 1：電商推薦系統 (E-Commerce Recommender System)

☑ 1.1 資料來源

類型	說明
使用者行為資料	點擊紀錄 (Clickstream)、購買紀錄、加購、收藏等
產品內容資料	商品類別、品牌、價格、促銷標籤、描述文案
時間特徵	點擊時間、瀏覽時段、用戶活躍週期 (例如：週末 vs 平日)

☑ 1.2 特徵工程

🕒 時序特徵

- Session 內行為序列編碼 (如：最後 5 次點擊商品類別)
- 滑動視窗統計 (如：過去 7 日內的平均瀏覽次數、轉換率)

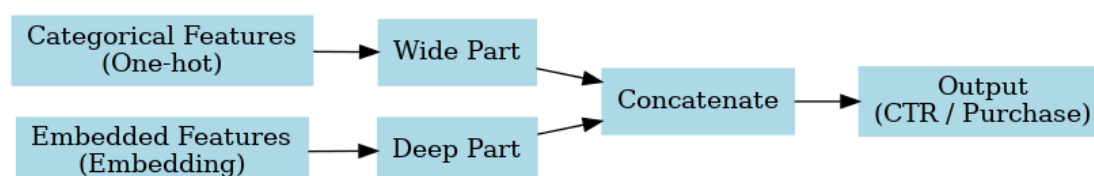
🌿 類別型商品資料嵌入 (Item Embedding)

- 使用 Word2Vec / Skip-gram 處理商品序列點擊：
 - 商品點擊序列 ⇨ 商品之間的共現語意嵌入
- 或利用 Embedding Layer：


```
item_input = Input(shape=(1,))
item_emb = Embedding(input_dim=num_items, output_dim=32)(item_input)
```

☑ 1.3 模型設計

🔹 Wide & Deep Model 架構



- Wide 部分：使用簡單線性模型捕捉記憶規則
- Deep 部分：利用 DNN 捕捉潛在特徵交互
- 支援端到端訓練，常見於 TensorFlow Recommenders

◆ XGBoost + Recall-Rerank Pipeline

1. Recall：使用商品向量搜尋（如 FAISS）召回前 300 項
2. Rerank：將候選商品輸入 XGBoost 排序模型
 - 特徵：使用者最近互動、商品類別距離、時間差等

☑ 1.4 效能評估

指標	說明
Precision@k	前 k 項商品中被點擊/購買的比例
NDCG	考慮推薦排序與真實點擊位置的加權評分
Coverage	系統能推薦多少不同商品，避免冷啟與集中效應

📁 案例 2：醫療診斷預測 (Medical Diagnosis Prediction)

☑ 2.1 資料來源

類別	說明
結構化檢驗資料	血壓、血糖、血脂、CRP、白血球數、肝腎功能等臨床檢驗值
病人基本資料	年齡、性別、住院次數、病史 ICD 編碼
時序紀錄	住院前/後檢測時間、症狀發生時間順序

☑ 2.2 特徵工程與轉換

- 類別轉數值編碼：ICD-10 編碼轉換為 Label/Embedding
- 時間特徵展平：使用 Sliding Window 或 Time Bucket 分段
- 缺值處理：醫療資料常含有 Missing at Random (MAR)，建議用 KNN/MICE

☑ 2.3 模型比較

模型類型	描述
Random Forest	非線性 Ensemble Tree，可解釋度高，支援特徵重要性計算
CNN on EHR	將時間序列結構化資料視為「特徵圖」，使用 1D CNN 做分類
其他補充模型	可選用 GRU、Transformer Encoder 作為比較

☑ 2.4 評估指標與解釋性分析

指標	說明
F1-score	平衡 Precision 與 Recall，適合不均衡樣本分類
Sensitivity	真陽性率，表示能夠正確偵測疾病（尤其用於癌症早期預測）

指標	說明
SHAP 分析	模型可解釋性工具，顯示各檢測數值對診斷預測的貢獻度

☑ 結語與比較

比較面向	Random Forest	CNN on EHR
可處理非線性	☑	☑
支援時序結構	✗	☑ (需轉換為固定長度輸入)
解釋性	高 (Feature Importance)	中 (需配合 SHAP, Grad-CAM)
運算成本	低	高 (需 GPU 與更多樣本)

L21302 AI 技術系統集成與部署

一、AI 技術系統開發與部署概論

(一)系統開發生命週期 (AI SDLC: AI Software Development Life Cycle)

這是一個涵蓋「資料到產品」全生命週期的技術流程，主要包含下列七大階段：

1. 問題定義 (Problem Framing)

🚩 目標：

明確界定商業/技術問題、可衡量的任務、預期輸出格式與限制條件。

🔍 技術重點：

- 使用 需求工程技術 (Requirement Elicitation)，如：User Story、Use Case Diagram
- 對應 AI 任務：分類、回歸、分群、生成、強化學習等
- 決策基礎：是否採用 rule-based / AI-based / hybrid 解決方案
- 初步定義 KPI：如提升準確率、減少錯誤偵測率、增加客戶轉換率等

💡 技術實務：

- 需求分析模型 (Requirement Modeling)
 - 使用工具：UML Use Case Diagram, BPMN
 - 描述問題背景、流程瓶頸、資料流動節點
- 任務分類
 - 監督式學習 (Supervised)：分類、回歸
 - 非監督式學習 (Unsupervised)：分群、降維
 - 強化學習 (Reinforcement)：動態決策場景，如物流、廣告投放
 - 生成任務 (Generative)：如圖像合成、文本生成

🛠️ 實作策略：

- 設定 KPI：如提升預測準確率 > 92%，降低客服處理時間 30%
- 界定約束條件 (如反應時間需 < 500ms、資料無法外傳)

2. 資料準備 (Data Preparation)

🚩 目標：

整理可供訓練、驗證與測試的資料，並將其轉換為模型可理解的格式。

🔍 技術重點：

- ETL/ELT 流程建構 (使用工具如 Apache Airflow, Spark, dbt)
- 資料清洗：缺失值處理、異常值偵測 (Z-score、IQR)、型態轉換
- 特徵工程 (Feature Engineering)：
 - 數值轉換 (標準化、正規化)
 - 類別編碼 (One-hot, Target Encoding)
 - 特徵交互 (Interaction Features)
- 資料集切分：Train / Validation / Test

🧠 技術實務：

(1) 資料蒐集

- API 抓取 (RESTful, GraphQL)
- ETL 工具：Apache Nifi / Talend / Airbyte
- 自建爬蟲 (Scrapy / Selenium)

(2) 資料清洗 (Data Cleaning)

- 缺失值處理：
 - 數值填補 (均值、中位數、KNN imputation)
 - 刪除樣本
- 異常值處理：
 - Z-score 過閾值、Isolation Forest
- 編碼處理：
 - 類別：One-hot / Label Encoding
 - 時間欄位：週期性特徵 (sin/cos 轉換)

(3) 資料切分

- 比例：Train / Val / Test = 60/20/20 或 70/15/15
- 分層抽樣 (Stratified Sampling) 以避免類別不平衡

3. 模型開發 (Model Development)

🚩 目標：

選擇適當的演算法與架構，進行訓練與初步評估。

🔍 技術重點：

- 模型選型：根據資料特性與任務選擇傳統 ML 或深度學習架構
 - ML：XGBoost, LightGBM, Random Forest, SVM
 - DL：CNN (影像)、LSTM/BERT (文本)、Transformer (多模態)、Diffusion

(生成式)

- 模型訓練：
 - 超參數調整 (Grid / Random / Bayesian Optimization)
 - 資料增強 (Data Augmentation) 與樣本不均處理 (如 SMOTE)
- 模型保存格式：Pickle, Joblib, ONNX, TorchScript

 技術實務：

(1) 模型選擇

任務類型	推薦演算法 / 架構
二元分類	XGBoost, Logistic Regression, BERT
多類別分類	LightGBM, Decision Tree, Transformer
回歸任務	Linear Regression, Random Forest
時序預測	ARIMA, LSTM, Prophet
生成任務	GAN, VAE, Diffusion Models

(2) 超參數優化

- Random Search、Grid Search、Bayesian Optimization (Optuna, HyperOpt)

(3) 訓練技巧

- Early Stopping (防止過擬合)
- Learning Rate Scheduler (餵養策略調整)
- Regularization：L1 / L2、Dropout, BatchNorm
- Class Imbalance 處理：
 - SMOTE 增樣本、Focal Loss (處理困難樣本加權)

4. 模型評估 (Model Evaluation)

 目標：

透過多元指標檢驗模型是否達成預期任務目標。

 技術重點：

- 指標選擇：
 - 分類：Accuracy, Precision, Recall, F1-score, ROC-AUC, PR Curve
 - 回歸：RMSE, MAE, R²
 - 多類別或排序：Top-K Accuracy, NDCG, MAP
- 可解釋性分析 (XAI)：
 - LIME、SHAP、Integrated Gradient
 - Feature Importance Rank
- 偏差偵測：
 - Data Drift / Concept Drift (使用 EDA + Data Drift 檢測)

💡 技術實務：

(1) 性能指標

任務	指標
分類	Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion Matrix
回歸	MAE, MSE, RMSE, R ²
多分類	Macro-F1, Weighted-F1, Top-k Accuracy
排序	NDCG, MAP, Recall@k

(2) 模型解釋性

- Local : LIME、SHAP (解釋個別預測)
- Global : Feature Importance、PDP、ICE plots

(3) 偏差分析

- 資料漂移 (Covariate Shift) : 分布比較
- 模型偏誤 (Fairness) : Equal Opportunity, Disparate Impact

5. 系統集成 (System Integration)

🚩 目標：

將模型封裝為服務，並與業務邏輯或用戶介面進行整合。

🔍 技術重點：

- API 封裝 (Flask, FastAPI, gRPC)
- 模型服務框架：
 - TorchServe、BentoML、Seldon Core、KServe
- 介接資料層：RESTful JSON、gRPC Protocol Buffer、WebSocket
- 版本控制與模型註冊：MLflow Model Registry / SageMaker Model Registry

💡 技術實務：

(1) 模型封裝

- API 框架：Flask / FastAPI / gRPC
- Inference Optimization：ONNX、TorchScript、TensorRT

(2) 多模型管理

- Routing Layer：根據任務類型選擇模型 (Ensemble / Cascade)

(3) 輸入驗證

- Schema 驗證工具：Pydantic (FastAPI 內建)、Marshmallow

6. 上線部署 (Deployment)

🚩 目標：

將 AI 模型正式部署至產品環境並能穩定運行。

🔍 技術重點：

- 部署形式：
 - 雲端託管 (SageMaker, GCP Vertex AI, Azure ML)
 - 容器化部署 (Docker, Kubernetes, Helm)
 - Edge AI / Mobile (CoreML, TensorRT, TFLite)
- 彈性調度與負載平衡：K8s + HPA + Ingress Controller
- 安全性考量：Token 驗證、API rate limit、防止 prompt injection

💡 技術實務：

(1) 容器化部署

- 使用 Docker 打包模型與依賴
- 建立 CI/CD pipeline：GitHub Actions → Docker Registry → Kubernetes cluster

(2) 雲原生部署架構

- 使用 K8s + Helm Charts 管理版本與彈性擴展
- GPU 排程與資源隔離：Node Affinity + Resource Quota

(3) API Gateway 與限流

- Kong / Istio：流量治理、API Rate Limiting、JWT 驗證

7. 監控與更新 (Monitoring & Updating)

📌 目標：

確保模型與系統長期穩定運作，並具備更新調整能力。

🔍 技術重點：

- 模型服務健康監控：效能、延遲、錯誤率
- 模型品質監控：資料漂移、輸出異常、精度下降
- 自動回訓與更新機制：
 - 週期性 retraining pipeline (如 Airflow / Kubeflow)
 - Online Learning / Incremental Training 應用場景
- 更新策略：
 - Canary Release、Shadow Deployment、Blue-Green Deployment

💡 技術實務：

(1) 效能監控

- 指標：Latency、Throughput、Resource Usage (CPU/GPU)
- 工具：Prometheus + Grafana

(2) 模型品質監控

- 資料漂移：Kolmogorov–Smirnov test, Population Stability Index (PSI)
- 預測結果監控：Confusion Matrix Over Time、精確率走勢

(3) 自動更新機制

- 使用 Airflow/Kubeflow 建立 retraining pipeline
- 當 $PSI > 0.2$ 自動觸發 retrain & redeploy

(二) DevOps 與 MLOps 整合 (CI/CD in AI Pipeline)

✦ MLOps (Machine Learning Operations) 三大核心：

階段	任務	工具建議
管理階段	模型版本控制、追蹤、審計	MLflow, DVC, Neptune.ai
自動化階段	訓練、測試、部署自動化	GitHub Actions, Argo Workflows, Kubeflow Pipelines
運營階段	上線監控、更新與警報	Prometheus, NannyML, Grafana

✦ MLOps 與 DevOps 差異：

- DevOps：聚焦於程式碼版本控制與自動化部署
- MLOps：需納入資料漂移、模型版本、特徵一致性與持續回訓機制

二、系統架構設計

◆ (一) 模組化架構 (Modular Architecture)

☑ 目標

提升 AI 系統的可重構性與可維護性，使模型與服務能獨立部署、熱更新、快速擴展與異構整合。

🔧 技術要點

┃ A. 可插拔 (Pluggable) 模型設計

- 概念：
 - 模型或演算法以模組方式封裝，可獨立熱更新 (hot-swap) 與版本升級
- 技術實作方式：
 - 使用抽象接口設計 (如 Python abc.ABC 或 Java Interface)
 - 設定模型路由與版本控制機制 (如 /v1/model_A、/v2/model_B)
 - 模型熱部署框架：TorchServe、ONNX Runtime, Triton Inference Server

┃ B. 微服務架構 (Microservices)

- 基本設計模式：
 - 將 AI 系統分解為獨立服務模組，每個模組具備單一職責。
- 主要模組說明：

模組	功能說明
API Gateway	負責請求分發、驗證、日誌收集，可使用 Kong、Istio

模組	功能說明
Model Service	處理模型推論，支援模型版本切換與 AB test
Data Service	管理資料存取、特徵查詢，可接 MySQL, Redis, MongoDB
Logging Service	集中式記錄與監控，如 ELK Stack (Elasticsearch、Logstash、Kibana)

- 部署技術：
 - 容器化：Docker
 - 編排與擴展：Kubernetes (支援 Auto-scaling、Rolling Update、Canary)

◆ (二) 資料流架構 (Data Flow Pipeline)

☑ 目標

實現從資料蒐集、清洗、特徵提取到模型推論間的資料一致性、高效流通與監控能力。

🔧 技術要點

┃ A. ETL / ELT 流程建構

類型	說明
ETL (Extract → Transform → Load)	傳統流程，資料清洗後寫入資料倉儲
ELT (Extract → Load → Transform)	雲端常見流程，先儲存再轉換，提高靈活性

- 支援工具：
 - Apache Kafka：即時資料串流平台，支援 pub/sub 模式
 - Apache Airflow：建構與排程 ETL DAG (工作流程)
 - Apache Flink：即時資料處理與事件驅動處理

┃ B. 特徵服務層 (Feature Store)

- 用途：解決訓練與推論特徵不一致問題 (Training/Serving Skew)
- 工具建議：
 - [Feast](#)：主流開源特徵管理平台
 - Databricks Feature Store (商用)
- 功能特色：
 - 線上 / 離線特徵同步
 - 特徵版本控制與重用
 - 特徵查詢封裝與封存 (封裝為 API 供模型呼叫)

◆ (三) 雲原生部署架構 (Cloud-native AI Deployment Architecture)

☑ 目標

使用雲端原生技術支撐模型部署、監控、擴展與服務治理，提升彈性與可用性。

🔧 架構構成元件

元件	技術 / 框架	功能描述
Inference Server	TorchServe / Triton / BentoML	模型推論、版本管理、自動 batch
API Gateway	Kong / Istio	請求路由、安全憑證、流量管制
AutoScaler	K8s HPA / VPA	根據 CPU/GPU 使用率自動擴縮
Monitor	Prometheus + Grafana	收集推論延遲、請求量、資源利用率等指標
Alert Manager	Slack / PagerDuty 整合	偵測異常指標，自動發送警報通知

雲端推論服務模式

■ A. Batch Inference (離線批次推論)

- 適合：定期執行、資料量大、非即時需求場景 (如用戶行為分析)
- 工具建議：
 - Spark MLlib、AWS Batch、GCP Dataflow、Azure ML pipeline

■ B. Online Inference (即時推論)

- 適合：互動式應用、API 呼叫型服務 (如聊天機器人、圖片分類)
- 工具建議：
 - TorchServe、BentoML、Triton Inference Server
 - FastAPI/Flask + Gunicorn/Uvicorn

■ C. 多租戶推論系統 (Multi-tenancy Serving)

- 設計挑戰：
 - 客戶間模型隔離
 - 資源彈性調度
 - 資料存取隔離 (RBAC)
- 隔離方案：
 - Namespace 隔離 (Kubernetes)
 - 模型容器封裝、執行環境虛擬化 (如 KServe + Istio)

◆ (四) 即時推論架構 (Online Inference Architecture)

☒ 目標

構建能夠處理低延遲、高併發請求的 AI 模型推論系統，支援 API 存取與自動監控、動態擴展能力。

✿ 架構組件說明

模組	功能描述	技術選型
API Gateway	統一入口，進行請求驗證與路由	Kong, Istio, NGINX
模型推論服務 (Model Server)	接收預測請求並執行推論	TorchServe, Triton, BentoML
輸入驗證與轉換層 (Preprocessor)	將 JSON 等輸入轉換為張量，進行格式檢查	FastAPI middleware / custom Flask wrapper
後處理層 (Postprocessor)	將模型輸出轉為業務格式或文字標籤	pandas, NumPy, JSON Schema
模型快取與冷熱備援	維持熱門模型於記憶體內，加速推論反應	Redis + LRU cache policy
監控模組	記錄 QPS、延遲、成功率、資源使用率	Prometheus + Grafana
AutoScaler	根據負載動態增加實例	Kubernetes HPA, KEDA

💡 進階設計建議

- A/B 測試與模型比較：使用 request header 註明 X-Model-Version 路由至指定模型
- Low Latency 模型加速：使用 ONNX + TensorRT + INT8 Quantization
- GPU 資源配置最佳化：
 - 模型分組綁定 GPU 核心區塊 (CUDA stream pool)
 - 在 Kubernetes 加入 resource limit 配置 (GPU request)

◆ (五) 多模型動態路由架構 (Multi-model Dynamic Routing Architecture)

☑ 目標

支援根據任務、租戶、使用者偏好、自訂模型版本等條件，自動選擇最佳模型服務路由。

✿ 路由策略 (Routing Logic)

類型	說明	範例
任務類型路由	根據任務分類 (分類 vs 回歸) 路由不同模型	/predict?task=classification
版本控制路由	客戶端指定版本進行實驗測試或 rollback	/predict?version=v2.1
使用者群組路由	VIP 使用者使用精緻模型，普通用戶走基礎模型	Token 授權後進行分類

類型	說明	範例
模型 Ensemble 路由	多模型預測後進行投票或加權平均	結合 Random Forest + XGBoost + LSTM

技術組件

組件	功能說明
Model Router API	根據策略選擇後端模型路由，如 Flask/FastAPI 實作邏輯
Model Registry	儲存所有可用模型與版本、Meta info
Routing Policy Engine	根據 request metadata 決定使用哪個模型
Logging + AB Testing Analytics	收集各版本模型推論結果與實驗統計

實作範例片段 (FastAPI + model router)

```
@app.post("/predict")
```

```
async def predict(input: InputData, model_version: Optional[str] = "latest"):
```

```
    model = model_registry.get_model("fraud_detection", version = model_version)
```

```
    output = model.predict(input)
```

```
    return format_response(output)
```

應用場景示意

應用領域	路由應用
金融風控	根據國家 / 法規套用不同地區模型
醫療診斷	使用不同 AI 模型診斷心臟病 / 癌症，依照性別或年齡群分流
客服機器人	自然語言任務分類 → 意圖分類模型 / FAQ 模型 / 多輪對話模型
廣告投放	使用 A/B 模型評估新推薦模型轉換率差異

三、模型優化

模型效能與壓縮技術 (Model Optimization & Compression)

◆ (一) 知識蒸餾 (Knowledge Distillation)

目標

將大型教師模型 (Teacher Model) 的知識轉移給較小的學生模型 (Student Model)，在保有準確度的前提下，縮減模型大小與運算成本。

核心技術結構

組件	說明
教師模型 (Teacher)	精度高但計算成本大，如 BERT-Large、ResNet-152

組件	說明
學生模型 (Student)	精簡化模型，如 MobileBERT、TinyBERT、DistilBERT、MobileNet
軟標籤 (Soft Targets)	教師模型輸出的機率分布，用於訓練學生
損失函數 (Loss)	結合學生與教師的 KL Divergence + 傳統分類 Loss (如 CrossEntropy)

💡 損失函數範例 (PyTorch):

$$loss = \alpha * KLDivLoss(student_logits, teacher_logits) + (1 - \alpha) * CrossEntropyLoss(student_logits, labels)$$

🔍 應用範例：

- TinyBERT：用於 NLP，推論速度為原始 BERT 的 6.1 倍，模型僅 14.5MB。
- DistilBERT：訓練時間縮短 60%、大小減半、精度保留 95%。

◆ (二) 模型量化 (Quantization)

🎯 目標

將模型中的浮點權重從 FP32 降為 INT8 / INT4 等低位元格式，以減少模型大小、加快推論速度並降低記憶體與功耗需求。

✂ 主要類型

類型	說明	精度影響
Post-training Quantization (PTQ)	訓練後將 FP32 模型轉為 INT8	精度可能下降，需評估
Quantization Aware Training (QAT)	模擬量化誤差並重新訓練	精度更穩定，需重訓
Dynamic Quantization	僅在推論時將部分權重轉為低精度	常用於 NLP

🔧 技術要點

- 量化張量 (Quantized Tensor)：將浮點值轉換為整數範圍並儲存 scale/zero_point
- 量化運算 (Quantized Ops)：需用 INT8 支援的運算，如 qlinear, qconv
- 使用工具：
 - PyTorch：torch.quantization
 - TensorFlow Lite：TFLiteConverter
 - ONNX Runtime：支援動態與靜態量化

⚙ 實作範例 (PyTorch 動態量化)：

```
import torch.quantization
quantized_model = torch.quantization.quantize_dynamic(
    model, {torch.nn.Linear}, dtype = torch.qint8
)
```

成效實例

- BERT-base 動態量化：
 - 模型大小從 420MB → 110MB
 - 延遲下降 40%
 - 準確率下降小於 1%

◆ (三) 模型剪枝 (Pruning)

目標

透過移除對結果影響小的權重或神經元，提升模型稀疏性，進而降低推論時間與資源消耗。

技術分類

類型	說明	應用層次
非結構性剪枝 (Unstructured Pruning)	移除單一參數 (如小於某閾值的權重)	Tensor 層內
結構性剪枝 (Structured Pruning)	移除整個 filter / channel / neuron	層級架構優化
動態剪枝 (Dynamic Pruning)	根據輸入動態關閉不重要通道	需要支援 runtime 控制

技術細節

- 重要性指標：
 - Weight Magnitude ($|w|$ 小者移除)
 - 梯度變化量 (Saliency)
 - L1 / L2 正則化懲罰項
- 稀疏性率 (Sparsity Rate)：剪除比例，如 20%、50%、80%

實作流程範例 (PyTorch)：

```
import torch.nn.utils.prune as prune
prune.global_unstructured(
    model.parameters(),
    pruning_method = prune.L1Unstructured,
    amount = 0.4,    # 40% weights will be pruned
)
```

🔍 應用實例：

- ResNet-50 剪枝：
 - FLOPs 減少 60%
 - 精度下降小於 1%
- Transformer Head Pruning：
 - 移除注意力頭提升推論效率

📊 成效比較表

技術	模型壓縮比例	精度損失 (低/中/高)	推論加速	適用場景
蒸餾	2× ~ 6×	低	中	所有場景 (NLP 尤其有效)
量化	3× ~ 4×	低至中	高	Edge AI / Mobile AI
剪枝	1.5× ~ 3×	中	中至高	Vision / CNN / Transformers

◆ (四) 模型壓縮技術組合策略 (Hybrid Compression Pipeline)

☑ 組合目標

不同技術擅長不同面向，透過多策略組合能在保留準確率的同時，最大化壓縮比與推論速度，最終部署於 Edge / Mobile / Production API 上。

🔧 技術搭配細節：

組合順序	技術解釋	效果加成說明
Pruning → Distillation	使用教師模型的邊際資訊來訓練被剪枝過的學生模型	提升稀疏模型準確率與穩定性
Distillation → Quantization	將 soft label 知識傳遞至可壓縮模型	保留模型表現，適合推論設備部署
Pruning + Quantization	去除多餘參數再進行整數化	強化 Edge 裝置記憶體與效能要求

■ 實作案例 (結合三種技術)：

◆ 結合 DistilBERT + QAT + Head Pruning

原始模型：BERT-base →

→ TinyBERT (蒸餾) →

→ 去除 50% 頭 (剪枝) →

→ QAT 訓練 (量化) →

→ TorchScript / TFLite 部署

- 模型壓縮比：6x
- 延遲下降：> 60%
- 精度下降：< 1%

◆ (五) 壓縮實作最佳實踐：任務導向架構建議

📚 NLP 模型 (GPT/BERT) 壓縮實務

☑ 應用場景：文字分類、問答、NER、聊天機器人 (Edge/嵌入式)

🔧 建議策略：

技術	實作工具	重點應用與說明
Knowledge Distillation	TinyBERT / DistilBERT	使用教師模型 (BERT-large) 產生 Soft Target，學生模型為 3~6 層 Transformer
Pruning	Head Pruning / Layer Dropout	減少注意力頭 (head)、移除中層如 Layer 4/8，降低 FLOPs
Quantization	QAT / PTQ via HuggingFace + ONNX / TFLite	支援 INT8 嵌入與線性層，特別是 torch.int8_linear 最有效

🔍 組合範例：

教師模型：BERT-base (12 層)

學生模型：TinyBERT (6 層)

剪枝：保留每層前 4 個 attention heads

量化：全模型 QAT 訓練

最終模型大小：13MB

推論速度：CPU 上提升 5.4x

🧠 GPT 類生成式模型壓縮

☑ 應用場景：LLM Chatbot、摘要、翻譯 (雲端成本優化)

🔧 建議策略：

技術	實作工具	特殊建議
LoRA + Distillation	HuggingFace PEFT / QLoRA	使用 LoRA 模塊降低參數，配合蒸餾加速學習
Weight-only Quantization	GPTQ, AWQ	針對 LLaMA、OPT、GPT2 等模型，使用 INT4/INT8
Pruning	Block Pruning (MoE Layer)	移除低使用率的 expert 模塊，或清除中間層 FFN

🔥 實例：QLoRA + AWQ 壓縮 LLaMA-7B

- QLoRA 減少顯存使用：原本 28GB → 6GB 可訓練
- AWQ + GPTQ 進行 INT4 壓縮：推論速度提升 4~5 倍

- 最終應用部署：FastChat / vLLM

CNN 模型 (圖像應用) 壓縮實務

- ☒ 應用場景：物件偵測、影像分類、邊緣視覺分析

 建議策略：

技術	實作工具	說明
蒸餾	ResNet101 → MobileNetV2	學生模型學習高階語意分類表示
剪枝	Structured Pruning (channel filter)	ResNet/YOLO 可透過 l1-norm 移除低貢獻通道
量化	TensorRT / TFLite	對 Conv2D 與 FC 層進行 INT8 硬體加速推論

 實例：YOLOv5s 剪枝 + INT8 量化

- 剪枝比例：60% channel
- 模型大小：27MB → 4.8MB
- 推論速度：RTX3050 上提升 3.8x
- 精度下降：小於 1.5%

◆ 壓縮策略技術對照總表

模型類型	壓縮技術組合	推薦工具	最終部署格式
BERT (NLP)	蒸餾 + Head Pruning + QAT	HuggingFace Transformers, ONNX	ONNX, TFLite, TorchScript
GPT / LLaMA	LoRA + GPTQ + AWQ	HuggingFace PEFT, GPTQ, llama.cpp	GGUF / INT4 Runtime
CNN / Vision	Filter Pruning + KD + PTQ	Torch, TensorRT, OpenVINO	INT8 Engine, TFLite
多語者模型 (MoE)	專家稀疏剪枝 + QLoRA + 蒸餾	FlashAttention, DeepSpeed	Triton, vLLM

四、應用整合與 API 開發

 (一) API 設計原則 (Design Principles for AI Inference APIs)

☒ a. API 通訊協定選擇：RESTful vs gRPC

特性	RESTful API	gRPC
格式	JSON over HTTP	Protocol Buffers over HTTP/2

特性	RESTful API	gRPC
適用場景	Web 應用、前端互動	高效能機器對機器 (M2M) 通訊
優點	易於整合與測試，工具支援廣泛	延遲低、序列化效率高、支援 bi-directional streaming
缺點	序列化效率低、無 schema 驗證	需 protobuf schema 定義，開發門檻較高
實例	FastAPI、Flask	TensorFlow Serving、gRPC + TorchServe

建議策略：

- Web / 手機 API → 建議採 REST
- 內部服務 / 高頻 RPC → 建議採 gRPC

☒ b. 輸入驗證 (Input Schema Validation)

🔍 驗證工具比較：

工具	支援框架	特點
Pydantic	FastAPI	支援 Python 型別提示、自動生成 Swagger UI
Marshmallow	Flask / Django	支援自定義序列化與反序列化邏輯
Cerberus	輕量級框架通用	JSON 驗證輕便快速，靈活性高

☒ FastAPI with Pydantic 範例：

```
from pydantic import BaseModel
class ImageInput(BaseModel):
    user_id: str
    image_base64: str
@app.post("/predict_image")
def predict(data: ImageInput):
    img = decode_base64(data.image_base64)
    result = model.predict(img)
    return {"prediction": result}
```

☒ c. 支援多模態輸入 (Multimodal Input)

🔍 常見格式與應對方式：

資料型態	格式	傳輸方法
圖像	Base64 / multipart	multipart/form-data, base64 in JSON
音訊	WAV / MP3 / PCM	binary stream 或 base64
文字	JSON 字串	application/json
混合模態	JSON + 文件	FormData 或 GraphQL Multipart

✦ FastAPI 支援 multipart 示例：

```
@app.post("/upload_image/")
async def upload(file: UploadFile = File(...)):
    image = Image.open(file.file)
    result = model.predict(image)
    return {"label": result}
```

■ (二) AI 模型包裝成服務 (Model Serving)

☑ a. 主流 Model Serving 框架比較

框架	支援模型	特點	適用場景
FastAPI / Flask	所有模型	自訂彈性高、API 建構快速	原型開發 / Web
TorchServe	PyTorch	多模型管理、內建 handler 支援	PyTorch 正式部署
BentoML	多框架	快速封裝模型、支援 Docker / Kubernetes 部署	跨框架服務化
Seldon Core	多框架	Kubernetes 原生支援、支援 A/B test / Canary	MLOps 專案
TF Serving	TensorFlow	效能最佳化、gRPC 高效推理	TensorFlow 生產環境

☑ b. FastAPI + PyTorch 範例封裝服務

```
import torch
from fastapi import FastAPI
from pydantic import BaseModel
class TextInput(BaseModel):
    text: str
model = torch.load("text_classifier.pt")
model.eval()
app = FastAPI()
@app.post("/predict")
def predict(data: TextInput):
    input_tensor = tokenize(data.text)
    with torch.no_grad():
        output = model(input_tensor)
    return {"label": output.argmax().item()}
```

☑ c. 多模型管理與熱部署 (Multi-model Management & Hot Reloading)

🔧 技術構成：

元件	功能
Model Registry	儲存模型版本、Meta 資訊 (如 MLflow、S3)
Router / Dispatcher	根據模型名稱與版本分派請求
Model Loader	支援模型動態載入、記憶體管理
版本路由機制	/predict/modelA/v1 或 header 中指定版本資訊

🔥 BentoML 多模型動態服務範例：

```
# bento_service.py
import bentoml
from bentoml.io import NumpyNdarray
@bentoml.service()
class IrisClassifier:
    @bentoml.api(input = NumpyNdarray(),output = NumpyNdarray())
    def predict(self,input_array):
        model = bentoml.sklearn.get("iris_clf:latest").to_runner()
        return model.run(input_array)
```

- 可使用 bentoml models list 管理多版本模型
- 使用 Docker / K8s 部署後可自動路由至最新版

☑ d. 部署策略補充：

策略	說明
模型冷熱切換 (Hot-swappable)	TorchServe / BentoML 支援 REST 動態註冊模型
A/B 測試部署	Seldon Core 可根據流量比例切換模型版本
異常自動降版 (Rollback)	Model Router 根據效能監控指標自動切換版本
多租戶模型隔離	RESTful endpoint 加入租戶識別 /predict/tenantA/modelX

🔧 補充：服務效能監控與限流機制 (進階)

- 監控延遲、錯誤率：Prometheus + Grafana + FastAPI middleware
- API 頻率限制：使用 fastapi-limiter 或 kong 網關管理
- 安全性：JWT token 授權、HTTPS 加密、防止 prompt injection (對 LLM 特別重要)

◆ (三) Seldon Core / KServe + Istio：MLOps 整合應用架構展開

🎯 架構目標

建立一個具備以下能力的 MLOps 生產級平台：

-  低延遲高可用的模型服務架構
-  模型版本控制與熱更新 (rolling / canary)
-  推論效能與準確度監控
-  A/B 測試與流量治理
-  多租戶 / RBAC 權限分層服務管理
-  原生支援 Kubernetes、雲原生部署

✦ 架構元件構成

元件	功能	常用工具
Model Serving	部署單一或多模型服務	KServe / Seldon Core
模型註冊 / 管理	模型版本與元資料管理	MLflow, S3, Git, HuggingFace Hub
流量治理	控制 API 請求路由	Istio (Envoy Proxy)
API Gateway	外部請求入口控制器	Istio Ingress Gateway, Ambassador
觀察監控	效能、延遲、輸出紀錄	Prometheus, Grafana, Loki, Kiali
模型訓練 CI/CD	自動建置與部署	Argo Workflows / GitHub Actions
資料漂移監控	模型品質持續追蹤	Evidently AI, NannyML, WhyLabs

🔧 關鍵技術細節與應用實例

☑ 1. KServe / Seldon Core 模型部署形式

類型	說明	特色
單模型部署 (SingleModel)	每個服務只有一個模型 container	簡單、適合入門或小型專案
複合推論流程 (Inference Graph)	多模型組合，如前處理 → 主模型 → 後處理	適合複雜決策流程 (如 NLP)
Router 模型 (ABTest / MAB)	同時部署多版本模型，進行流量切分	適用於 A/B 測試、Online Learning

📄 Seldon 單模型部署 YAML 範例：

```
apiVersion: machinelearning.seldon.io/v1
```

```
kind: SeldonDeployment
```

```
metadata:
```

```
  name: text – classifier
```

```
spec:
```

```
  predictors:
```

```
    – name: default
```

```
      replicas: 2
```

graph:
 name: classifier
 implementation: SKLEARN_SERVER
 modelUri: s3://models/sklearn/v1
 traffic: 100

☒ 2. 使用 Istio 實現 A/B 測試與流量控制

 A/B Test 流量分配

traffic:

- name: model-v1

weight: 70

- name: model-v2

weight: 30

- weight 可設為 100%、50/50、或逐步擴大 (Canary Deployment)
- 透過 Istio VirtualService 控制 HTTP Header → 模型版本映射

☒ 3. 模型觀測與漂移監控

模組	功能	工具
效能監控	QPS、延遲、資源佔用	Prometheus + Grafana
輸出記錄	推論輸出與原始資料紀錄	Loki / Kafka
資料漂移 / 概念漂移偵測	數據分布變化、自動告警	Evidently AI, NannyML

Prometheus 指標範例：

- seldon_api_server_response_time_seconds_bucket
- seldon_api_server_requests_total

☒ 4. 多租戶 / 多模型治理實務

功能	工具 / 技術
Namespace 分離	每個客戶部署至獨立 K8s namespace
RBAC 權限控制	Kubernetes Role + ClusterRole
路徑分流	/tenantA/modelX 對應特定部署

五、自動化 CI/CD 與部署管線

階段	工具	說明
訓練與註冊	GitHub Actions / Argo Workflows	訓練完成後自動提交模型至 MLflow / S3

階段	工具	說明
驗證與封裝	Docker + kustomize	模型封裝為容器，驗證 API 效能
部署與路由設定	Kustomize + ArgoCD / Flux	更新部署 YAML，自動推至 K8s
驗證	pytest + k6	API 回歸測試 + 壓力測試

六、實務應用場景

領域	應用說明
金融	多策略信用評分模型 A/B 測試、自動風控流量管控
健保	不同醫院模型版本切換、病患群體特化模型部署
電商推薦	根據不同客群偏好部署多個推薦引擎版本
聊天機器人	輕量版 / 深度生成版模型動態切換與評估

七、延伸技術資源

◆ (一) 異常偵測與自動回訓機制 (Anomaly Detection & Auto Retraining)

🎯 目標

在模型服務中自動偵測輸入或預測的異常行為 (如資料漂移、模型性能下降)，並根據規則或觸發條件，執行自動回訓與模型替換。

🔧 關鍵技術模組

模組	工具	說明
資料/概念漂移監測	Evidently AI, NannyML	監測 features 分布與預測信心下降
模型效能監控	Prometheus + Custom Metrics	比較預測結果與實際真值間的錯誤
任務排程	Argo Workflows / Airflow	自動化回訓觸發與部署流程
模型註冊	MLflow / BentoML Store	儲存模型版本與訓練紀錄
自動熱部署	KServe REST API / Helm Upgrade	滾動部署更新模型版本

☑ 判斷觸發條件建議

條件	判斷方法
PSI > 0.2	分類資料分布偏移
RMSE 上升超過 10%	預測值偏離真值
預測 confidence 平均 < 0.5	多分類信心不確定
異常輸入比例 > 5%	利用 Isolation Forest 或 AutoEncoder 偵測

 模型驗證通過後，可透過：

- kserve modelmesh 進行多版本併存與動態切換
- 利用 kubectl patch 或 KServe InferenceService 自動更新 endpoint

◆ (二) Prompt Injection 防禦部署 (Prompt Injection Defense for LLM APIs)

 目標

防止使用者透過惡意 prompt 對 LLM 發出逃逸指令 (如「忽略之前的規則...」)，導致模型產生不當內容或執行危險操作。

 防禦機制分類

類別	說明	實作工具
前處理防禦	解析並檢查 Prompt 是否含逃逸句型、反斜線逃脫等關鍵詞	PromptGuard, Rebuff, Regex Filter
類型分類器	使用文本分類模型辨識 Prompt 意圖 (如 Jailbreak / 正常 / 廣告)	OpenAI Classifier, LlamaGuard
LLM 防衛 Prompt 套件	為 prompt 增加 context 鎖定、強化語義限制	Chain of Trust, Constitutional AI
代理層控管	API Gateway 或 Agent 層使用指令白名單策略	LangChain Router, Secure Agents

 快速部署實作建議 (以 FastAPI + Guardrails.ai 為例)

```
from guardrails.hub import PromptGuard
@api.post("/chat")
def chat(user_input: str):
    guard = PromptGuard()
    if guard.is_malicious(user_input):
        return {"error": "Detected prompt injection"}
    result = llm_chain(user_input)
    return {"response": result}
```

 安全部署配套措施

- 設定 Token 限流與最大 context 長度 (防止 prompt overflow)
- 加入自動黑名單學習機制 (監控 prompt log)
- Prompt 版本鎖定 + 語境一致性驗證 (如 system_prompt_id)

◆ (三) 多語言 / 多地區多租戶管理架構 (Multilingual & Multi-region Multi-tenancy)

 目標

在同一個 AI 系統架構下，同時支援不同語言地區與租戶的模型隔離、路由管理與效能治理。

📁 架構設計維度

租戶隔離層 (Tenant Isolation)

層級	方法	工具
計算資源	Kubernetes Namespace / Node Affinity	每租戶獨立資源池
模型版本	Model Registry per tenant	/tenantA/modelX, /tenantB/modelX
資料儲存	Volume / DB 分離	PostgreSQL Schema / S3 Bucket 分隔
日誌分析	Loki / ELK per tenant index	可分帳記錄與權限稽核

多語言路由策略 (Language Routing)

模型組態	說明
載入不同語言特化模型	如 BERT-zh, BERT-en, GPT-fr
語言偵測自動分流	FastText, langdetect 先行判別語言後調用對應模型
URL or Header 指定語言	/predict/zh, Header: X-Language: fr

多地區部署 (Multi-region Federation)

技術	說明
K8s Federation v2 / Multi-Cluster Ingress	根據使用者 IP 進行就近推理服務
CDN 配合模型快取	圖片 / 音訊模型在區域節點快取加速
地區模型差異管理	透過 GitOps 於地區分支各自更新模型 YAML

🧰 延伸建議工具組

功能	建議工具
多語語言偵測	FastText, langid.py, Compact Language Detector
租戶認證	Auth0, Ory Kratos, Keycloak
跨地區同步	GitOps with FluxCD / ArgoCD
API 金鑰與流量控管	Kong + JWT plugin / Envoy RBAC

八、系統測試與驗證

(一) 測試核心

🔧 系統測試三核心：單元測試、整合測試、壓力測試

☑ 1. 單元測試 (Unit Test) : 模型輸入輸出正確性驗證

🎯 測試目的 :

針對模型函數、模組、API 邏輯等最小單元進行測試，確保其回傳結果正確、邏輯正確、能處理邊界狀況與錯誤輸入。

📌 適用範圍 :

- 模型推論函數 (predict, transform)
- 特徵處理 / 前處理模組
- 模型輸入 / 輸出格式
- Schema 驗證與異常處理

🧰 工具 :

- pytest (Python 主流測試框架)
- unittest (Python 標準模組)
- pydantic + mypy 做 schema 驗證輔助

📘 測試範例 : 驗證 NLP 分類模型的輸出結構

```
from model import predict
```

```
def test_predict_format():
```

```
    input_data = {"text": "This product is amazing!"}
```

```
    result = predict(input_data)
```

```
    assert isinstance(result, dict)
```

```
    assert "label" in result
```

```
    assert result["label"] in ["positive", "neutral", "negative"]
```

🔍 高階技巧 :

- 使用 pytest.parametrize 測多筆資料
- 針對極端值 (空字串、null) 與非預期格式進行例外測試

```
import pytest
```

```
@pytest.mark.parametrize("text", [ "", None, 123 ])
```

```
def test_predict_invalid_input(text):
```

```
    with pytest.raises(ValueError):
```

```
        predict({"text": text})
```

☑ 2. 整合測試 (Integration Test) : 跨模組 / API 接合測試

🎯 測試目的 :

驗證多個模組間 (如 API → 模型 → 資料庫) 的整體資料流與連接正確性，確保部署後的整體系統運作符合預期。

📌 測試對象 :

- API server + 模型推理服務
- 模型 + 特徵資料庫 (如 Redis / Feature Store)

- 跨容器系統 (Docker Compose)
- 模型上游 / 下游處理流程 (前處理 → 模型 → 後處理)

 工具：

- FastAPI TestClient, requests (API 模擬)
- Docker Compose + pytest-docker (模組整合環境)
- tox, pytest-postgresql, pytest-redis (環境模擬)

 FastAPI 服務整合測試範例：

```
from fastapi.testclient import TestClient
from app.main import app
client = TestClient(app)
def test_predict_api():
    response = client.post("/predict", json={"text": "great app!"})
    assert response.status_code == 200
    data = response.json()
    assert "label" in data
    assert isinstance(data["label"], str)
```

 延伸策略：

- 驗證跨模組傳遞參數是否一致 (如模型返回與前端預期格式)
- 可整合 Docker Compose 啟動 Redis + Model + API · 模擬實際部署

3. 壓力測試 (Load Test)：模擬高併發場景與系統壓力

 測試目的：

模擬真實世界下大量使用者同時存取模型 API 的情境，檢查系統在高負載下的反應速度、錯誤率與資源佔用。

 可評估項目：

- 最大 QPS (Queries per second)
- 延遲統計 (平均 / P95 / 最大)
- 錯誤率 (500 / 429 等)
- 資源使用率 (CPU / GPU / RAM)

 工具：

工具	優點	適合情境
Locust	使用 Python 腳本模擬使用者行為	模擬真實多場景
k6	JavaScript + CLI 控制，整合 Grafana	CI/CD 友善、自動化測試
JMeter	GUI 配置，支援多協定測試	傳統平台系統測試

(二) 模型驗證指標

◆ 1. 分類任務指標 (Classification Metrics)

☒ 準確率 (Accuracy)

- 代表意義：整體預測正確的比例。用來衡量模型有多大比例預測正確。
- 適用情境：任務中正負樣本數量相對平衡時 (例如：狗 vs 貓圖像分類)。
- 常見誤區：在資料不平衡的情況下容易誤導，例如：如果 99% 是正常樣本，模型都預測正常就有 99% 的準確率，但實際完全沒偵測異常。

☒ 召回率 (Recall)

- 代表意義：模型找出了多少「應該找出」的正樣本。
- 適用情境：重視「不能漏掉」的場景，例如：疾病診斷、詐騙偵測、工業異常識別。
- 特點：提高召回率會犧牲精確率 (多抓一些可能是錯的也沒關係，只要別漏抓真的就好)。

☒ F1-score

- 代表意義：召回率與精確率的綜合指標，當兩者都很重要時使用。
- 適用情境：資料不平衡的任務 (如：自然語言情緒判斷、法律判決分類)。
- 特點：是一種折衷，避免只看召回或只看精確導致誤判模型好壞。

☒ AUC (ROC 曲線下面積)

- 代表意義：綜合所有預測閾值下，模型將正例與負例區分開的能力。
- 適用情境：需要根據預測機率進行風險排序的場景 (如信用評分、廣告點擊預測)。
- 特點：不依賴單一閾值，評估模型整體表現穩定性，數值越接近 1 越好。

◆ 2. 回歸任務指標 (Regression Metrics)

☒ RMSE (均方根誤差)

- 代表意義：平均預測與實際值的誤差強調「大錯誤」的懲罰。
- 適用情境：例如：房價預測、風速預測，對預測精準度要求高的任務。
- 特點：對離群值敏感，適合希望模型避免出現大錯的場景。

☒ MAE (平均絕對誤差)

- 代表意義：每次預測與真實值誤差的平均，不放大大誤差。
- 適用情境：用於成本計算、資源預估等需要穩定預測誤差的任務。
- 特點：更穩健，不會被極端值大幅影響，解釋性強。

☒ R^2 (決定係數)

- 代表意義：衡量模型能解釋資料變異程度的比例。

- 適用情境：用來判斷回歸模型「整體好不好」，常見於氣象預測、銷售預測。
- 特點：越接近 1 越好，越接近 0 表示幾乎沒預測效果，負數表示表現比直接猜平均還差。

◆ 3. 推薦系統指標 (Recommendation Metrics)

☑ 精確率@K (Precision@K)

- 代表意義：推薦清單前 K 個中，有多少是使用者真正喜歡的項目。
- 適用情境：首頁推薦、搜尋結果、Top-N 商品推薦等場景。
- 特點：越高代表模型能準確挑出用戶感興趣的項目，特別重要於「首頁有限位置」的應用。

☑ NDCG@K (排序折扣累積增益)

- 代表意義：不只是看推薦對不對，還看「對的項目是否排在前面」。
- 適用情境：電子商務推薦、影音平台排序、搜尋結果呈現。
- 特點：能反映推薦項目在列表中排序的重要性，越靠前越有價值。

✳ 小結：指標選用建議

任務類型	重點關注	建議指標
分類 (均衡)	整體正確	Accuracy
分類 (不平衡)	錯誤不能漏	F1-score / Recall
風險排序	機率穩定性	AUC
回歸 (避免大誤差)	精準預測	RMSE
回歸 (解釋性)	絕對穩定	MAE, R ²
推薦 (是否點中)	命中率	Precision@K
推薦 (排序越前越好)	使用者體驗	NDCG@K

■ 模型評估指標應用組合方案設計

◆ (一) 分類任務 (Classification)

🎯 應用情境細分與組合策略

應用類型	資料特性	重點指標組合	說明
醫療診斷 / 疾病預測	正類極少	Recall、F1-score、AUC	不可漏判病人，召回優先，搭配 AUC 驗證穩定性
信用風控 / 詐欺偵測	資料極度不平衡	AUC、Precision、F1-score	降低誤判成本，控制 Precision 的同時確保 Recall
情感分析 / 用	類別均衡	Accuracy、F1-score	可使用綜合性指標，F1 作為分類穩

應用類型	資料特性	重點指標組合	說明
戶分類			定性補充
多類別產品分類	類別較多且均衡	Macro F1-score、Accuracy	評估每類表現，Macro F1 防止偏向大類別
客服工單分類 / NLU	少量類別偏斜	Weighted F1、Confusion Matrix	強化模型對小眾類別的表現監控

✿ 分類評估擴充策略：

- 閾值分析 (Threshold Tuning)：根據業務目標設定最佳分界，例如保留 95% recall 再微調 precision。
- 錯誤類型分析 (Error Case Breakdown)：使用 confusion matrix 對錯誤類別進行行為分析。

◆ (二) 回歸任務 (Regression)

🎯 應用情境細分與組合策略

應用類型	預測特性	指標組合	說明
房價預測 / 醫療花費預估	金額波動大、需準確預測	RMSE、MAE、 R^2	RMSE 反映大型錯誤，MAE 增強解釋力， R^2 為模型穩定性指標
銷售預測 / 成交率預測	精準數值預估 + 規模評估	MAE、 R^2	MAE 可對決策誤差進行實際解釋
電力 / 天氣 / 水位預測	高頻時間序列、極端值重要	RMSE、 R^2 、分群殘差分析	RMSE 適合異常預警， R^2 看總體趨勢是否能捕捉
廣告點擊率 (CTR)	預測實數 [0,1] 的機率	Log Loss、Brier Score	強調預測機率準確性 (非單點)

✿ 回歸評估擴充策略：

- 區間分析 (Error by Segment)：對不同價格區段、使用者族群進行分段 MAE 分析。
- 分佈偏誤偵測 (Residual Drift)：結合概念漂移分析觀察殘差分佈是否變化。

◆ (三) 推薦系統任務 (Recommendation)

🎯 應用情境細分與組合策略

應用類型	系統目標	指標組合	說明
電商首頁推薦	排序重要、精準命中商品	Precision@K、NDCG@K	需要命中熱門商品，排序位置對轉換率影響大
影音平台 / 新聞推薦	內容個人化為主	NDCG@K、MAP、Recall@K	排序與召回同時評估，NDCG 反映使用者是否點擊前排

應用類型	系統目標	指標組合	說明
搜尋補全推薦	預測輸入與最可能點擊	MRR (平均排名倒數)、Recall@K	使用者行為敏感，排序越前面越有影響力
廣告 / 內容推薦	精準導流，高營收轉換	CTR, Precision@K、Lift	要評估行銷 ROI，可結合 A/B 測試獲取轉換率提升幅度

✖ 推薦評估擴充策略：

- 個人化 vs 全球效能分析：比較用戶分群的精準率變化，如新用戶 / 老用戶。
- 推薦覆蓋率 (Coverage)：評估推薦系統是否偏好只推薦熱門內容，失去多樣性。